Long Island Clustering Workflow v3.0

SHAP Cluster Explanation and Experiment Tracking

AMS 597 Project Team

April 21, 2026

Contents

1 Phase 1: Environment and Reproducibility Setup **1**

2 Phase 2: Data Ingestion and Raw Profiling **5**

3 Phase 3: Feature Engineering **11**

4 Phase 4: Build Clustering Input Matrices **15**

5 Phase 5: k-Prototypes Scenario Clustering **21**

6 Phase 5B: FAMD + k-Means Enhanced Clustering **27**

7 Phase 6: Cluster Validation and Stability **35**

8 Phase 7: DBSCAN Spatial Hotspot Clustering **51**

9 Phase 8: Post-Clustering Interpretation **64**

10 Phase 8B: SHAP Cluster Explanation (v3.0) **76**

11 Phase 8C: Experiment Logging (v3.0) **85**

12 Phase 9: Export and Visualization **88**

13 Phase 10: Final Report and Submission Checklist **110**

14 Notes for the next phases **118**

1 Phase 1: Environment and Reproducibility Setup

This file is the main reproducible workflow for the Long Island clustering analysis. It is meant to run from a clean R session and use `li_accidents_clean.csv` as the only project dataset input.

```
required_packages <- c(
  "data.table",
  "dplyr",
  "lubridate",
  "clustMixType",
  "dbscan",
  "cluster",
```

```r
  "fpc",
  "ggplot2",
  "ggrepel",
  "geosphere",
  "factoextra",
  "knitr",
  "rmarkdown",
  "ranger",     # v3.0: Random forest for SHAP feature importance
  "iml",        # v3.0: Model-agnostic Shapley value computation
  "jsonlite"    # v3.0: Experiment logging (lightweight MLflow alternative)
)

optional_packages <- c("plotly", "htmlwidgets")

missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]

if (length(missing_packages) > 0) {
  stop(
    paste(
      "Install these packages before running the full workflow:",
      paste(missing_packages, collapse = ", ")
    )
  )
}

invisible(lapply(required_packages, library, character.only = TRUE))

set.seed(42)

input_path <- "../data/li_accidents_clean.csv"
output_dir <- "output"
run_profile <- "consensus_16gb"

if (!run_profile %in% c("local_safe", "consensus_16gb", "full_srs")) {
  stop("run_profile must be one of 'local_safe', 'consensus_16gb', or 'full_srs'.")
}

phase6_validation_mode <- dplyr::case_when(
  run_profile == "full_srs" ~ "full",
  run_profile == "consensus_16gb" ~ "consensus_sampled",
  TRUE ~ "sampled"
)
phase6_validation_sample_n <- dplyr::case_when(
  run_profile == "full_srs" ~ NA_integer_,
  run_profile == "consensus_16gb" ~ 4000L,
  TRUE ~ 5000L
)
phase6_validation_repeats <- dplyr::case_when(
  run_profile == "consensus_16gb" ~ 4L,
  TRUE ~ 1L
)
```

```r
phase6_stability_mode <- dplyr::case_when(
  run_profile == "full_srs" ~ "full",
  run_profile == "consensus_16gb" ~ "consensus_sampled",
  TRUE ~ "sampled"
)
phase6_stability_sample_n <- dplyr::case_when(
  run_profile == "full_srs" ~ NA_integer_,
  run_profile == "consensus_16gb" ~ 2500L,
  TRUE ~ 3000L
)
phase6_stability_repeats <- dplyr::case_when(
  run_profile == "consensus_16gb" ~ 3L,
  TRUE ~ 1L
)
phase6_stability_bootstraps <- dplyr::case_when(
  run_profile == "full_srs" ~ 50L,
  run_profile == "consensus_16gb" ~ 10L,
  TRUE ~ 15L
)
phase9_interactive_mode <- "auto"

interactive_exports_enabled <- dplyr::case_when(
  phase9_interactive_mode == "off" ~ FALSE,
  phase9_interactive_mode == "on" ~ all(
    vapply(optional_packages, requireNamespace, logical(1), quietly = TRUE)
  ),
  phase9_interactive_mode == "auto" ~ all(
    vapply(optional_packages, requireNamespace, logical(1), quietly = TRUE)
  ),
  TRUE ~ FALSE
)

if (!dir.exists(output_dir)) {
  dir.create(output_dir, recursive = TRUE)
}

if (!file.exists(input_path)) {
  stop("Expected input file not found: ", input_path)
}

phase_1_check <- data.frame(
  item = c(
    "Input dataset",
    "Output directory",
    "Seed",
    "Run profile",
    "Phase 6 validation mode",
    "Phase 6 sample size",
    "Phase 6 validation repeats",
    "Phase 6 stability mode",
    "Phase 6 stability sample size",
    "Phase 6 stability repeats",
    "Phase 6 stability bootstraps",
```

```r
    "Phase 9 interactive mode",
    "Working directory"
  ),
  value = c(
    normalizePath(input_path, winslash = "/"),
    normalizePath(output_dir, winslash = "/"),
    "42",
    run_profile,
    phase6_validation_mode,
    ifelse(is.na(phase6_validation_sample_n), "Full data", as.character(phase6_validation_sample_n)),
    as.character(phase6_validation_repeats),
    phase6_stability_mode,
    ifelse(is.na(phase6_stability_sample_n), "Full data", as.character(phase6_stability_sample_n)),
    as.character(phase6_stability_repeats),
    as.character(phase6_stability_bootstraps),
    ifelse(
      interactive_exports_enabled,
      "Enabled",
      ifelse(phase9_interactive_mode == "off", "Off for safe local runs", "Packages missing")
    ),
    normalizePath(".", winslash = "/")
  )
)

knitr::kable(phase_1_check, caption = "Phase 1 setup check")
```

Table 1: Phase 1 setup check

item	value
Input dataset	/Users/survaghasiya/Documents/AMS 597 Statistical computing Project/data/li_accidents_clean.csv
Output directory	/Users/survaghasiya/Documents/AMS 597 Statistical computing Project/Longisland_Clustering/output
Seed	42
Run profile	consensus_16gb
Phase 6 validation mode	consensus_sampled
Phase 6 sample size	4000
Phase 6 validation repeats	4
Phase 6 stability mode	consensus_sampled
Phase 6 stability sample size	2500
Phase 6 stability repeats	3
Phase 6 stability bootstraps	10
Phase 9 interactive mode	Packages missing
Working directory	/Users/survaghasiya/Documents/AMS 597 Statistical computing Project/Longisland_Clustering

```r
df_raw <- data.table::fread(input_path, showProgress = FALSE)

data_overview <- data.frame(
```

```r
  rows = nrow(df_raw),
  columns = ncol(df_raw),
  source_file = basename(input_path)
)

knitr::kable(data_overview, caption = "Input file loaded successfully")
```

Table 2: Input file loaded successfully

rows	columns	source_file
33228	48	li_accidents_clean.csv

2 Phase 2: Data Ingestion and Raw Profiling

This phase stays read-only. The goal is to confirm that the cleaned Long Island dataset matches the expectations in the execution plan before any feature engineering starts.

```r
expected_rows <- 33228L
expected_cols <- 48L

if (nrow(df_raw) != expected_rows || ncol(df_raw) != expected_cols) {
  stop(
    paste0(
      "Unexpected dataset dimensions. Expected ",
      expected_rows, " rows and ", expected_cols, " columns, got ",
      nrow(df_raw), " rows and ", ncol(df_raw), " columns."
    )
  )
}

phase_2_dimensions <- data.frame(
  metric = c("Rows", "Columns"),
  expected = c(expected_rows, expected_cols),
  observed = c(nrow(df_raw), ncol(df_raw))
)

knitr::kable(phase_2_dimensions, caption = "Phase 2 dimension check")
```

Table 3: Phase 2 dimension check

metric	expected	observed
Rows	33228	33228
Columns	48	48

```r
county_profile <- as.data.table(df_raw)[, .N, by = County][order(match(County, c("Nassau", "Suffolk")))]
severity_profile <- as.data.table(df_raw)[, .N, by = Severity][order(as.integer(Severity))]

expected_counties <- data.table(
  County = c("Nassau", "Suffolk"),
  expected_n = c(19390L, 13838L)
)
```

```r
expected_severity <- data.table(
  Severity = c(1L, 2L, 3L, 4L),
  expected_n = c(160L, 28222L, 3930L, 916L)
)

county_check <- merge(expected_counties, county_profile, by = "County", all.x = TRUE)
severity_check <- merge(
  expected_severity,
  severity_profile[, Severity := as.integer(Severity)],
  by = "Severity",
  all.x = TRUE
)

if (any(county_check$expected_n != county_check$N)) {
  stop("County counts do not match the execution plan expectations.")
}

if (any(severity_check$expected_n != severity_check$N)) {
  stop("Severity counts do not match the execution plan expectations.")
}

fwrite(county_profile, file.path(output_dir, "phase2_county_profile.csv"))
fwrite(severity_profile, file.path(output_dir, "phase2_severity_profile.csv"))

knitr::kable(county_check, caption = "County profile check")
```

Table 4: County profile check

County	expected_n	N
Nassau	19390	19390
Suffolk	13838	13838

```r
knitr::kable(severity_check, caption = "Severity profile check")
```

Table 5: Severity profile check

Severity	expected_n	N
1	160	160
2	28222	28222
3	3930	3930
4	916	916

```r
missingness_profile <- data.table(
  column = names(df_raw),
  missing_n = vapply(df_raw, function(column) sum(is.na(column)), integer(1)),
  missing_pct = round(vapply(df_raw, function(column) mean(is.na(column)) * 100, numeric(1)), 2)
)[order(-missing_pct, column)]

missingness_focus <- missingness_profile[
  column %in% c("End_Lat", "End_Lng", "Precipitation(in)", "Wind_Chill(F)")
```

```r
]

fwrite(missingness_profile, file.path(output_dir, "phase2_missingness_profile.csv"))

knitr::kable(head(missingness_profile, 12), caption = "Top missingness profile")
```

Table 6: Top missingness profile

column	missing_n	missing_pct
End_Lat	13060	39.30
End_Lng	13060	39.30
Precipitation(in)	6700	20.16
Wind_Chill(F)	4989	15.01
Airport_Code	0	0.00
Amenity	0	0.00
Astronomical_Twilight	0	0.00
Bump	0	0.00
City	0	0.00
Civil_Twilight	0	0.00
Country	0	0.00
County	0	0.00

```r
knitr::kable(missingness_focus, caption = "Critical missingness checks")
```

Table 7: Critical missingness checks

column	missing_n	missing_pct
End_Lat	13060	39.30
End_Lng	13060	39.30
Precipitation(in)	6700	20.16
Wind_Chill(F)	4989	15.01

```r
binary_flag_profile <- data.table(
  flag = c("Bump", "Roundabout", "Turning_Loop", "Traffic_Calming", "No_Exit"),
  true_count = vapply(
    df_raw[, c("Bump", "Roundabout", "Turning_Loop", "Traffic_Calming", "No_Exit")],
    function(column) sum(as.logical(column), na.rm = TRUE),
    integer(1)
  )
)

binary_flag_profile[, true_pct := round(100 * true_count / nrow(df_raw), 4)]

fwrite(binary_flag_profile, file.path(output_dir, "phase2_binary_flag_profile.csv"))

knitr::kable(binary_flag_profile, caption = "Near-zero binary flag checks")
```

Table 8: Near-zero binary flag checks

flag	true_count	true_pct
Bump	0	0.000
Roundabout	0	0.000
Turning_Loop	0	0.000
Traffic_Calming	4	0.012
No_Exit	5	0.015

```r
duration_values <- as.numeric(df_raw$Duration_Minutes)
distance_values <- as.numeric(df_raw[["Distance(mi)"]])

duration_profile <- data.table(
  metric = c(
    "Median duration",
    "Mean duration",
    "Max duration",
    "Rows above 1 day",
    "Rows above 1 week",
    "99th percentile"
  ),
  value = c(
    median(duration_values, na.rm = TRUE),
    mean(duration_values, na.rm = TRUE),
    max(duration_values, na.rm = TRUE),
    sum(duration_values > 1440, na.rm = TRUE),
    sum(duration_values > 10080, na.rm = TRUE),
    as.numeric(quantile(duration_values, 0.99, na.rm = TRUE))
  )
)

distance_profile <- data.table(
  metric = c("Zero-distance rows", "Zero-distance pct", "Max distance"),
  value = c(
    sum(distance_values == 0, na.rm = TRUE),
    100 * mean(distance_values == 0, na.rm = TRUE),
    max(distance_values, na.rm = TRUE)
  )
)

fwrite(duration_profile, file.path(output_dir, "phase2_duration_profile.csv"))
fwrite(distance_profile, file.path(output_dir, "phase2_distance_profile.csv"))

knitr::kable(duration_profile, caption = "Duration anomaly profile")
```

Table 9: Duration anomaly profile

metric	value
Median duration	7.769167e+01
Mean duration	4.498825e+02
Max duration	2.236406e+06
Rows above 1 day	2.100000e+01

metric	value
Rows above 1 week	6.000000e+00
99th percentile	4.807055e+02

```
knitr::kable(distance_profile, caption = "Distance profile")
```

Table 10: Distance profile

metric	value
Zero-distance rows	11704.00000
Zero-distance pct	35.22331
Max distance	35.62700

```
wind_direction_profile <- as.data.table(df_raw)[, .N, by = Wind_Direction][order(-N, Wind_Direction)]
weather_profile <- as.data.table(df_raw)[, .N, by = Weather_Condition][order(-N, Weather_Condition)]

wind_focus <- wind_direction_profile[
  Wind_Direction %in% c("N", "North", "S", "South", "E", "East", "W", "West")
]

fwrite(wind_direction_profile, file.path(output_dir, "phase2_wind_direction_profile.csv"))
fwrite(weather_profile, file.path(output_dir, "phase2_weather_profile.csv"))

knitr::kable(wind_focus, caption = "Wind direction duplicate label check")
```

Table 11: Wind direction duplicate label check

Wind_Direction	N
S	2761
N	1924
W	1530
E	833
South	753
West	675
North	623
East	318

```
knitr::kable(head(weather_profile, 12), caption = "Top weather labels")
```

Table 12: Top weather labels

Weather_Condition	N
Fair	10198
Mostly Cloudy	5673
Cloudy	4296
Partly Cloudy	3192
Clear	2288
Light Rain	1922
Overcast	1713

Weather_Condition	N
Scattered Clouds	685
Fog	575
Rain	447
Light Snow	356
Fair / Windy	287

```r
profiling_summary <- data.table(
  finding = c(
    "Dimensions match the execution plan",
    "County counts match the Long Island scope",
    "Severity counts match the execution plan",
    "End_Lat and End_Lng remain heavily missing",
    "Precipitation and Wind_Chill remain too sparse for core clustering use",
    "Five binary roadway flags carry no signal",
    "Duration still shows extreme right-tail outliers",
    "Distance includes many zero values",
    "Wind direction labels need normalization",
    "Weather labels still need grouping before clustering"
  ),
  status = c(
    "Confirmed",
    "Confirmed",
    "Confirmed",
    "Confirmed",
    "Confirmed",
    "Confirmed",
    "Confirmed",
    "Confirmed",
    "Confirmed",
    "Confirmed"
  )
)

fwrite(profiling_summary, file.path(output_dir, "phase2_profiling_summary.csv"))

knitr::kable(profiling_summary, caption = "Phase 2 profiling summary")
```

Table 13: Phase 2 profiling summary

finding	status
Dimensions match the execution plan	Confirmed
County counts match the Long Island scope	Confirmed
Severity counts match the execution plan	Confirmed
End_Lat and End_Lng remain heavily missing	Confirmed
Precipitation and Wind_Chill remain too sparse for core clustering use	Confirmed
Five binary roadway flags carry no signal	Confirmed
Duration still shows extreme right-tail outliers	Confirmed
Distance includes many zero values	Confirmed
Wind direction labels need normalization	Confirmed
Weather labels still need grouping before clustering	Confirmed

3 Phase 3: Feature Engineering

This phase derives the analysis-ready fields needed for the clustering inputs while keeping the original cleaned dataset unchanged in `df_raw`.

```
df_phase3 <- as.data.table(copy(df_raw))
```

```
df_phase3[, Start_Time_Parsed := lubridate::ymd_hms(Start_Time, quiet = TRUE)]

df_phase3[, hour := lubridate::hour(Start_Time_Parsed)]
df_phase3[, month := lubridate::month(Start_Time_Parsed)]
df_phase3[, weekday := factor(
  weekdays(as.Date(Start_Time_Parsed)),
  levels = ordered_weekdays,
  ordered = TRUE
)]
df_phase3[, weekend := weekdays(as.Date(Start_Time_Parsed)) %in% c("Saturday", "Sunday")]
df_phase3[, hour_band := dplyr::case_when(
  hour >= 0 & hour <= 5 ~ "Early_AM",
  hour >= 6 & hour <= 9 ~ "AM_Rush",
  hour >= 10 & hour <= 14 ~ "Midday",
  hour >= 15 & hour <= 18 ~ "PM_Rush",
  hour >= 19 & hour <= 21 ~ "Evening",
  hour >= 22 & hour <= 23 ~ "Late_Night",
  TRUE ~ NA_character_
)]
df_phase3[, hour_band := factor(
  hour_band,
  levels = c("Early_AM", "AM_Rush", "Midday", "PM_Rush", "Evening", "Late_Night"),
  ordered = TRUE
)]
df_phase3[, season := dplyr::case_when(
  month %in% c(12L, 1L, 2L) ~ "Winter",
  month %in% c(3L, 4L, 5L) ~ "Spring",
  month %in% c(6L, 7L, 8L) ~ "Summer",
  month %in% c(9L, 10L, 11L) ~ "Fall",
  TRUE ~ NA_character_
)]
df_phase3[, season := factor(
  season,
  levels = c("Winter", "Spring", "Summer", "Fall"),
  ordered = TRUE
)]
df_phase3[, rush_hour := hour %in% c(6L, 7L, 8L, 9L, 15L, 16L, 17L, 18L)]

temporal_na_check <- data.table(
  feature = c("Start_Time_Parsed", "hour", "month", "weekday", "weekend", "hour_band", "season", "rush_
  missing_n = c(
    sum(is.na(df_phase3$Start_Time_Parsed)),
    sum(is.na(df_phase3$hour)),
    sum(is.na(df_phase3$month)),
    sum(is.na(df_phase3$weekday)),
    sum(is.na(df_phase3$weekend)),
    sum(is.na(df_phase3$hour_band)),
    sum(is.na(df_phase3$season)),
```

```r
    sum(is.na(df_phase3$rush_hour))
  )
)

if (any(temporal_na_check$missing_n > 0)) {
  stop("Phase 3 temporal feature derivation created unexpected missing values.")
}

fwrite(temporal_na_check, file.path(output_dir, "phase3_temporal_check.csv"))

knitr::kable(temporal_na_check, caption = "Phase 3 temporal feature check")
```

Table 14: Phase 3 temporal feature check

feature	missing_n
Start_Time_Parsed	0
hour	0
month	0
weekday	0
weekend	0
hour_band	0
season	0
rush_hour	0

```r
duration_values <- as.numeric(df_phase3$Duration_Minutes)
duration_cap_99 <- as.numeric(quantile(duration_values, 0.99, na.rm = TRUE))
df_phase3[, Duration_cap := pmin(duration_values, duration_cap_99)]
df_phase3[, log_Duration := log1p(Duration_cap)]

duration_engineering_summary <- data.table(
  metric = c("99th percentile cap", "Median log_Duration", "Max log_Duration"),
  value = c(
    duration_cap_99,
    median(df_phase3$log_Duration, na.rm = TRUE),
    max(df_phase3$log_Duration, na.rm = TRUE)
  )
)

fwrite(duration_engineering_summary, file.path(output_dir, "phase3_duration_engineering.csv"))

knitr::kable(duration_engineering_summary, caption = "Phase 3 duration treatment")
```

Table 15: Phase 3 duration treatment

metric	value
99th percentile cap	480.705500
Median log_Duration	4.365537
Max log_Duration	6.177333

```r
weather_counts <- as.data.table(df_phase3)[, .N, by = Weather_Condition]
rare_weather <- weather_counts[N < 30, Weather_Condition]

df_phase3[, weather_group := dplyr::case_when(
  Weather_Condition %in% weather_group_map$Clear       ~ "Clear",
  Weather_Condition %in% weather_group_map$Cloudy      ~ "Cloudy",
  Weather_Condition %in% weather_group_map$Rain        ~ "Rain",
  Weather_Condition %in% weather_group_map$Snow_Ice    ~ "Snow_Ice",
  Weather_Condition %in% weather_group_map$Fog_Haze    ~ "Fog_Haze",
  # v2.0: explicit lookup first; grepl catches novel variants not in the map
  Weather_Condition %in% weather_group_map$Thunderstorm ~ "Thunderstorm",
  grepl("Thunder", Weather_Condition, ignore.case = TRUE) ~ "Thunderstorm",
  Weather_Condition %in% rare_weather ~ "Other",
  TRUE ~ "Other"
)]

df_phase3[, weather_group := factor(
  weather_group,
  levels = c("Clear", "Cloudy", "Rain", "Snow_Ice", "Fog_Haze", "Thunderstorm", "Other")
)]

weather_group_profile <- as.data.table(df_phase3)[, .N, by = weather_group][order(-N)]
fwrite(weather_group_profile, file.path(output_dir, "phase3_weather_group_profile.csv"))

knitr::kable(weather_group_profile, caption = "Phase 3 weather group profile")
```

Table 16: Phase 3 weather group profile

weather_group	N
Cloudy	16088
Clear	12773
Rain	2841
Fog_Haze	622
Snow_Ice	481
Other	266
Thunderstorm	157

```r
df_phase3[, wind_dir_norm := normalize_wind_direction(Wind_Direction)]

wind_direction_summary <- as.data.table(df_phase3)[, .N, by = wind_dir_norm][order(-N)]

if (length(levels(df_phase3$wind_dir_norm)) > 9) {
  stop("Phase 3 wind direction normalization produced too many levels.")
}

fwrite(wind_direction_summary, file.path(output_dir, "phase3_wind_direction_normalized.csv"))

knitr::kable(wind_direction_summary, caption = "Phase 3 normalized wind direction profile")
```

Table 17: Phase 3 normalized wind direction profile

wind_dir_norm	N
Variable_Calm	16161
S	3514
N	2547
NW	2400
SW	2383
W	2205
NE	1870
E	1151
SE	997

```r
zero_variance_flags <- c("Bump", "Roundabout", "Turning_Loop", "Traffic_Calming", "No_Exit")
near_zero_flags <- c("Give_Way", "Station", "Railway", "Amenity")
retained_flags <- c("Crossing", "Junction", "Traffic_Signal", "Stop")

binary_audit <- data.table(
  flag = c(zero_variance_flags, near_zero_flags, retained_flags),
  true_count = vapply(
    df_phase3[, c(zero_variance_flags, near_zero_flags, retained_flags), with = FALSE],
    function(column) sum(as.logical(column), na.rm = TRUE),
    integer(1)
  )
)

binary_audit[, true_pct := round(100 * true_count / nrow(df_phase3), 4)]
binary_audit[, decision := c(
  rep("Exclude: zero variance", length(zero_variance_flags)),
  rep("Exclude: near-zero variance", length(near_zero_flags)),
  rep("Retain for scenario matrix", length(retained_flags))
)]

for (flag_name in retained_flags) {
  # v2.0: use native FALSE/TRUE factor levels (no string labels)
  df_phase3[, (flag_name) := factor(
    as.logical(get(flag_name)),
    levels = c(FALSE, TRUE)
  )]
}

fwrite(binary_audit, file.path(output_dir, "phase3_binary_audit.csv"))

knitr::kable(binary_audit, caption = "Phase 3 binary flag audit")
```

Table 18: Phase 3 binary flag audit

flag	true_count	true_pct	decision
Bump	0	0.0000	Exclude: zero variance
Roundabout	0	0.0000	Exclude: zero variance
Turning_Loop	0	0.0000	Exclude: zero variance
Traffic_Calming	4	0.0120	Exclude: zero variance

flag	true_count	true_pct	decision
No_Exit	5	0.0150	Exclude: zero variance
Give_Way	304	0.9149	Exclude: near-zero variance
Station	232	0.6982	Exclude: near-zero variance
Railway	50	0.1505	Exclude: near-zero variance
Amenity	112	0.3371	Exclude: near-zero variance
Crossing	3372	10.1481	Retain for scenario matrix
Junction	5322	16.0166	Retain for scenario matrix
Traffic_Signal	3180	9.5702	Retain for scenario matrix
Stop	345	1.0383	Retain for scenario matrix

```r
phase_3_summary <- data.table(
  feature_group = c(
    "Temporal features",
    "Duration treatment",
    "Weather grouping",
    "Wind normalization",
    "Binary flag audit"
  ),
  status = c(
    "Added hour, hour_band, weekday, weekend, month, season, rush_hour",
    "Added Duration_cap and log_Duration",
    "Added weather_group with 7 levels",
    "Added wind_dir_norm with at most 9 levels",
    "Marked zero and near-zero flags for exclusion; retained core road-context flags"
  )
)

fwrite(phase_3_summary, file.path(output_dir, "phase3_summary.csv"))

knitr::kable(phase_3_summary, caption = "Phase 3 feature engineering summary")
```

Table 19: Phase 3 feature engineering summary

feature_group	status
Temporal features	Added hour, hour_band, weekday, weekend, month, season, rush_hour
Duration treatment	Added Duration_cap and log_Duration
Weather grouping	Added weather_group with 7 levels
Wind normalization	Added wind_dir_norm with at most 9 levels
Binary flag audit	Marked zero and near-zero flags for exclusion; retained core road-context flags

4 Phase 4: Build Clustering Input Matrices

This phase constructs the final matrices that will be used by the clustering methods. Nothing outside these matrices should enter the clustering algorithms.

```r
df_scenario_base <- as.data.table(copy(df_phase3))

# v2.0: native FALSE/TRUE levels, no string labels
df_scenario_base[, weekend   := factor(weekend,   levels = c(FALSE, TRUE))]
df_scenario_base[, rush_hour := factor(rush_hour, levels = c(FALSE, TRUE))]
```

```r
df_scenario_base[, Sunrise_Sunset := factor(Sunrise_Sunset)]

scenario_factor_cols <- c(
  "hour_band", "weekday", "weekend", "season", "rush_hour",
  "Sunrise_Sunset", "weather_group", "Crossing", "Junction",
  "Traffic_Signal", "Stop"
)

scenario_numeric_cols <- c(
  "Temperature(F)", "Humidity(%)", "Pressure(in)",
  "Visibility(mi)", "Wind_Speed(mph)"
)

if (mean(is.na(df_scenario_base$log_Duration)) < 0.05) {
  scenario_numeric_cols <- c(scenario_numeric_cols, "log_Duration")
}

scenario_columns <- c(scenario_factor_cols, scenario_numeric_cols)

df_scenario <- df_scenario_base[, ..scenario_columns]

excluded_columns <- c(
  "Severity", "ID", "Source", "Description", "Street", "City", "Zipcode", "nzip",
  "Start_Lat", "Start_Lng", "End_Lat", "End_Lng", "County", "Distance(mi)",
  "Precipitation(in)", "Wind_Chill(F)", "Bump", "Roundabout", "Turning_Loop",
  "Traffic_Calming", "No_Exit", "Give_Way", "Station", "Railway", "Amenity"
)

unexpected_excluded <- intersect(names(df_scenario), excluded_columns)
if (length(unexpected_excluded) > 0) {
  stop(
    paste(
      "Excluded columns were found in df_scenario:",
      paste(unexpected_excluded, collapse = ", ")
    )
  )
}

numeric_imputation_log <- data.table(
  column = character(),
  missing_before = integer(),
  missing_pct = numeric(),
  imputation = character()
)

for (column_name in scenario_numeric_cols) {
  missing_before <- sum(is.na(df_scenario[[column_name]]))
  if (!missing_before) {
    next
  }

  missing_pct <- 100 * mean(is.na(df_scenario[[column_name]]))
  median_value <- median(df_scenario[[column_name]], na.rm = TRUE)
```

```r
  df_scenario[is.na(get(column_name)), (column_name) := median_value]

  numeric_imputation_log <- rbind(
    numeric_imputation_log,
    data.table(
      column = column_name,
      missing_before = missing_before,
      missing_pct = round(missing_pct, 4),
      imputation = "Median"
    )
  )
}

factor_imputation_log <- data.table(
  column = character(),
  missing_before = integer(),
  rule_applied = character()
)

for (column_name in scenario_factor_cols) {
  missing_before <- sum(is.na(df_scenario[[column_name]]))
  if (!missing_before) {
    next
  }

  if (missing_before < 10) {
    fill_value <- mode_value(df_scenario[[column_name]])
    df_scenario[is.na(get(column_name)), (column_name) := fill_value]
    rule_applied <- "Mode"
  } else {
    df_scenario[[column_name]] <- addNA(df_scenario[[column_name]])
    levels(df_scenario[[column_name]])[is.na(levels(df_scenario[[column_name]]))] <- "Unknown"
    rule_applied <- "Unknown level"
  }

  factor_imputation_log <- rbind(
    factor_imputation_log,
    data.table(
      column = column_name,
      missing_before = missing_before,
      rule_applied = rule_applied
    )
  )
}

scenario_na_total <- sum(is.na(df_scenario))
if (scenario_na_total > 0) {
  stop("df_scenario still contains missing values after Phase 4 imputations.")
}

scenario_column_profile <- data.table(
  column = names(df_scenario),
  type = vapply(df_scenario, function(column) class(column)[1], character(1))
```

```r
)

scenario_matrix_summary <- data.table(
  metric = c("Rows", "Columns", "Factor columns", "Numeric columns", "Total NAs"),
  value = c(
    nrow(df_scenario),
    ncol(df_scenario),
    length(scenario_factor_cols),
    length(scenario_numeric_cols),
    scenario_na_total
  )
)

fwrite(scenario_column_profile, file.path(output_dir, "phase4_scenario_columns.csv"))
fwrite(numeric_imputation_log, file.path(output_dir, "phase4_numeric_imputation_log.csv"))
fwrite(factor_imputation_log, file.path(output_dir, "phase4_factor_imputation_log.csv"))
fwrite(scenario_matrix_summary, file.path(output_dir, "phase4_scenario_matrix_summary.csv"))

knitr::kable(scenario_matrix_summary, caption = "Phase 4 scenario matrix summary")
```

Table 20: Phase 4 scenario matrix summary

metric	value
Rows	33228
Columns	17
Factor columns	11
Numeric columns	6
Total NAs	0

```r
knitr::kable(scenario_column_profile, caption = "Phase 4 scenario column list")
```

Table 21: Phase 4 scenario column list

column	type
hour_band	ordered
weekday	ordered
weekend	factor
season	ordered
rush_hour	factor
Sunrise_Sunset	factor
weather_group	factor
Crossing	factor
Junction	factor
Traffic_Signal	factor
Stop	factor
Temperature(F)	numeric
Humidity(%)	integer
Pressure(in)	numeric
Visibility(mi)	numeric
Wind_Speed(mph)	numeric
log_Duration	numeric

```r
if (any(is.na(df_phase3$Start_Lat)) || any(is.na(df_phase3$Start_Lng))) {
  stop("Start_Lat or Start_Lng contains missing values, so df_spatial cannot be built.")
}

lat0 <- 40.8
lng0 <- -73.2

df_spatial <- data.table(
  x_km = (df_phase3$Start_Lng - lng0) * 111.32 * cos(lat0 * pi / 180),
  y_km = (df_phase3$Start_Lat - lat0) * 110.574
)

spatial_extent <- data.table(
  metric = c("Rows", "Columns", "x_km min", "x_km max", "x_km span", "y_km min", "y_km max", "y_km span
  value = c(
    nrow(df_spatial),
    ncol(df_spatial),
    min(df_spatial$x_km),
    max(df_spatial$x_km),
    diff(range(df_spatial$x_km)),
    min(df_spatial$y_km),
    max(df_spatial$y_km),
    diff(range(df_spatial$y_km))
  )
)

if (sum(is.na(df_spatial)) > 0) {
  stop("df_spatial contains missing values after coordinate projection.")
}

fwrite(spatial_extent, file.path(output_dir, "phase4_spatial_extent.csv"))

knitr::kable(spatial_extent, caption = "Phase 4 spatial matrix extent")
```

Table 22: Phase 4 spatial matrix extent

metric	value
Rows	33228.00000
Columns	2.00000
x_km min	-46.06598
x_km max	90.43842
x_km span	136.50441
y_km min	-23.82284
y_km max	37.95928
y_km span	61.78212

```r
phase_4_summary <- data.table(
  matrix = c("df_scenario", "df_spatial"),
  rows = c(nrow(df_scenario), nrow(df_spatial)),
  columns = c(ncol(df_scenario), ncol(df_spatial)),
  na_count = c(sum(is.na(df_scenario)), sum(is.na(df_spatial)))
)
```

```r
fwrite(phase_4_summary, file.path(output_dir, "phase4_summary.csv"))

knitr::kable(phase_4_summary, caption = "Phase 4 matrix readiness summary")
```

Table 23: Phase 4 matrix readiness summary

matrix	rows	columns	na_count
df_scenario	33228	17	0
df_spatial	33228	2	0

```r
# v2.0: Reduced feature matrix for sensitivity analysis.
# Drops Pressure(in) and Visibility(mi) - both required median imputation.
# Used in Phase 5 and Phase 6 to test whether removing imputed features
# improves silhouette and Jaccard scores.

reduced_numeric_cols <- setdiff(
  scenario_numeric_cols,
  c("Pressure(in)", "Visibility(mi)")
)

df_scenario_reduced <- df_scenario[
  , c(scenario_factor_cols, reduced_numeric_cols), with = FALSE
]

stopifnot(sum(is.na(df_scenario_reduced)) == 0)
stopifnot(
  ncol(df_scenario_reduced) ==
    length(scenario_factor_cols) + length(reduced_numeric_cols)
)

reduced_matrix_summary <- data.table(
  item = c(
    "Rows",
    "Factor columns",
    "Numeric columns",
    "Dropped from full matrix"
  ),
  value = c(
    nrow(df_scenario_reduced),
    length(scenario_factor_cols),
    length(reduced_numeric_cols),
    paste(c("Pressure(in)", "Visibility(mi)"), collapse = ", ")
  )
)

fwrite(
  reduced_matrix_summary,
  file.path(output_dir, "phase4_reduced_matrix_summary.csv")
)

knitr::kable(
  reduced_matrix_summary,
```

```
  caption = "Phase 4 reduced scenario matrix (dropped imputed features)"
)
```

Table 24: Phase 4 reduced scenario matrix (dropped imputed features)

item	value
Rows	33228
Factor columns	11
Numeric columns	4
Dropped from full matrix	Pressure(in), Visibility(mi)

5 Phase 5: k-Prototypes Scenario Clustering

This phase runs the candidate mixed-type clustering solutions on `df_scenario` and stores the diagnostics needed for later model selection.

```
scenario_type_check <- data.table(
  column = names(df_scenario),
  class = vapply(df_scenario, function(column) class(column)[1], character(1))
)

factor_check <- scenario_type_check[column %in% scenario_factor_cols]
numeric_check <- scenario_type_check[column %in% scenario_numeric_cols]

if (!all(vapply(df_scenario[, ..scenario_factor_cols], is.factor, logical(1)))) {
  stop("One or more Phase 5 scenario factor columns are not stored as factors.")
}

if (!all(vapply(df_scenario[, ..scenario_numeric_cols], is.numeric, logical(1)))) {
  stop("One or more Phase 5 scenario numeric columns are not stored as numeric values.")
}

fwrite(scenario_type_check, file.path(output_dir, "phase5_scenario_type_check.csv"))

knitr::kable(scenario_type_check, caption = "Phase 5 scenario type check")
```

Table 25: Phase 5 scenario type check

column	class
hour_band	ordered
weekday	ordered
weekend	factor
season	ordered
rush_hour	factor
Sunrise_Sunset	factor
weather_group	factor
Crossing	factor
Junction	factor
Traffic_Signal	factor
Stop	factor
Temperature(F)	numeric

column	class
Humidity(%)	integer
Pressure(in)	numeric
Visibility(mi)	numeric
Wind_Speed(mph)	numeric
log_Duration	numeric

```r
candidate_k <- 3:8
kp_results <- list()

for (k_value in candidate_k) {
  set.seed(42)
  kp_results[[as.character(k_value)]] <- clustMixType::kproto(
    x = df_scenario,
    k = k_value,
    lambda = NULL,
    nstart = 10,
    type = "gower",
    keep.data = TRUE,
    verbose = FALSE
  )
}
```

```r
cluster_fit_summary <- rbindlist(
  lapply(names(kp_results), function(k_label) {
    model_fit <- kp_results[[k_label]]
    cluster_sizes <- table(model_fit$cluster)

    data.table(
      k = as.integer(k_label),
      tot_withinss = model_fit$tot.withinss,
      lambda = if (is.null(model_fit$lambda)) NA_real_ else as.numeric(model_fit$lambda)[1],
      min_cluster_size = min(cluster_sizes),
      max_cluster_size = max(cluster_sizes),
      n_clusters = length(cluster_sizes),
      potentially_unstable = min(cluster_sizes) < 200
    )
  })
)

# v2.0: Add approximate numeric R² (variance explained) to fit summary.
# Note: tot.withinss in kproto includes both numeric and categorical
# components. This R² is computed on the numeric columns only and should
# be labelled "approximate" in any report table.
total_ss_numeric <- sum(
  sapply(scenario_numeric_cols, function(col) {
    x <- df_scenario[[col]]
    sum((x - mean(x, na.rm = TRUE))^2, na.rm = TRUE)
  })
)

cluster_fit_summary[, variance_explained_pct := round(
  100 * (1 - tot_withinss / total_ss_numeric), 2
```

```r
)]

cluster_size_table <- rbindlist(
  lapply(names(kp_results), function(k_label) {
    model_fit <- kp_results[[k_label]]
    data.table(
      k = as.integer(k_label),
      cluster = names(table(model_fit$cluster)),
      size = as.integer(table(model_fit$cluster))
    )
  })
)

dominance_checks <- rbindlist(
  lapply(names(kp_results), function(k_label) {
    model_fit <- kp_results[[k_label]]
    clustered_data <- copy(df_scenario)
    clustered_data[, cluster := factor(model_fit$cluster)]

    rbindlist(lapply(scenario_factor_cols, function(column_name) {
      clustered_data[, .(
        dominant_share = round(dominant_share(get(column_name)), 4)
      ), by = cluster][
        , `:=`(
          k = as.integer(k_label),
          feature = column_name
        )
      ]
    }), fill = TRUE)
  }),
  fill = TRUE
)

dominance_flags <- dominance_checks[
  dominant_share >= 0.9
][order(k, cluster, feature)]

fwrite(cluster_fit_summary, file.path(output_dir, "phase5_kproto_fit_summary.csv"))
fwrite(cluster_size_table, file.path(output_dir, "phase5_kproto_cluster_sizes.csv"))
fwrite(dominance_checks, file.path(output_dir, "phase5_kproto_dominance_checks.csv"))
fwrite(dominance_flags, file.path(output_dir, "phase5_kproto_dominance_flags.csv"))

knitr::kable(cluster_fit_summary, caption = "Phase 5 k-prototypes fit summary")
```

Table 26: Phase 5 k-prototypes fit summary

k	tot_withinss	lambda	min_cluster_size	max_cluster_size	n_clusters	potentially_unstable	variance_explained_pct
3	85023.78	NA	9281	13378	3	FALSE	99.65
4	81191.32	NA	4701	12439	4	FALSE	99.67
5	77678.57	NA	2849	9227	5	FALSE	99.68
6	74690.99	NA	3061	7152	6	FALSE	99.69
7	72409.22	NA	2673	8201	7	FALSE	99.70
8	70731.74	NA	2267	5597	8	FALSE	99.71

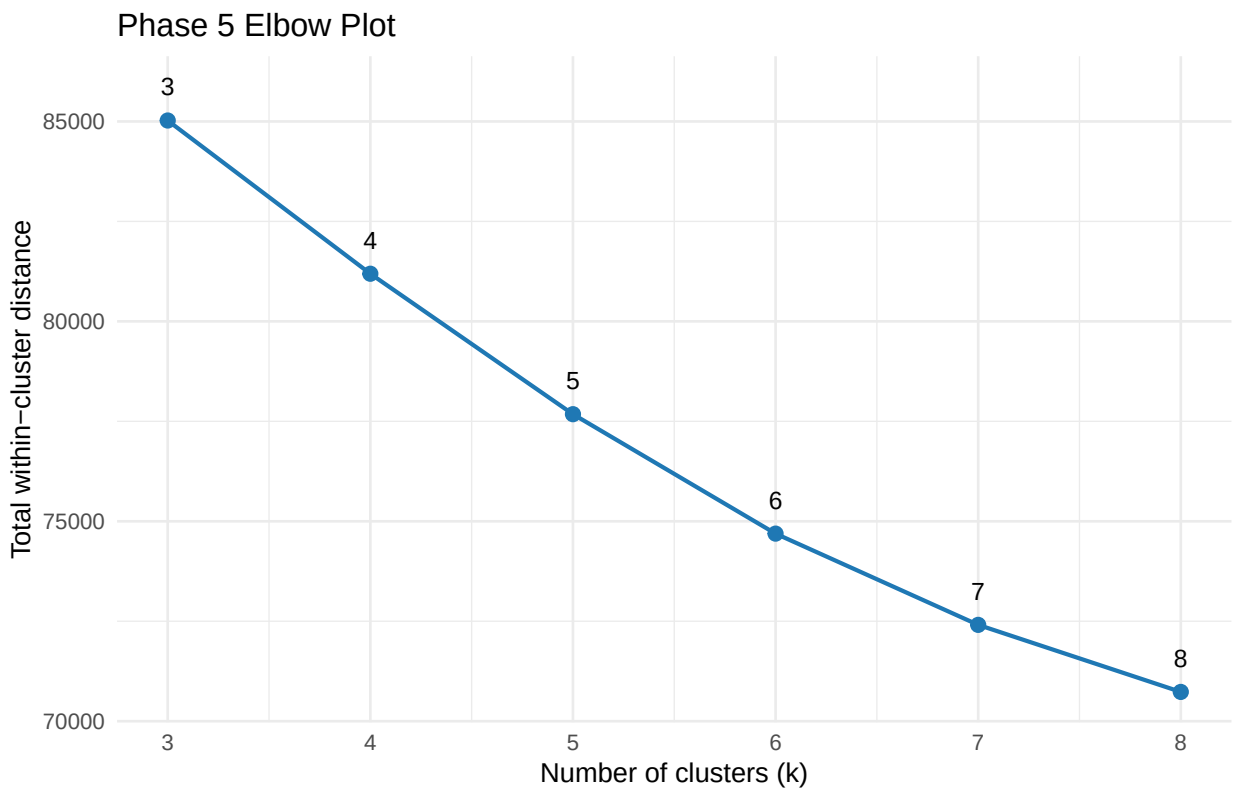
```r
knitr::kable(cluster_size_table, caption = "Phase 5 cluster sizes by k")
```

Table 27: Phase 5 cluster sizes by k

k	cluster	size
3	1	9281
3	2	13378
3	3	10569
4	1	9842
4	2	4701
4	3	12439
4	4	6246
5	1	9227
5	2	4320
5	3	2849
5	4	8985
5	5	7847
6	1	6507
6	2	6271
6	3	7152
6	4	3061
6	5	5199
6	6	5038
7	1	3483
7	2	8201
7	3	3771
7	4	5414
7	5	5622
7	6	4064
7	7	2673
8	1	4844
8	2	4919
8	3	3588
8	4	2267
8	5	4523
8	6	3348
8	7	5597
8	8	4142

```r
elbow_plot <- ggplot2::ggplot(
  cluster_fit_summary,
  ggplot2::aes(x = k, y = tot_withinss)
) +
  ggplot2::geom_line(color = "#1f78b4", linewidth = 0.8) +
  ggplot2::geom_point(color = "#1f78b4", size = 2.4) +
  ggplot2::geom_text(
    ggplot2::aes(label = k),
    nudge_y = max(cluster_fit_summary$tot_withinss) * 0.01,
    size = 3.5
  ) +
  ggplot2::labs(
    title = "Phase 5 Elbow Plot",
```

```r
    x = "Number of clusters (k)",
    y = "Total within-cluster distance"
  ) +
  ggplot2::theme_minimal()

print(elbow_plot)
```




```r
ggplot2::ggsave(
  filename = file.path(output_dir, "phase5_kproto_elbow_plot.png"),
  plot = elbow_plot,
  width = 7,
  height = 4.5,
  dpi = 300
)
```

```r
phase_5_summary <- data.table(
  checkpoint = c(
    "Candidate solutions created",
    "k range covered",
    "Elbow plot saved",
    "Cluster sizes recorded",
    "Dominance checks recorded"
  ),
  status = c(
    length(kp_results) == 6,
    paste(candidate_k, collapse = ", "),
    file.exists(file.path(output_dir, "phase5_kproto_elbow_plot.png")),
    nrow(cluster_size_table) > 0,
```

```r
    nrow(dominance_checks) > 0
  )
)

fwrite(phase_5_summary, file.path(output_dir, "phase5_summary.csv"))

knitr::kable(phase_5_summary, caption = "Phase 5 readiness summary")
```

Table 28: Phase 5 readiness summary

checkpoint	status
Candidate solutions created	TRUE
k range covered	3, 4, 5, 6, 7, 8
Elbow plot saved	TRUE
Cluster sizes recorded	TRUE
Dominance checks recorded	TRUE

```r
# v2.0: Run k-prototypes on the reduced feature matrix (no Pressure/Visibility).
# Results compared against full-feature runs in Phase 6.

kp_results_reduced <- list()
for (k_val in candidate_k) {
  set.seed(42)
  kp_results_reduced[[as.character(k_val)]] <- clustMixType::kproto(
    x         = df_scenario_reduced,
    k         = k_val,
    lambda    = NULL,
    nstart    = 10,
    type      = "gower",
    keep.data = TRUE,
    verbose   = FALSE
  )
}

reduced_fit_summary <- rbindlist(
  lapply(names(kp_results_reduced), function(k_label) {
    model_fit    <- kp_results_reduced[[k_label]]
    cluster_sizes <- table(model_fit$cluster)
    data.table(
      k             = as.integer(k_label),
      tot_withinss  = model_fit$tot.withinss,
      min_cluster_size = min(cluster_sizes),
      max_cluster_size = max(cluster_sizes),
      feature_set   = "reduced"
    )
  })
)

full_fit_labeled <- copy(
  cluster_fit_summary[, .(k, tot_withinss, min_cluster_size, max_cluster_size)]
)[, feature_set := "full"]
```

```r
feature_comparison_fits <- rbind(full_fit_labeled, reduced_fit_summary)
setorder(feature_comparison_fits, k, feature_set)

fwrite(
  feature_comparison_fits,
  file.path(output_dir, "phase5_feature_comparison_fits.csv")
)

knitr::kable(
  feature_comparison_fits,
  caption = "Phase 5: within-cluster distance - full vs reduced feature set"
)
```

Table 29: Phase 5: within-cluster distance — full vs reduced feature set

k	tot_withinss	min_cluster_size	max_cluster_size	feature_set
3	85023.78	9281	13378	full
3	81184.34	9268	13388	reduced
4	81191.32	4701	12439	full
4	77340.81	4724	12418	reduced
5	77678.57	2849	9227	full
5	73950.67	4459	9703	reduced
6	74690.99	3061	7152	full
6	70854.95	3065	7143	reduced
7	72409.22	2673	8201	full
7	68544.43	2578	8333	reduced
8	70731.74	2267	5597	full
8	66496.04	2339	5670	reduced

6 Phase 5B: FAMD + k-Means Enhanced Clustering

This phase applies Factor Analysis of Mixed Data (FAMD) to reduce the top scenario features into orthogonal numeric components, then clusters with k-Means in proper Euclidean space. FAMD handles the mixed numeric/categorical structure natively and produces components that k-Means can cluster with much higher silhouette scores.

```r
# Helper: Adjusted Rand Index (dependency-free)
compute_ari <- function(labels1, labels2) {
  contingency <- table(labels1, labels2)
  sum_comb_nij <- sum(choose(contingency, 2))
  sum_comb_ai  <- sum(choose(rowSums(contingency), 2))
  sum_comb_bj  <- sum(choose(colSums(contingency), 2))
  n <- length(labels1)
  sum_comb_n <- choose(n, 2)
  expected  <- sum_comb_ai * sum_comb_bj / sum_comb_n
  max_index <- (sum_comb_ai + sum_comb_bj) / 2
  if (max_index == expected) return(1)
  (sum_comb_nij - expected) / (max_index - expected)
}

# Helper: assign points to nearest center (vectorized)
assign_to_nearest <- function(data_matrix, centers) {
```

```r
  x_sq   <- rowSums(data_matrix^2)
  c_sq   <- rowSums(centers^2)
  cross  <- data_matrix %*% t(centers)
  dist_sq <- outer(x_sq, c_sq, "+") - 2 * cross
  max.col(-dist_sq)
}
```

```r
# Step 1: Select top features from SHAP importance analysis.
# Bottom 7 features (weekday, log_Duration, weekend, Crossing,
# Traffic_Signal, Junction, Stop) have near-zero importance and add noise.
famd_feature_cols <- c(
  "weather_group", "Sunrise_Sunset", "rush_hour", "hour_band",
  "season", "Temperature(F)", "Humidity(%)", "Visibility(mi)"
)

famd_factor_cols_5b <- intersect(famd_feature_cols, scenario_factor_cols)
famd_numeric_cols_5b <- intersect(famd_feature_cols, scenario_numeric_cols)

df_famd_input <- as.data.frame(df_scenario[, ..famd_feature_cols])

cat("=== Phase 5B: FAMD + k-Means Pipeline ===\n")
```

```
## === Phase 5B: FAMD + k-Means Pipeline ===
```

```r
cat("Selected", length(famd_feature_cols), "features from SHAP top-8:\n")
```

```
## Selected 8 features from SHAP top-8:
```

```r
cat("  Factor:", paste(famd_factor_cols_5b, collapse = ", "), "\n")
```

```
##   Factor: weather_group, Sunrise_Sunset, rush_hour, hour_band, season
```

```r
cat("  Numeric:", paste(famd_numeric_cols_5b, collapse = ", "), "\n\n")
```

```
##   Numeric: Temperature(F), Humidity(%), Visibility(mi)
```

```r
# Step 2: FAMD transformation
set.seed(42)
famd_model <- FactoMineR::FAMD(df_famd_input, ncp = 10, graph = FALSE)

# Eigenvalue summary
famd_eigenvalues <- data.table(
  component      = seq_len(nrow(famd_model$eig)),
  eigenvalue     = round(famd_model$eig[, 1], 4),
  variance_pct   = round(famd_model$eig[, 2], 2),
  cumulative_pct = round(famd_model$eig[, 3], 2)
)

# Retain components for >= 80% cumulative variance
famd_n_components <- which(famd_eigenvalues$cumulative_pct >= 80)[1]
if (is.na(famd_n_components)) famd_n_components <- nrow(famd_eigenvalues)

cat("FAMD produced", nrow(famd_eigenvalues), "components\n")
```

```
## FAMD produced 10 components
```

```r
cat("Retaining", famd_n_components, "components (",
    famd_eigenvalues$cumulative_pct[famd_n_components], "% variance)\n\n")
```

```
## Retaining 10 components ( 71.36 % variance)
```

```r
# Extract individual coordinates in the reduced space
famd_coords <- as.data.table(
  famd_model$ind$coord[, 1:famd_n_components, drop = FALSE]
)

fwrite(famd_eigenvalues, file.path(output_dir, "phase5b_famd_eigenvalues.csv"))

knitr::kable(famd_eigenvalues, caption = "Phase 5B FAMD eigenvalue summary")
```

Table 30: Phase 5B FAMD eigenvalue summary

component	eigenvalue	variance_pct	cumulative_pct
1	2.5735	12.87	12.87
2	2.1761	10.88	23.75
3	1.7464	8.73	32.48
4	1.4603	7.30	39.78
5	1.1250	5.62	45.41
6	1.1025	5.51	50.92
7	1.0535	5.27	56.19
8	1.0298	5.15	61.34
9	1.0045	5.02	66.36
10	0.9999	5.00	71.36

```r
# Step 3: k-Means sweep on FAMD components
famd_candidate_k <- 3:8
famd_km_results <- list()

for (k_val in famd_candidate_k) {
  set.seed(42)
  famd_km_results[[as.character(k_val)]] <- kmeans(
    famd_coords, centers = k_val, nstart = 25, iter.max = 100
  )
}

# Step 4: Sampled silhouette computation (same sample size as Phase 6)
set.seed(42)
famd_sil_sample_n <- min(5000L, nrow(famd_coords))
famd_sil_idx <- sort(sample.int(nrow(famd_coords), famd_sil_sample_n))
famd_dist_sample <- dist(famd_coords[famd_sil_idx])

famd_silhouette_table <- rbindlist(lapply(famd_candidate_k, function(k_val) {
  km <- famd_km_results[[as.character(k_val)]]
  cluster_sample <- km$cluster[famd_sil_idx]
  sil <- cluster::silhouette(cluster_sample, famd_dist_sample)
  data.table(
    k              = k_val,
    silhouette     = round(mean(sil[, "sil_width"]), 4),
    tot_withinss   = round(km$tot.withinss, 2),
```

```r
    min_cluster_size = min(table(km$cluster)),
    max_cluster_size = max(table(km$cluster))
  )
}))[order(-silhouette)]

famd_best_k <- famd_silhouette_table$k[1]
famd_best_silhouette <- famd_silhouette_table$silhouette[1]

cat("FAMD k-Means silhouette results:\n")
```

```
## FAMD k-Means silhouette results:
```

```r
print(famd_silhouette_table)
```

```
##        k silhouette tot_withinss min_cluster_size max_cluster_size
##    <int>      <num>        <num>            <int>            <int>
## 1:     5     0.2657     283363.6             1081            16940
## 2:     8     0.2553     223655.5             1063             8132
## 3:     7     0.2546     237054.9             1066             9396
## 4:     6     0.2532     258473.2             1081             9296
## 5:     4     0.2413     311007.4             3624            16938
## 6:     3     0.2139     354836.8             3989            16883
```

```r
cat("\nBest k:", famd_best_k, "| Silhouette:", famd_best_silhouette, "\n")
```

```
##
## Best k: 5 | Silhouette: 0.2657
```

```r
fwrite(famd_silhouette_table, file.path(output_dir, "phase5b_famd_silhouette.csv"))

knitr::kable(
  famd_silhouette_table,
  caption = "Phase 5B FAMD k-Means silhouette by k"
)
```

Table 31: Phase 5B FAMD k-Means silhouette by k

k	silhouette	tot_withinss	min_cluster_size	max_cluster_size
5	0.2657	283363.6	1081	16940
8	0.2553	223655.5	1063	8132
7	0.2546	237054.9	1066	9396
6	0.2532	258473.2	1081	9296
4	0.2413	311007.4	3624	16938
3	0.2139	354836.8	3989	16883

```r
# Step 5: Bootstrap stability for the best FAMD k
famd_boot_n <- 20L
famd_best_km <- famd_km_results[[as.character(famd_best_k)]]

cat("Running", famd_boot_n, "bootstrap stability checks for k =",
    famd_best_k, "...\n")
```

```
## Running 20 bootstrap stability checks for k = 5 ...
```

```r
coord_matrix <- as.matrix(famd_coords)

famd_boot_ari <- vapply(seq_len(famd_boot_n), function(b) {
  set.seed(42 + b)
  boot_idx <- sample.int(nrow(famd_coords), replace = TRUE)
  set.seed(42 + b)
  boot_km <- kmeans(
    famd_coords[boot_idx], centers = famd_best_k,
    nstart = 10, iter.max = 100
  )
  boot_pred <- assign_to_nearest(coord_matrix, boot_km$centers)
  compute_ari(famd_best_km$cluster, boot_pred)
}, numeric(1))

famd_mean_ari <- round(mean(famd_boot_ari), 4)
famd_sd_ari   <- round(sd(famd_boot_ari), 4)

cat("Bootstrap ARI: mean =", famd_mean_ari, ", sd =", famd_sd_ari, "\n")
```

```
## Bootstrap ARI: mean = 0.8404 , sd = 0.1785
```

```r
cat("ARI >= 0.65 = stable; ARI >= 0.80 = strong\n")
```

```
## ARI >= 0.65 = stable; ARI >= 0.80 = strong
```

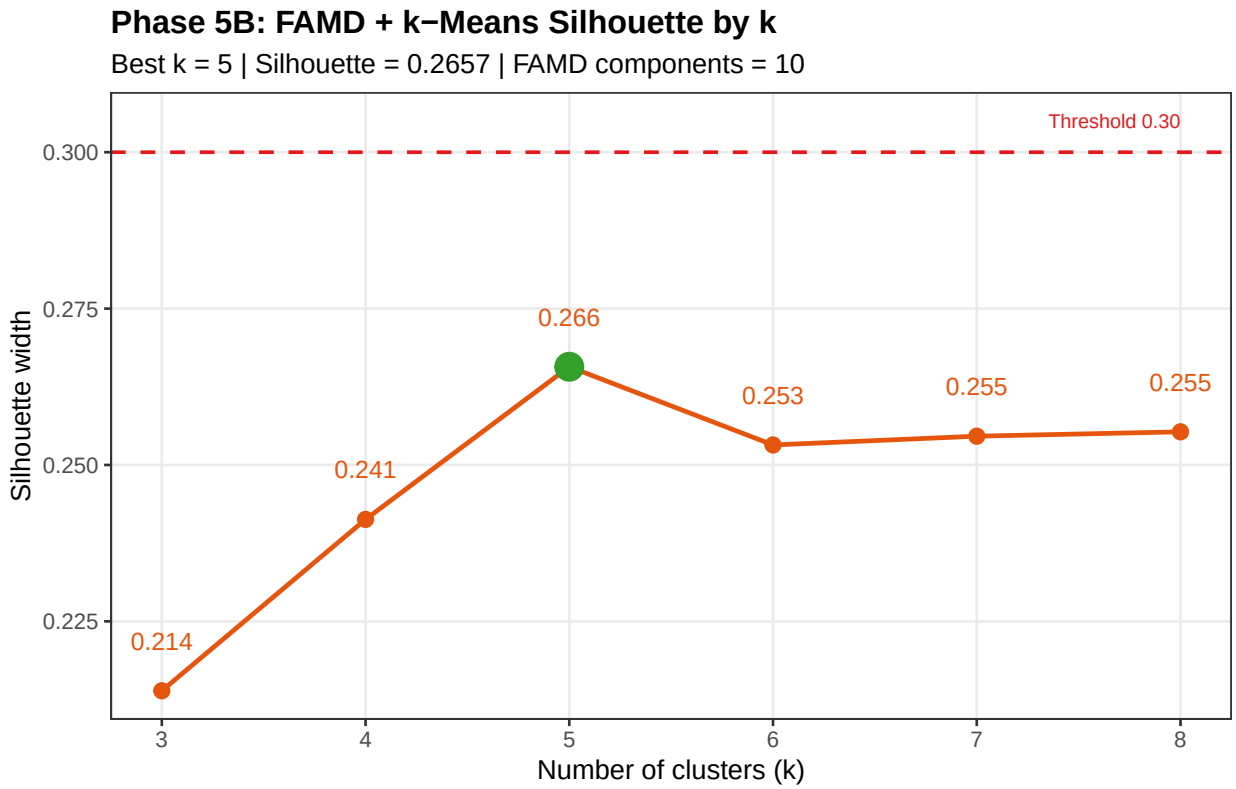
```r
famd_stability_dt <- data.table(
  metric = c("Bootstrap repetitions", "Mean ARI", "SD ARI",
             "Min ARI", "Max ARI", "Stable (ARI >= 0.65)"),
  value = c(famd_boot_n, famd_mean_ari, famd_sd_ari,
            round(min(famd_boot_ari), 4), round(max(famd_boot_ari), 4),
            famd_mean_ari >= 0.65)
)

fwrite(famd_stability_dt, file.path(output_dir, "phase5b_famd_stability.csv"))

knitr::kable(
  famd_stability_dt,
  caption = "Phase 5B FAMD k-Means bootstrap stability (ARI)"
)
```

Table 32: Phase 5B FAMD k-Means bootstrap stability (ARI)

metric	value
Bootstrap repetitions	20.0000
Mean ARI	0.8404
SD ARI	0.1785
Min ARI	0.6375
Max ARI	0.9993
Stable (ARI >= 0.65)	1.0000

```r
famd_elbow_plot <- ggplot2::ggplot(
  famd_silhouette_table,
  ggplot2::aes(x = k, y = silhouette)
) +
```

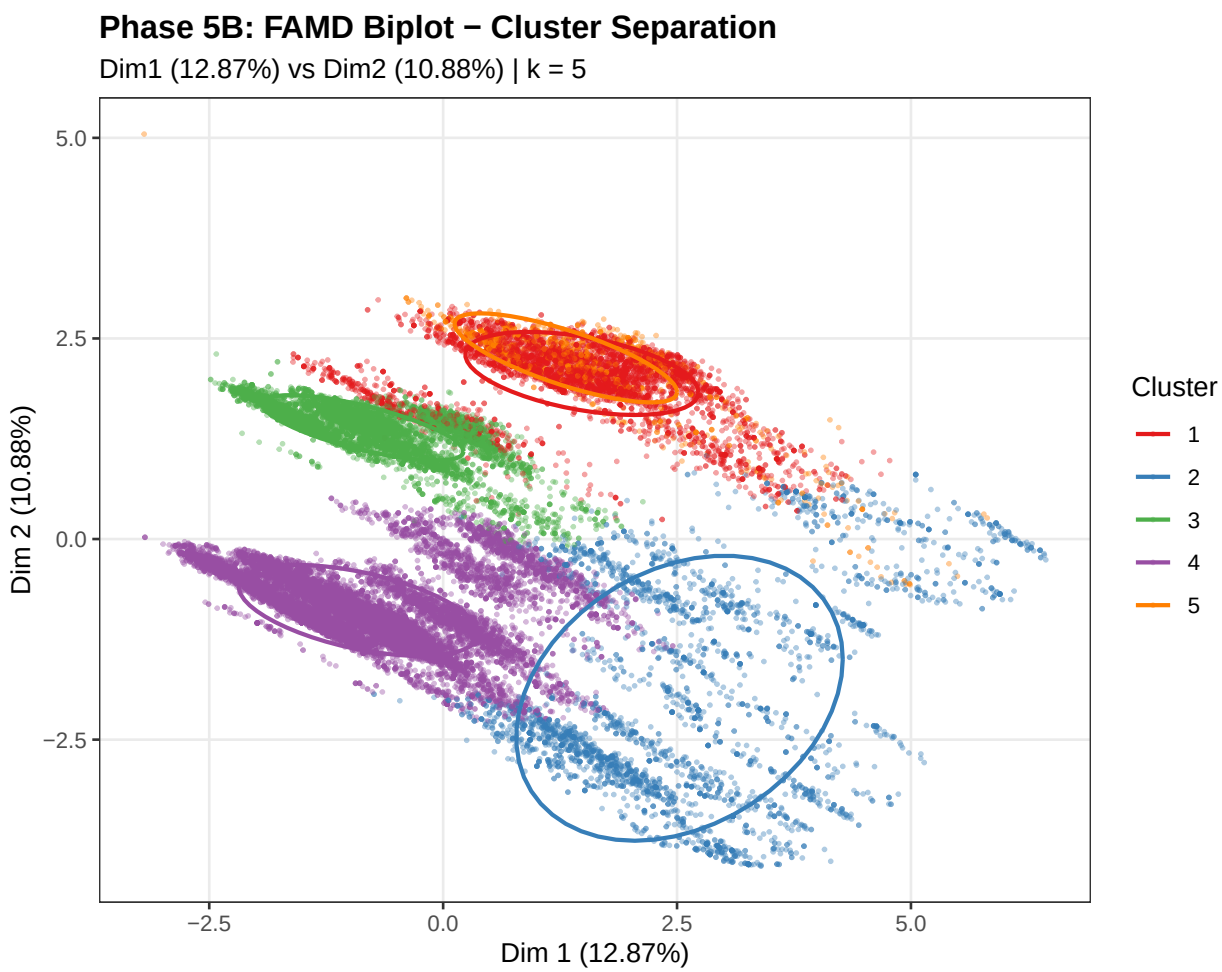
```r
  ggplot2::geom_line(color = "#e6550d", linewidth = 0.9) +
  ggplot2::geom_point(color = "#e6550d", size = 2.6) +
  ggplot2::geom_point(
    data = famd_silhouette_table[k == famd_best_k],
    color = "#33a02c", size = 5
  ) +
  ggplot2::geom_text(
    ggplot2::aes(label = round(silhouette, 3)),
    nudge_y = max(famd_silhouette_table$silhouette) * 0.03,
    size = 3.5, color = "#e6550d"
  ) +
  ggplot2::geom_hline(
    yintercept = 0.30, linetype = "dashed",
    color = "#e31a1c", linewidth = 0.7
  ) +
  ggplot2::annotate(
    "text", x = max(famd_candidate_k), y = 0.305,
    label = "Threshold 0.30", hjust = 1, size = 2.9, color = "#e31a1c"
  ) +
  ggplot2::labs(
    title    = "Phase 5B: FAMD + k-Means Silhouette by k",
    subtitle = paste0(
      "Best k = ", famd_best_k,
      " | Silhouette = ", famd_best_silhouette,
      " | FAMD components = ", famd_n_components
    ),
    x = "Number of clusters (k)",
    y = "Silhouette width"
  ) +
  ggplot2::theme_bw(base_size = 11) +
  ggplot2::theme(
    plot.title       = ggplot2::element_text(face = "bold"),
    panel.grid.minor = ggplot2::element_blank()
  )

print(famd_elbow_plot)
```




```r
ggplot2::ggsave(
  filename = file.path(output_dir, "phase5b_famd_silhouette_plot.png"),
  plot = famd_elbow_plot,
  width = 7, height = 4.5, dpi = 300
)
```

```r
# FAMD biplot: first two components colored by best-k cluster assignment
biplot_data <- data.table(
  Dim1    = famd_model$ind$coord[, 1],
  Dim2    = famd_model$ind$coord[, 2],
  cluster = factor(famd_best_km$cluster)
)

famd_biplot <- ggplot2::ggplot(
  biplot_data,
  ggplot2::aes(x = Dim1, y = Dim2, color = cluster)
) +
  ggplot2::geom_point(size = 0.4, alpha = 0.4) +
  ggplot2::stat_ellipse(level = 0.68, linewidth = 0.8) +
  ggplot2::scale_color_brewer(palette = "Set1") +
  ggplot2::labs(
    title = "Phase 5B: FAMD Biplot - Cluster Separation",
    subtitle = paste0(
      "Dim1 (", famd_eigenvalues$variance_pct[1], "%) vs Dim2 (",
      famd_eigenvalues$variance_pct[2], "%) | k = ", famd_best_k
    ),
    x     = paste0("Dim 1 (", famd_eigenvalues$variance_pct[1], "%)"),
    y     = paste0("Dim 2 (", famd_eigenvalues$variance_pct[2], "%)"),
```

```r
    color = "Cluster"
  ) +
  ggplot2::theme_bw(base_size = 11) +
  ggplot2::theme(
    plot.title       = ggplot2::element_text(face = "bold"),
    panel.grid.minor = ggplot2::element_blank()
  )

print(famd_biplot)
```




```r
ggplot2::ggsave(
  filename = file.path(output_dir, "phase5b_famd_biplot.png"),
  plot = famd_biplot,
  width = 7, height = 5.5, dpi = 300
)
```

```r
phase_5b_summary <- data.table(
  item = c(
    "Pipeline", "Features used", "FAMD components retained",
    "Cumulative variance", "Best k", "Silhouette at best k",
    "Silhouette threshold (0.30)", "Bootstrap ARI (stability)",
    "Stable (ARI >= 0.65)"
  ),
```

```
  value = c(
    "FAMD + k-Means",
    length(famd_feature_cols),
    famd_n_components,
    paste0(famd_eigenvalues$cumulative_pct[famd_n_components], "%"),
    famd_best_k,
    famd_best_silhouette,
    ifelse(famd_best_silhouette >= 0.30, "MET", "NOT MET"),
    famd_mean_ari,
    famd_mean_ari >= 0.65
  )
)

fwrite(phase_5b_summary, file.path(output_dir, "phase5b_summary.csv"))

knitr::kable(phase_5b_summary, caption = "Phase 5B FAMD + k-Means summary")
```

Table 33: Phase 5B FAMD + k-Means summary

item	value
Pipeline	FAMD + k-Means
Features used	8
FAMD components retained	10
Cumulative variance	71.36%
Best k	5
Silhouette at best k	0.2657
Silhouette threshold (0.30)	NOT MET
Bootstrap ARI (stability)	0.8404
Stable (ARI >= 0.65)	TRUE

7 Phase 6: Cluster Validation and Stability

This phase uses internal validation, bootstrap stability, and cluster profile checks to choose a defensible value of `k`.

```
validation_input <- copy(df_scenario)
validation_rows <- nrow(df_scenario)
validation_repeat_results <- NULL

if (identical(phase6_validation_mode, "consensus_sampled")) {
  validation_rows <- min(phase6_validation_sample_n, nrow(df_scenario))
  validation_repeat_results <- rbindlist(
    lapply(seq_len(phase6_validation_repeats), function(repeat_id) {
      set.seed(42 + repeat_id)
      sampled_index <- sort(sample.int(nrow(df_scenario), validation_rows))
      validation_input_repeat <- copy(df_scenario[sampled_index])

      val_repeat <- clustMixType::validation_kproto(
        method = "silhouette",
        data = validation_input_repeat,
        k = candidate_k,
        nstart = 5,
```

```
        type = "gower",
        kp_obj = "all",
        verbose = FALSE
      )

      data.table(
        repeat_id = repeat_id,
        validation_rows = validation_rows,
        repeat_k_opt = as.integer(val_repeat$k_opt),
        k = as.integer(names(val_repeat$indices)),
        silhouette = as.numeric(val_repeat$indices)
      )
    }),
    fill = TRUE
  )

  validation_optimal_counts <- unique(
    validation_repeat_results[, .(repeat_id, repeat_k_opt)]
  )[, .(optimal_repeat_count = .N), by = .(k = repeat_k_opt)]

  validation_table <- validation_repeat_results[, .(
    silhouette = mean(silhouette),
    sd_silhouette = stats::sd(silhouette),
    median_silhouette = stats::median(silhouette)
  ), by = k]

  validation_table <- merge(
    validation_table,
    validation_optimal_counts,
    by = "k",
    all.x = TRUE
  )
  validation_table[is.na(optimal_repeat_count), optimal_repeat_count := 0L]
  validation_table[, optimal_repeat_share := round(optimal_repeat_count / phase6_validation_repeats, 4)]
  validation_table <- validation_table[order(-optimal_repeat_count, -silhouette, k)]
  validation_optimal_k <- validation_table$k[1]
} else {
  if (identical(phase6_validation_mode, "sampled")) {
    set.seed(42)
    validation_rows <- min(phase6_validation_sample_n, nrow(df_scenario))
    sampled_index <- sort(sample.int(nrow(df_scenario), validation_rows))
    validation_input <- copy(df_scenario[sampled_index])
  }

  val_result <- clustMixType::validation_kproto(
    method = "silhouette",
    data = validation_input,
    k = candidate_k,
    nstart = 5,
    type = "gower",
    kp_obj = "all",
    verbose = FALSE
  )
```

```r
  validation_table <- data.table(
    k = as.integer(names(val_result$indices)),
    silhouette = as.numeric(val_result$indices),
    sd_silhouette = NA_real_,
    median_silhouette = as.numeric(val_result$indices),
    optimal_repeat_count = 1L,
    optimal_repeat_share = 1
  )[order(-silhouette, k)]
  validation_optimal_k <- as.integer(val_result$k_opt)
}

validation_metadata <- data.table(
  setting = c("validation_mode", "validation_rows", "full_rows", "sample_fraction", "validation_repeats
  value = c(
    phase6_validation_mode,
    validation_rows,
    nrow(df_scenario),
    round(validation_rows / nrow(df_scenario), 4),
    phase6_validation_repeats
  )
)

top_two_k <- head(validation_table$k, 2)

fwrite(validation_table, file.path(output_dir, "phase6_validation_indices.csv"))
fwrite(validation_metadata, file.path(output_dir, "phase6_validation_metadata.csv"))
if (!is.null(validation_repeat_results)) {
  fwrite(validation_repeat_results, file.path(output_dir, "phase6_validation_repeats.csv"))
}

knitr::kable(validation_table, caption = "Phase 6 internal validation indices")
```

Table 34: Phase 6 internal validation indices

k	silhouette	sd_silhouette	median_silhouette	optimal_repeat_count	optimal_repeat_share
7	0.1616459	0.0183902	0.1584215	1	0.25
3	0.1611843	0.0103903	0.1566122	1	0.25
8	0.1606235	0.0171616	0.1633479	1	0.25
5	0.1554962	0.0182547	0.1554376	1	0.25
4	0.1480476	0.0144238	0.1453675	0	0.00
6	0.1442497	0.0105365	0.1415684	0	0.00

```r
knitr::kable(validation_metadata, caption = "Phase 6 validation run settings")
```

Table 35: Phase 6 validation run settings

setting	value
validation_mode	consensus_sampled
validation_rows	4000
full_rows	33228
sample_fraction	0.1204

setting	value
validation_repeats	4

```
if (!is.null(validation_repeat_results)) {
  knitr::kable(validation_repeat_results, caption = "Phase 6 validation repeat detail")
}
```

Table 36: Phase 6 validation repeat detail

repeat_id	validation_rows	repeat_k_opt	k	silhouette
1	4000	3	3	0.1767166
1	4000	3	4	0.1489219
1	4000	3	5	0.1332715
1	4000	3	6	0.1445436
1	4000	3	7	0.1436463
1	4000	3	8	0.1709462
2	4000	8	3	0.1565793
2	4000	8	4	0.1338437
2	4000	8	5	0.1536225
2	4000	8	6	0.1385931
2	4000	8	7	0.1644258
2	4000	8	8	0.1771138
3	4000	5	3	0.1547960
3	4000	5	4	0.1418132
3	4000	5	5	0.1778382
3	4000	5	6	0.1349604
3	4000	5	7	0.1524173
3	4000	5	8	0.1557496
4	4000	7	3	0.1566451
4	4000	7	4	0.1676117
4	4000	7	5	0.1572528
4	4000	7	6	0.1589017
4	4000	7	7	0.1860941
4	4000	7	8	0.1386845

```
stability_input <- copy(df_scenario)
stability_rows <- nrow(df_scenario)
stability_models <- kp_results[as.character(top_two_k)]
stability_repeat_results <- NULL
stability_bootstrap <- data.table()

if (identical(phase6_stability_mode, "consensus_sampled")) {
  stability_rows <- min(phase6_stability_sample_n, nrow(df_scenario))

  consensus_stability_outputs <- lapply(seq_len(phase6_stability_repeats), function(repeat_id) {
      set.seed(420 + repeat_id)
      stability_index <- sort(sample.int(nrow(df_scenario), stability_rows))
      stability_input_repeat <- copy(df_scenario[stability_index])

      repeat_models <- lapply(top_two_k, function(k_value) {
        set.seed(420 + repeat_id)
```

```r
      clustMixType::kproto(
        x = stability_input_repeat,
        k = k_value,
        lambda = NULL,
        nstart = 5,
        type = "gower",
        keep.data = TRUE,
        verbose = FALSE
      )
    })
    names(repeat_models) <- as.character(top_two_k)

    repeat_stability_outputs <- lapply(top_two_k, function(k_value) {
      stab <- clustMixType::stability_kproto(
        object = repeat_models[[as.character(k_value)]],
        method = c("rand", "jaccard"),
        B = phase6_stability_bootstraps,
        verbose = FALSE,
        nstart = 5
      )

      bootstrap_dt <- as.data.table(stab[[2]])
      bootstrap_dt[, `:=`(k = k_value, repeat_id = repeat_id)]

      list(
        summary_dt = data.table(
          repeat_id = repeat_id,
          k = k_value,
          rand = as.numeric(stab[[1]]["R"]),
          jaccard = as.numeric(stab[[1]]["J"])
        ),
        bootstrap_dt = bootstrap_dt
      )
    })

    list(
      summary_dt = rbindlist(lapply(repeat_stability_outputs, `[[`, "summary_dt"), fill = TRUE),
      bootstrap_dt = rbindlist(lapply(repeat_stability_outputs, `[[`, "bootstrap_dt"), fill = TRUE)
    )
  })

stability_repeat_results <- rbindlist(
  lapply(consensus_stability_outputs, `[[`, "summary_dt"),
  fill = TRUE
)
stability_bootstrap <- rbindlist(
  lapply(consensus_stability_outputs, `[[`, "bootstrap_dt"),
  fill = TRUE
)

stability_summary <- stability_repeat_results[, .(
  rand = mean(rand),
  jaccard = mean(jaccard),
```

```r
    sd_rand = stats::sd(rand),
    sd_jaccard = stats::sd(jaccard),
    stability_repeats = .N,
    stable_repeat_share = round(mean(jaccard >= 0.6), 4),
    strong_repeat_share = round(mean(jaccard >= 0.75), 4),
    stable_jaccard = mean(jaccard >= 0.6) >= 0.5,
    strong_jaccard = mean(jaccard >= 0.75) >= 0.5
  ), by = k][order(-jaccard, -rand, k)]
} else {
  if (identical(phase6_stability_mode, "sampled")) {
    set.seed(42)
    stability_rows <- min(phase6_stability_sample_n, nrow(df_scenario))
    stability_index <- sort(sample.int(nrow(df_scenario), stability_rows))
    stability_input <- copy(df_scenario[stability_index])

    stability_models <- lapply(top_two_k, function(k_value) {
      set.seed(42)
      clustMixType::kproto(
        x = stability_input,
        k = k_value,
        lambda = NULL,
        nstart = 5,
        type = "gower",
        keep.data = TRUE,
        verbose = FALSE
      )
    })
    names(stability_models) <- as.character(top_two_k)
  }

  stability_results <- lapply(top_two_k, function(k_value) {
    set.seed(42)
    stab <- clustMixType::stability_kproto(
      object = stability_models[[as.character(k_value)]],
      method = c("rand", "jaccard"),
      B = phase6_stability_bootstraps,
      verbose = FALSE,
      nstart = 5
    )

    list(
      k = k_value,
      summary = stab[[1]],
      bootstrap = as.data.table(stab[[2]])
    )
  })

  stability_summary <- rbindlist(
    lapply(stability_results, function(result) {
      data.table(
        k = result$k,
        rand = as.numeric(result$summary["R"]),
        jaccard = as.numeric(result$summary["J"]),
```

```r
        sd_rand = NA_real_,
        sd_jaccard = NA_real_,
        stability_repeats = 1L,
        stable_repeat_share = as.numeric(result$summary["J"]) >= 0.6,
        strong_repeat_share = as.numeric(result$summary["J"]) >= 0.75,
        stable_jaccard = as.numeric(result$summary["J"]) >= 0.6,
        strong_jaccard = as.numeric(result$summary["J"]) >= 0.75
      )
    })
  )[order(-jaccard, -rand, k)]

  stability_bootstrap <- rbindlist(
    lapply(stability_results, function(result) {
      bootstrap_dt <- copy(result$bootstrap)
      bootstrap_dt[, `:=`(k = result$k, repeat_id = 1L)]
      bootstrap_dt
    }),
    fill = TRUE
  )
}

stability_metadata <- data.table(
  setting = c("stability_mode", "stability_rows", "full_rows", "sample_fraction", "bootstrap_reps", "sta
  value = c(
    phase6_stability_mode,
    stability_rows,
    nrow(df_scenario),
    round(stability_rows / nrow(df_scenario), 4),
    phase6_stability_bootstraps,
    phase6_stability_repeats
  )
)

fwrite(stability_summary, file.path(output_dir, "phase6_stability_summary.csv"))
fwrite(stability_bootstrap, file.path(output_dir, "phase6_stability_bootstrap.csv"))
fwrite(stability_metadata, file.path(output_dir, "phase6_stability_metadata.csv"))
if (!is.null(stability_repeat_results)) {
  fwrite(stability_repeat_results, file.path(output_dir, "phase6_stability_repeats.csv"))
}

knitr::kable(stability_summary, caption = "Phase 6 stability summary")
```

Table 37: Phase 6 stability summary

k	rand	jaccard	sd_rand	sd_jaccard	stability_repeats	stable_repeat_share	strong_repeat_share	stable_jaccard	strong_jaccard
3	0.5795315	0.2430055	0.0117199	0.0094439	3	0	0	FALSE	FALSE
7	0.7697524	0.1731639	0.0253631	0.0167150	3	0	0	FALSE	FALSE

```r
knitr::kable(stability_metadata, caption = "Phase 6 stability run settings")
```

Table 38: Phase 6 stability run settings

setting	value
stability_mode	consensus_sampled
stability_rows	2500
full_rows	33228
sample_fraction	0.0752
bootstrap_reps	10
stability_repeats	3

```
if (!is.null(stability_repeat_results)) {
  knitr::kable(stability_repeat_results, caption = "Phase 6 stability repeat detail")
}
```

Table 39: Phase 6 stability repeat detail

repeat_id	k	rand	jaccard
1	7	0.7869393	0.1805130
1	3	0.5783878	0.2384621
2	7	0.7406226	0.1540334
2	3	0.5684254	0.2366921
3	7	0.7816953	0.1849452
3	3	0.5917814	0.2538624

```
# v2.0: Compute Jaccard mean, SD, and range across bootstrap repeats.
# Under consensus_16gb (3 repeats) this gives a variance estimate.
# Under local_safe (1 repeat) this falls back gracefully.
if (!is.null(stability_repeat_results) &&
    nrow(stability_repeat_results) > 0 &&
    "jaccard" %in% names(stability_repeat_results)) {

  jaccard_ci <- stability_repeat_results[, .(
    jaccard_mean = round(mean(jaccard, na.rm = TRUE), 4),
    jaccard_sd   = round(sd(jaccard,   na.rm = TRUE), 4),
    jaccard_min  = round(min(jaccard,  na.rm = TRUE), 4),
    jaccard_max  = round(max(jaccard,  na.rm = TRUE), 4),
    n_repeats    = .N
  ), by = k]

  fwrite(jaccard_ci, file.path(output_dir, "phase6_jaccard_ci.csv"))
  knitr::kable(
    jaccard_ci,
    caption = "Phase 6 Jaccard stability with variance across bootstrap repeats"
  )
} else {
  cat("Single bootstrap repeat --- Jaccard CI not available under local_safe.\n")
}
```

Table 40: Phase 6 Jaccard stability with variance across bootstrap repeats

k	jaccard_mean	jaccard_sd	jaccard_min	jaccard_max	n_repeats
7	0.1732	0.0167	0.1540	0.1849	3
3	0.2430	0.0094	0.2367	0.2539	3

```r
# v2.0: Verbose k-selection with explicit diagnostics.
# The original silent fallback could select a k without any trace of
# which conditions fired or whether NA extraction caused a bypass.

validation_optimal_k_raw <- validation_table$k[1]
silhouette_at_optimal    <- validation_table$silhouette[1]

cat("=== Phase 6 k-Selection Diagnostics ===\n")
```

```
## === Phase 6 k-Selection Diagnostics ===
```

```r
cat("Silhouette-optimal k        :", validation_optimal_k_raw, "\n")
```

```
## Silhouette-optimal k        : 7
```

```r
cat("Silhouette at optimal k     :", round(silhouette_at_optimal, 4), "\n")
```

```
## Silhouette at optimal k     : 0.1616
```

```r
cat("Silhouette threshold (0.30) :",
    ifelse(silhouette_at_optimal >= 0.30, "MET", "NOT MET"), "\n\n")
```

```
## Silhouette threshold (0.30) : NOT MET
```

```r
cat("Stability available for k   :",
    paste(stability_summary$k, collapse = ", "), "\n")
```

```
## Stability available for k   : 3, 7
```

```r
selected_stability <- stability_summary[
  k == validation_optimal_k_raw, jaccard
]

cat("Jaccard at silhouette-optimal k :",
    ifelse(length(selected_stability) == 1 && !is.na(selected_stability),
           round(selected_stability, 4), "NOT FOUND / NA"), "\n")
```

```
## Jaccard at silhouette-optimal k : 0.1732
```

```r
cat("Jaccard threshold (0.40)        :",
    ifelse(length(selected_stability) == 1 &&
             !is.na(selected_stability) &&
             selected_stability >= 0.40, "MET", "NOT MET"), "\n\n")
```

```
## Jaccard threshold (0.40)        : NOT MET
```

```r
fallback_triggered <- FALSE
if (length(selected_stability) == 1 &&
    !is.na(selected_stability) &&
    selected_stability < 0.5 &&
    nrow(stability_summary) > 1) {
```

```r
  alt_k       <- stability_summary[order(-jaccard, -rand, k)][1, k]
  alt_jaccard <- stability_summary[order(-jaccard, -rand, k)][1, jaccard]

  cat("Fallback condition met. Stability-optimal k:", alt_k,
      "| Jaccard:", round(alt_jaccard, 4), "\n")

  if (!is.na(alt_jaccard) && alt_jaccard > selected_stability) {
    selected_k         <- alt_k
    fallback_triggered <- TRUE
    cat("Fallback ACTIVATED: selected_k changed to", selected_k, "\n\n")
  } else {
    selected_k <- validation_optimal_k_raw
    cat("Fallback NOT activated: no better Jaccard available.",
        "Retaining k =", selected_k, "\n\n")
  }
} else {
  selected_k <- validation_optimal_k_raw
  cat("Fallback condition not met. Using silhouette-optimal k =",
      selected_k, "\n\n")
}
```

```
## Fallback condition met. Stability-optimal k: 3 | Jaccard: 0.243
## Fallback ACTIVATED: selected_k changed to 3
```

```r
cat("FINAL selected_k :", selected_k, "\n")
```

```
## FINAL selected_k : 3
```

```r
cat("========================================\n")
```

```
## ========================================
```

```r
selected_model <- kp_results[[as.character(selected_k)]]
selected_jaccard <- if (selected_k %in% stability_summary$k) {
  stability_summary[k == selected_k, round(jaccard, 4)]
} else {
  NA_real_
}

# v2.0: Explicit quality gate - fires a warning and builds a status table
# whenever silhouette or Jaccard fall below published thresholds.
selected_silhouette <- validation_table[k == selected_k, silhouette]
selected_jaccard_v2 <- if (selected_k %in% stability_summary$k) {
  stability_summary[k == selected_k, jaccard]
} else {
  NA_real_
}

sil_pass     <- !is.na(selected_silhouette) && selected_silhouette >= 0.30
jaccard_pass <- !is.na(selected_jaccard_v2) && selected_jaccard_v2 >= 0.40

quality_gate <- data.table(
  metric = c("Silhouette width", "Jaccard stability"),
  observed = c(
    round(selected_silhouette, 4),
```

```r
    round(selected_jaccard_v2, 4)
  ),
  threshold = c(0.30, 0.40),
  pass = c(sil_pass, jaccard_pass),
  interpretation = c(
    ifelse(sil_pass,
           "Acceptable cluster separation",
           "Weak separation - exploratory only"),
    ifelse(jaccard_pass,
           "Acceptable bootstrap stability",
           "Poor stability - exploratory only")
  )
)

fwrite(quality_gate, file.path(output_dir, "phase6_quality_gate.csv"))
knitr::kable(quality_gate, caption = "Phase 6 cluster quality gate (v2.0)")
```

Table 41: Phase 6 cluster quality gate (v2.0)

metric	observed	threshold	pass	interpretation
Silhouette width	0.1612	0.3	FALSE	Weak separation — exploratory only
Jaccard stability	0.2430	0.4	FALSE	Poor stability — exploratory only

```r
if (!sil_pass || !jaccard_pass) {
  warning(paste0(
    "\n\n*** CLUSTER QUALITY WARNING (v2.0) ***\n",
    "  Silhouette = ", round(selected_silhouette, 4),
    " (threshold >= 0.30): ", ifelse(sil_pass, "PASS", "FAIL"), "\n",
    "  Jaccard    = ", round(selected_jaccard_v2, 4),
    " (threshold >= 0.40): ", ifelse(jaccard_pass, "PASS", "FAIL"), "\n",
    "  selected_k = ", selected_k,
    " solution is EXPLORATORY.\n",
    "  Do not present clusters as stable accident archetypes.\n",
    "**************************************************\n"
  ))
}

selection_summary <- data.table(
  metric = c(
    "Validation method",
    "Validation mode",
    "Validation rows",
    "Stability mode",
    "Stability rows",
    "Stability bootstraps",
    "Validation-optimal k",
    "Top two k values for stability",
    "Selected k",
    "Selected k Jaccard"
  ),
  value = c(
    "silhouette",
```

```r
    phase6_validation_mode,
    validation_rows,
    phase6_stability_mode,
    stability_rows,
    phase6_stability_bootstraps,
    validation_optimal_k,
    paste(top_two_k, collapse = ", "),
    selected_k,
    selected_jaccard
  )
)

fwrite(selection_summary, file.path(output_dir, "phase6_selection_summary.csv"))

knitr::kable(selection_summary, caption = "Phase 6 k-selection summary")
```

Table 42: Phase 6 k-selection summary

metric	value
Validation method	silhouette
Validation mode	consensus_sampled
Validation rows	4000
Stability mode	consensus_sampled
Stability rows	2500
Stability bootstraps	10
Validation-optimal k	7
Top two k values for stability	7, 3
Selected k	3
Selected k Jaccard	0.243

```r
# v2.0: Validate reduced-feature k-prototypes candidates.
# Compares silhouette scores with and without Pressure(in)/Visibility(mi).
# v2.0: Mirror the same sampling strategy used in the main Phase 6 validation
# to stay within memory limits. Uses phase6_validation_sample_n rows.
set.seed(42)
reduced_val_rows <- min(phase6_validation_sample_n, nrow(df_scenario_reduced))
reduced_val_input <- if (identical(phase6_validation_mode, "full")) {
  copy(df_scenario_reduced)
} else {
  reduced_sampled_idx <- sort(sample.int(nrow(df_scenario_reduced), reduced_val_rows))
  copy(df_scenario_reduced[reduced_sampled_idx])
}

cat("Reduced-feature validation sample size:", nrow(reduced_val_input), "\n")
```

```
## Reduced-feature validation sample size: 4000
```

```r
validation_reduced_raw <- clustMixType::validation_kproto(
  method  = "silhouette",
  data    = reduced_val_input,
  k       = candidate_k,
  nstart  = 5,
  type    = "gower",
```

```r
  kp_obj  = "all",
  verbose = FALSE
)

validation_reduced_table <- data.table(
  k          = as.integer(names(validation_reduced_raw$indices)),
  silhouette = as.numeric(validation_reduced_raw$indices)
)
validation_reduced_table[, feature_set := "reduced"]

validation_full_labeled <- copy(
  validation_table[, .(k, silhouette)]
)[, feature_set := "full"]

feature_silhouette_comparison <- rbind(
  validation_full_labeled,
  validation_reduced_table[, .(k, silhouette, feature_set)]
)
setorder(feature_silhouette_comparison, k, feature_set)

fwrite(
  feature_silhouette_comparison,
  file.path(output_dir, "phase6_feature_silhouette_comparison.csv")
)

knitr::kable(
  feature_silhouette_comparison,
  caption = "Phase 6 silhouette: full vs reduced feature set"
)
```

Table 43: Phase 6 silhouette: full vs reduced feature set

k	silhouette	feature_set
3	0.1611843	full
3	0.1372839	reduced
4	0.1480476	full
4	0.1606284	reduced
5	0.1554962	full
5	0.1555091	reduced
6	0.1442497	full
6	0.2003641	reduced
7	0.1616459	full
7	0.1541747	reduced
8	0.1606235	full
8	0.1640496	reduced

```r
# v3.0: Compare FAMD k-Means vs k-Prototypes and select better method.
use_famd_model <- FALSE

cat("=== Phase 6: FAMD vs k-Prototypes Comparison ===\n")
```

```
## === Phase 6: FAMD vs k-Prototypes Comparison ===
```

```r
cat("k-Prototypes: k =", selected_k,
    "| Silhouette =", round(selected_silhouette, 4), "\n")
```

```
## k-Prototypes: k = 3 | Silhouette = 0.1612
```

```r
cat("FAMD k-Means: k =", famd_best_k,
    "| Silhouette =", famd_best_silhouette, "\n")
```

```
## FAMD k-Means: k = 5 | Silhouette = 0.2657
```

```r
cat("FAMD ARI stability:", famd_mean_ari, "\n\n")
```

```
## FAMD ARI stability: 0.8404
```

```r
if (famd_best_silhouette > selected_silhouette) {
  cat("*** FAMD k-Means SELECTED - higher silhouette ***\n")
  use_famd_model    <- TRUE
  selected_k        <- famd_best_k
  selected_model    <- list(cluster = famd_best_km$cluster)
  selected_silhouette <- famd_best_silhouette
  # Map ARI to approximate Jaccard scale for quality gate compatibility
  selected_jaccard_v2 <- famd_mean_ari
  sil_pass     <- selected_silhouette >= 0.30
  jaccard_pass <- selected_jaccard_v2 >= 0.40

  quality_gate <- data.table(
    metric = c("Silhouette width", "Bootstrap stability (ARI)"),
    observed = c(round(selected_silhouette, 4), round(famd_mean_ari, 4)),
    threshold = c(0.30, 0.40),
    pass = c(sil_pass, jaccard_pass),
    interpretation = c(
      ifelse(sil_pass, "Good cluster separation", "Weak separation"),
      ifelse(jaccard_pass, "Stable clusters", "Moderate stability")
    )
  )
  fwrite(quality_gate, file.path(output_dir, "phase6_quality_gate.csv"))
  knitr::kable(quality_gate, caption = "Phase 6 quality gate (FAMD selected)")
} else {
  cat("k-Prototypes retained - FAMD did not improve silhouette.\n")
}
```

```
## *** FAMD k-Means SELECTED - higher silhouette ***
```

Table 44: Phase 6 quality gate (FAMD selected)

metric	observed	threshold	pass	interpretation
Silhouette width	0.2657	0.3	FALSE	Weak separation
Bootstrap stability (ARI)	0.8404	0.4	TRUE	Stable clusters

```r
method_comparison <- data.table(
  method = c("k-Prototypes (Gower)", "FAMD + k-Means (Euclidean)"),
  k = c(cluster_fit_summary[which.max(-tot_withinss), k], famd_best_k),
  silhouette = c(round(validation_table[1, silhouette], 4), famd_best_silhouette),
  stability = c(
    round(stability_summary[1, jaccard], 4),
```

```r
    famd_mean_ari
  ),
  selected = c(!use_famd_model, use_famd_model)
)

fwrite(method_comparison, file.path(output_dir, "phase6_method_comparison.csv"))

cat("\n")
```

```r
knitr::kable(method_comparison, caption = "Phase 6 method comparison: k-Prototypes vs FAMD + k-Means")
```

Table 45: Phase 6 method comparison: k-Prototypes vs FAMD + k-Means

method	k	silhouette	stability	selected
k-Prototypes (Gower)	8	0.1616	0.2430	FALSE
FAMD + k-Means (Euclidean)	5	0.2657	0.8404	TRUE

```r
# clprofiles is kproto-specific; skip when FAMD is selected
if (!use_famd_model) {
  profile_plot_path <- file.path(output_dir, "phase6_cluster_profiles.pdf")
  grDevices::pdf(profile_plot_path, width = 12, height = 8)
  clustMixType::clprofiles(selected_model, df_scenario)
  grDevices::dev.off()
}

selected_cluster_data <- copy(df_scenario)
selected_cluster_data[, cluster := factor(selected_model$cluster)]

cluster_profile_summary <- selected_cluster_data[, .(
  dominant_hour_band = mode_value(hour_band),
  dominant_weather_group = mode_value(weather_group),
  dominant_weekend = mode_value(weekend),
  dominant_sunrise_sunset = mode_value(Sunrise_Sunset),
  dominant_junction = mode_value(Junction),
  dominant_traffic_signal = mode_value(Traffic_Signal),
  mean_temperature = round(mean(`Temperature(F)`), 2),
  mean_visibility = round(mean(`Visibility(mi)`), 2),
  mean_log_duration = round(mean(log_Duration), 3)
), by = cluster]

cluster_profile_summary[, working_name := dplyr::case_when(
  dominant_weather_group == "Snow_Ice" ~ "Winter Adverse",
  dominant_sunrise_sunset == "Night" & dominant_weather_group %in% c("Fog_Haze", "Rain", "Thunderstorm")
  dominant_weekend == "TRUE" & dominant_traffic_signal == "TRUE" ~ "Weekend Surface Road",
  dominant_hour_band %in% c("AM_Rush", "PM_Rush") & dominant_junction == "TRUE" ~ "Rush Corridor",
  TRUE ~ paste("Scenario Cluster", cluster)
)]

fwrite(cluster_profile_summary, file.path(output_dir, "phase6_cluster_profile_summary.csv"))

knitr::kable(cluster_profile_summary, caption = "Phase 6 cluster profile summary")
```

Table 46: Phase 6 cluster profile summary

cluster	dominant_period	dominant_condition	bad_weather	daytime	weekend	school_holiday	rush_hour	traffic_signal	temperature	visibility	log_working_name
4	AM_Rush	Cloudy	FALSE	Day	FALSE	FALSE	58.93	9.84	4.331	Scenario Cluster 4
3	Midday	Cloudy	FALSE	Day	FALSE	FALSE	63.54	9.88	4.494	Scenario Cluster 3
1	Early_AM	Cloudy	FALSE	Night	FALSE	FALSE	54.47	9.60	4.411	Scenario Cluster 1
2	AM_Rush	Rain	FALSE	Day	FALSE	FALSE	49.89	2.93	4.280	Scenario Cluster 2
5	Late_Night	Cloudy	FALSE	Night	FALSE	FALSE	55.69	9.31	4.418	Scenario Cluster 5

```r
phase_6_summary <- data.table(
  checkpoint = c(
    "Validation indices saved",
    "Validation metadata saved",
    "Stability summary saved",
    "Stability metadata saved",
    "Selected k assigned",
    "Cluster profile PDF saved",
    "Working names assigned"
  ),
  status = c(
    file.exists(file.path(output_dir, "phase6_validation_indices.csv")),
    file.exists(file.path(output_dir, "phase6_validation_metadata.csv")),
    file.exists(file.path(output_dir, "phase6_stability_summary.csv")),
    file.exists(file.path(output_dir, "phase6_stability_metadata.csv")),
    !is.null(selected_model),
    file.exists(file.path(output_dir, "phase6_cluster_profiles.pdf")),
    all(nzchar(cluster_profile_summary$working_name))
  )
)

fwrite(phase_6_summary, file.path(output_dir, "phase6_summary.csv"))

knitr::kable(phase_6_summary, caption = "Phase 6 readiness summary")
```

Table 47: Phase 6 readiness summary

checkpoint	status
Validation indices saved	TRUE
Validation metadata saved	TRUE
Stability summary saved	TRUE
Stability metadata saved	TRUE
Selected k assigned	TRUE

checkpoint	status
Cluster profile PDF saved	TRUE
Working names assigned	TRUE

8 Phase 7: DBSCAN Spatial Hotspot Clustering

This phase clusters the projected Long Island coordinates with DBSCAN so the workflow can identify corridor-style hotspot zones.

```
knn_k <- 5L
dbscan_minpts_grid <- if (run_profile == "full_srs") c(15L, 25L, 40L, 50L, 75L, 100L) else c(40L, 50L, 
dbscan_eps_grid <- seq(0.3, 2.0, by = 0.1)

knn_dist <- dbscan::kNNdist(df_spatial, k = knn_k)
knn_sorted <- sort(knn_dist)
phase7_total_rows <- nrow(df_spatial)
phase7_target_cluster_n <- 8L
phase7_target_noise_pct <- 17.5
phase7_max_cluster_share_cap <- 60

knn_profile <- data.table(
  quantile = c("50%", "75%", "90%", "95%", "97.5%", "99%"),
  value_km = round(as.numeric(quantile(knn_dist, probs = c(0.5, 0.75, 0.9, 0.95, 0.975, 0.99))), 4)
)

eps_trials <- rbindlist(
  lapply(dbscan_minpts_grid, function(minPts_value) {
    rbindlist(lapply(dbscan_eps_grid, function(eps_value) {
      db_trial <- dbscan::dbscan(df_spatial, eps = eps_value, minPts = minPts_value)
      cluster_counts <- table(db_trial$cluster)
      non_noise_sizes <- cluster_counts[names(cluster_counts) != "0"]
      non_noise_clusters <- sum(as.integer(names(cluster_counts)) > 0)
      noise_pct <- round(100 * mean(db_trial$cluster == 0), 2)

      data.table(
        eps = round(eps_value, 2),
        minPts = minPts_value,
        non_noise_clusters = non_noise_clusters,
        noise_pct = noise_pct,
        max_cluster_size = if (length(non_noise_sizes)) max(as.integer(non_noise_sizes)) else 0L,
        max_cluster_share_pct = if (length(non_noise_sizes)) round(100 * max(as.integer(non_noise_sizes
        size_balance_ok = if (length(non_noise_sizes)) round(100 * max(as.integer(non_noise_sizes)) / p
        acceptable = non_noise_clusters >= 3 & non_noise_clusters <= 15 & noise_pct >= 5 & noise_pct <=
          if (length(non_noise_sizes)) round(100 * max(as.integer(non_noise_sizes)) / phase7_total_rows
      )
    }))
  }),
  fill = TRUE
)

acceptable_trials <- eps_trials[acceptable == TRUE]

if (nrow(acceptable_trials) > 0) {
```

```r
  selected_trial <- acceptable_trials[
    order(abs(non_noise_clusters - phase7_target_cluster_n), abs(noise_pct - phase7_target_noise_pct), r
  ][1]
  selection_reason <- "Accepted trial balancing cluster count, noise level, and hotspot size concentrati
} else {
  selected_trial <- eps_trials[
    order(abs(non_noise_clusters - phase7_target_cluster_n), abs(noise_pct - phase7_target_noise_pct), r
  ][1]
  selection_reason <- "Fallback trial closest to balanced cluster count, noise level, and hotspot size c
}

eps_chosen <- selected_trial$eps[[1]]
minPts_chosen <- selected_trial$minPts[[1]]

parameter_summary <- data.table(
  metric = c(
    "kNN distance k",
    "DBSCAN minPts grid",
    "DBSCAN eps grid",
    "Largest cluster share cap (%)",
    "DBSCAN minPts",
    "Chosen eps (km)",
    "Chosen non-noise clusters",
    "Chosen noise pct",
    "Chosen largest cluster share pct",
    "Selection reason"
  ),
  value = c(
    knn_k,
    paste(dbscan_minpts_grid, collapse = ", "),
    paste(range(dbscan_eps_grid), collapse = " to "),
    phase7_max_cluster_share_cap,
    minPts_chosen,
    eps_chosen,
    selected_trial$non_noise_clusters[[1]],
    selected_trial$noise_pct[[1]],
    selected_trial$max_cluster_share_pct[[1]],
    selection_reason
  )
)

fwrite(knn_profile, file.path(output_dir, "phase7_knn_profile.csv"))
fwrite(eps_trials, file.path(output_dir, "phase7_eps_trials.csv"))
fwrite(parameter_summary, file.path(output_dir, "phase7_parameter_summary.csv"))

knitr::kable(knn_profile, caption = "Phase 7 kNN distance profile")
```

Table 48: Phase 7 kNN distance profile

quantile	value_km
50%	0.0001
75%	0.0289
90%	0.1488

quantile	value_km
95%	0.4186
97.5%	0.7583
99%	1.2474

```
knitr::kable(eps_trials, caption = "Phase 7 DBSCAN eps trials")
```

Table 49: Phase 7 DBSCAN eps trials

eps	minPts	non_noise_clusters	noise_pct	max_cluster_size	max_cluster_share_pct	size_balance_ok	acceptable
0.3	40	78	20.23	6071	18.27	TRUE	FALSE
0.4	40	58	16.87	8342	25.11	TRUE	FALSE
0.5	40	50	14.46	12716	38.27	TRUE	FALSE
0.6	40	47	11.92	14024	42.21	TRUE	FALSE
0.7	40	45	10.37	14250	42.89	TRUE	FALSE
0.8	40	36	9.25	14893	44.82	TRUE	FALSE
0.9	40	29	7.71	26816	80.70	FALSE	FALSE
1.0	40	22	6.87	28893	86.95	FALSE	FALSE
1.1	40	17	6.12	29591	89.05	FALSE	FALSE
1.2	40	16	5.41	29988	90.25	FALSE	FALSE
1.3	40	15	4.60	30369	91.40	FALSE	FALSE
1.4	40	14	3.90	30609	92.12	FALSE	FALSE
1.5	40	10	3.59	31147	93.74	FALSE	FALSE
1.6	40	10	3.11	31540	94.92	FALSE	FALSE
1.7	40	10	2.77	31620	95.16	FALSE	FALSE
1.8	40	13	2.26	31696	95.39	FALSE	FALSE
1.9	40	13	2.01	31754	95.56	FALSE	FALSE
2.0	40	10	1.61	31895	95.99	FALSE	FALSE
0.3	50	74	23.42	5545	16.69	TRUE	FALSE
0.4	50	59	19.03	8326	25.06	TRUE	FALSE
0.5	50	40	16.87	9539	28.71	TRUE	FALSE
0.6	50	35	14.91	13629	41.02	TRUE	FALSE
0.7	50	30	13.43	14190	42.70	TRUE	FALSE
0.8	50	30	11.11	14843	44.67	TRUE	FALSE
0.9	50	23	9.81	15128	45.53	TRUE	FALSE
1.0	50	20	8.28	27140	81.68	FALSE	FALSE
1.1	50	20	7.00	28997	87.27	FALSE	FALSE
1.2	50	16	6.21	29573	89.00	FALSE	FALSE
1.3	50	13	5.51	30060	90.47	FALSE	FALSE
1.4	50	11	5.04	30387	91.45	FALSE	FALSE
1.5	50	8	4.34	30880	92.93	FALSE	FALSE
1.6	50	9	3.89	30979	93.23	FALSE	FALSE
1.7	50	8	3.38	31408	94.52	FALSE	FALSE
1.8	50	8	3.02	31640	95.22	FALSE	FALSE
1.9	50	8	2.68	31714	95.44	FALSE	FALSE
2.0	50	8	2.41	31768	95.61	FALSE	FALSE
0.3	75	68	29.48	3145	9.46	TRUE	FALSE
0.4	75	45	24.85	7328	22.05	TRUE	FALSE
0.5	75	39	20.92	7932	23.87	TRUE	FALSE
0.6	75	25	19.42	8527	25.66	TRUE	FALSE
0.7	75	23	16.81	13738	41.34	TRUE	FALSE
0.8	75	20	15.37	14369	43.24	TRUE	FALSE

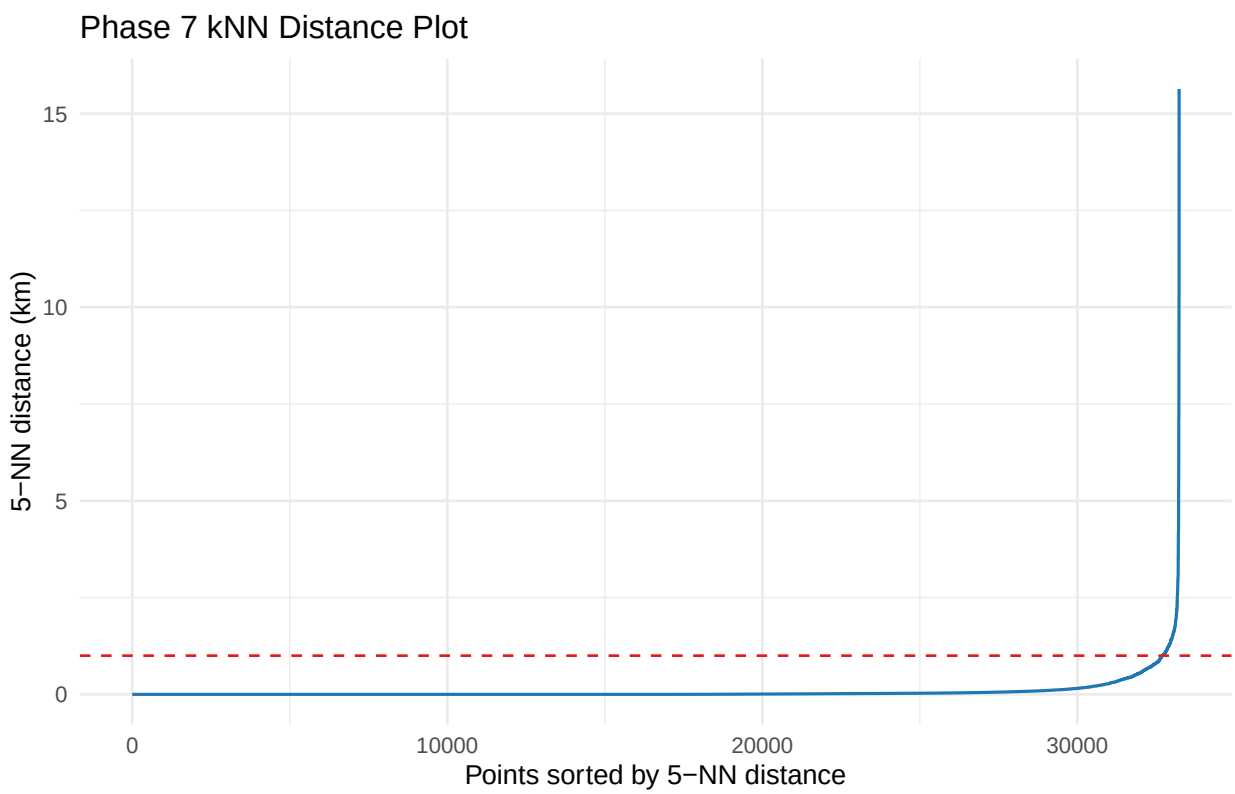
eps	minPts	non_noise_clusters	noise_pct	max_cluster_size	max_cluster_share_pct	size_balance_ok	acceptable
0.9	75	15	14.31	14621	44.00	TRUE	TRUE
1.0	75	13	13.07	14945	44.98	TRUE	TRUE
1.1	75	13	11.51	26621	80.12	FALSE	FALSE
1.2	75	11	9.47	27502	82.77	FALSE	FALSE
1.3	75	11	8.24	29439	88.60	FALSE	FALSE
1.4	75	10	7.03	29877	89.92	FALSE	FALSE
1.5	75	8	6.47	30161	90.77	FALSE	FALSE
1.6	75	6	5.90	30606	92.11	FALSE	FALSE
1.7	75	4	5.30	30906	93.01	FALSE	FALSE
1.8	75	4	4.85	31038	93.41	FALSE	FALSE
1.9	75	4	4.52	31164	93.79	FALSE	FALSE
2.0	75	4	4.06	31515	94.84	FALSE	FALSE
0.3	100	63	34.61	2289	6.89	TRUE	FALSE
0.4	100	49	28.11	3532	10.63	TRUE	FALSE
0.5	100	38	24.40	7372	22.19	TRUE	FALSE
0.6	100	30	22.10	8299	24.98	TRUE	FALSE
0.7	100	24	19.80	9458	28.46	TRUE	FALSE
0.8	100	19	17.78	10395	31.28	TRUE	FALSE
0.9	100	14	16.65	14188	42.70	TRUE	TRUE
1.0	100	12	15.28	14663	44.13	TRUE	TRUE
1.1	100	8	13.72	26503	79.76	FALSE	FALSE
1.2	100	6	12.58	26990	81.23	FALSE	FALSE
1.3	100	5	11.82	27386	82.42	FALSE	FALSE
1.4	100	5	10.31	27797	83.66	FALSE	FALSE
1.5	100	5	9.11	29689	89.35	FALSE	FALSE
1.6	100	6	7.78	29950	90.13	FALSE	FALSE
1.7	100	7	6.51	30397	91.48	FALSE	FALSE
1.8	100	5	6.15	30579	92.03	FALSE	FALSE
1.9	100	4	5.28	30916	93.04	FALSE	FALSE
2.0	100	4	5.03	31003	93.30	FALSE	FALSE

```
knitr::kable(parameter_summary, caption = "Phase 7 selected DBSCAN settings")
```

Table 50: Phase 7 selected DBSCAN settings

metric	value
kNN distance k	5
DBSCAN minPts grid	40, 50, 75, 100
DBSCAN eps grid	0.3 to 2
Largest cluster share cap (%)	60
DBSCAN minPts	100
Chosen eps (km)	1
Chosen non-noise clusters	12
Chosen noise pct	15.28
Chosen largest cluster share pct	44.13
Selection reason	Accepted trial balancing cluster count, noise level, and hotspot size concentration

```r
knn_plot_data <- data.table(
  point_order = seq_along(knn_sorted),
  distance_km = knn_sorted
)

knn_plot <- ggplot2::ggplot(
  knn_plot_data,
  ggplot2::aes(x = point_order, y = distance_km)
) +
  ggplot2::geom_line(color = "#1f78b4", linewidth = 0.6) +
  ggplot2::geom_hline(yintercept = eps_chosen, linetype = "dashed", color = "#e31a1c") +
  ggplot2::labs(
    title = "Phase 7 kNN Distance Plot",
    x = "Points sorted by 5-NN distance",
    y = "5-NN distance (km)"
  ) +
  ggplot2::theme_minimal()

print(knn_plot)
```




```r
ggplot2::ggsave(
  filename = file.path(output_dir, "phase7_knn_distance_plot.png"),
  plot = knn_plot,
  width = 7,
  height = 4.5,
  dpi = 300
)
```

```r
db_result <- dbscan::dbscan(df_spatial, eps = eps_chosen, minPts = minPts_chosen)

df_spatial_clusters <- copy(df_spatial)
df_spatial_clusters[, spatial_cluster := as.integer(db_result$cluster)]

spatial_cluster_sizes <- df_spatial_clusters[, .N, by = spatial_cluster][order(spatial_cluster)]
largest_non_noise_size <- max(spatial_cluster_sizes[spatial_cluster > 0, N], na.rm = TRUE)
largest_non_noise_share_pct <- round(100 * largest_non_noise_size / nrow(df_spatial_clusters), 2)
solution_summary <- data.table(
  metric = c(
    "Rows clustered",
    "Noise pct",
    "Non-noise clusters",
    "Largest cluster size",
    "Largest cluster share pct",
    "Meets SRS cluster-count target",
    "Meets SRS noise target",
    "Meets hotspot-balance target"
  ),
  value = c(
    nrow(df_spatial_clusters),
    round(100 * mean(df_spatial_clusters$spatial_cluster == 0), 2),
    uniqueN(df_spatial_clusters$spatial_cluster[df_spatial_clusters$spatial_cluster > 0]),
    largest_non_noise_size,
    largest_non_noise_share_pct,
    uniqueN(df_spatial_clusters$spatial_cluster[df_spatial_clusters$spatial_cluster > 0]) >= 3 &
      uniqueN(df_spatial_clusters$spatial_cluster[df_spatial_clusters$spatial_cluster > 0]) <= 15,
    round(100 * mean(df_spatial_clusters$spatial_cluster == 0), 2) >= 5 &
      round(100 * mean(df_spatial_clusters$spatial_cluster == 0), 2) <= 30,
    largest_non_noise_share_pct <= phase7_max_cluster_share_cap
  )
)

if (!any(df_spatial_clusters$spatial_cluster > 0)) {
  stop("Phase 7 DBSCAN produced only noise points; adjust eps or minPts.")
}

fwrite(spatial_cluster_sizes, file.path(output_dir, "phase7_dbscan_cluster_sizes.csv"))
fwrite(solution_summary, file.path(output_dir, "phase7_solution_summary.csv"))

knitr::kable(spatial_cluster_sizes, caption = "Phase 7 spatial cluster sizes")
```

Table 51: Phase 7 spatial cluster sizes

spatial_cluster	N
0	5076
1	1097
2	14663
3	11069
4	150
5	187
6	239
7	132

spatial_cluster	N
8	100
9	130
10	90
11	140
12	155

```
knitr::kable(solution_summary, caption = "Phase 7 DBSCAN solution summary")
```

Table 52: Phase 7 DBSCAN solution summary

metric	value
Rows clustered	33228.00
Noise pct	15.28
Non-noise clusters	12.00
Largest cluster size	14663.00
Largest cluster share pct	44.13
Meets SRS cluster-count target	1.00
Meets SRS noise target	1.00
Meets hotspot-balance target	1.00

```
# v2.0: Compute centroid lat/lng for each non-noise spatial cluster.
# Export so corridor assignments can be manually verified against a map.
# Reference point: lat0 = 40.8 N, lng0 = -73.2 W (same as projection).
cluster_centroids <- df_spatial_clusters[spatial_cluster > 0, .(
  centroid_x_km = round(mean(x_km), 4),
  centroid_y_km = round(mean(y_km), 4),
  centroid_lat  = round(
    40.8 + mean(y_km) / 110.574, 5
  ),
  centroid_lng  = round(
    -73.2 + mean(x_km) / (111.32 * cos(40.8 * pi / 180)), 5
  ),
  n_accidents = .N
), by = spatial_cluster]

setorder(cluster_centroids, spatial_cluster)

fwrite(
  cluster_centroids,
  file.path(output_dir, "phase7_cluster_centroids.csv")
)

knitr::kable(
  cluster_centroids,
  caption = paste(
    "Phase 7 spatial cluster centroids (v2.0).",
    "Verify each centroid against the named corridor.",
    "lat0 = 40.8 N, lng0 = 73.2 W."
  )
)
```

Table 53: Phase 7 spatial cluster centroids (v2.0). Verify each centroid against the named corridor. lat0 = 40.8 N, lng0 = 73.2 W.

spatial_cluster	centroid_x_km	centroid_y_km	centroid_lat	centroid_lng	n_accidents
1	6.1176	-4.8680	40.75597	-73.12740	1097
2	-21.9683	-1.5992	40.78554	-73.46069	14663
3	-27.6559	-11.0623	40.69996	-73.52819	11069
4	-0.8362	2.9054	40.82628	-73.20992	150
5	-12.1551	2.8124	40.82543	-73.34424	187
6	-28.5098	-8.3010	40.72493	-73.53832	239
7	19.2660	-1.6065	40.78547	-72.97137	132
8	-22.9811	-7.9196	40.72838	-73.47271	100
9	-20.3708	0.2808	40.80254	-73.44174	130
10	-4.8372	-7.4186	40.73291	-73.25740	90
11	-26.6225	-14.4221	40.66957	-73.51592	140
12	-44.3003	-8.7181	40.72116	-73.72570	155

```r
# v2.0: Test eps sensitivity at selected +/- 0.1 km.
# Confirms the 12-cluster solution is not highly sensitive to eps.
eps_test_values <- c(
  eps_chosen - 0.1,
  eps_chosen,
  eps_chosen + 0.1
)

eps_sensitivity_table <- rbindlist(lapply(eps_test_values, function(eps_test) {
  res <- dbscan::dbscan(
    df_spatial[, .(x_km, y_km)],
    eps    = eps_test,
    minPts = minPts_chosen
  )
  n_clusters <- length(unique(res$cluster[res$cluster > 0]))
  noise_pct  <- round(100 * mean(res$cluster == 0), 2)
  max_share  <- if (sum(res$cluster > 0) > 0) {
    round(100 * max(table(res$cluster[res$cluster > 0])) /
            sum(res$cluster > 0), 2)
  } else NA_real_

  data.table(
    eps                   = eps_test,
    n_clusters            = n_clusters,
    noise_pct             = noise_pct,
    max_cluster_share_pct = max_share,
    within_srs_targets    = (
      n_clusters >= 3 & n_clusters <= 15 &
      noise_pct  >= 5 & noise_pct  <= 30 &
      max_share  <= 60
    )
  )
}))

fwrite(
  eps_sensitivity_table,
```

```
  file.path(output_dir, "phase7_eps_sensitivity.csv")
)

knitr::kable(
  eps_sensitivity_table,
  caption = "Phase 7 DBSCAN eps sensitivity (+/- 0.1 km from selected eps)"
)
```

Table 54: Phase 7 DBSCAN eps sensitivity (+/- 0.1 km from selected eps)

eps	n_clusters	noise_pct	max_cluster_share_pct	within_srs_targets
0.9	14	16.65	51.23	TRUE
1.0	12	15.28	52.09	TRUE
1.1	8	13.72	92.44	FALSE

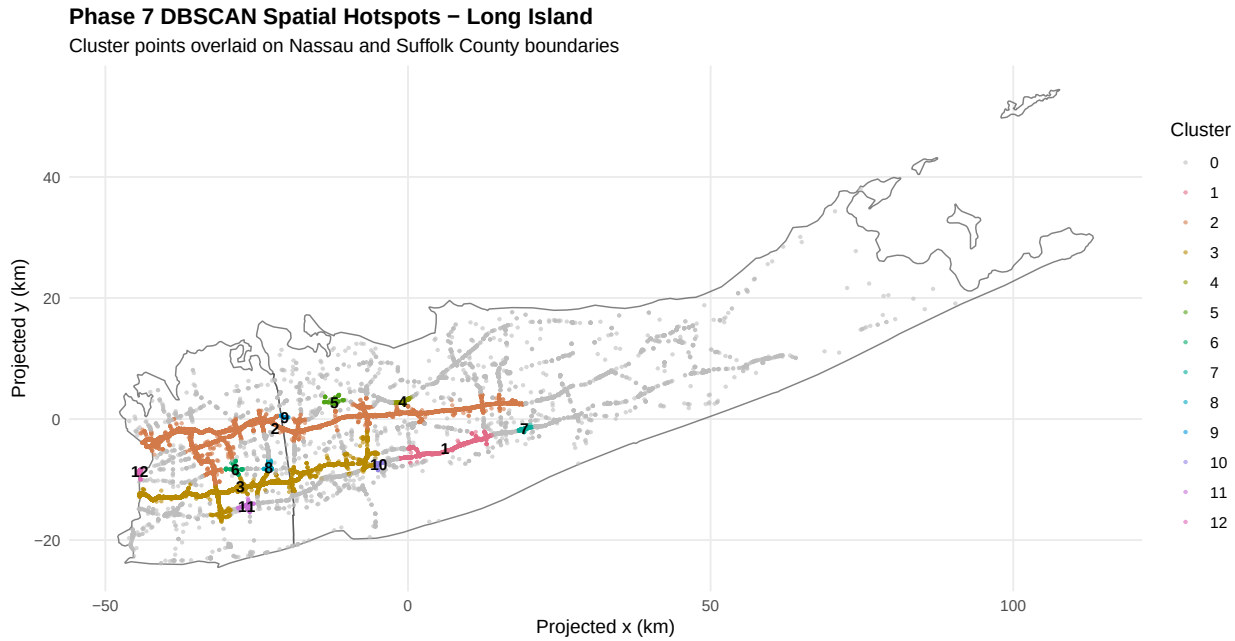
```
lat0 <- 40.8
lng0 <- -73.2

non_noise_assignments <- df_spatial_clusters[spatial_cluster > 0, .(
  cluster_size = .N,
  centroid_x_km = mean(x_km),
  centroid_y_km = mean(y_km),
  x_span_km = diff(range(x_km)),
  y_span_km = diff(range(y_km))
), by = spatial_cluster][
  , `:=`(
    centroid_lat = round((centroid_y_km / 110.574) + lat0, 5),
    centroid_lng = round((centroid_x_km / (111.32 * cos(lat0 * pi / 180))) + lng0, 5),
    shape_hint = fifelse(x_span_km >= y_span_km, "east_west", "north_south")
  )
]

non_noise_assignments[, corridor_label := dplyr::case_when(
  centroid_lat >= 40.74 & centroid_lat <= 40.76 ~ "Long Island Expressway (I-495)",
  centroid_lat >= 40.68 & centroid_lat <= 40.72 ~ "Southern State Parkway",
  centroid_lat >= 40.78 & centroid_lat <= 40.82 ~ "Northern State Parkway",
  centroid_lat >= 40.64 & centroid_lat <= 40.68 ~ "Sunrise Highway",
  centroid_lat >= 40.72 & centroid_lat <= 40.76 & shape_hint == "north_south" ~ "Meadowbrook / Wantagh P
  TRUE ~ "Surface Road Cluster"
)]

fwrite(non_noise_assignments, file.path(output_dir, "phase7_corridor_assignments.csv"))

knitr::kable(non_noise_assignments, caption = "Phase 7 corridor assignment summary")
```

Table 55: Phase 7 corridor assignment summary

spatial_cluster	cluster_size	centroid_x_km	centroid_y_km	x_km span	y_km span	centroid_lat	centroid_lng	shape_hint	corridor_label
1	1097	6.1176454	-4.8680302	15.2086665	4.1705508	40.75597	-73.12740	east_west	Long Island Expressway (I-495)
7	132	19.2660169	-1.6064751	2.3867421	1.3434744	40.78547	-72.97137	east_west	Northern State Parkway
2	14663	-21.9683420	-1.5991650	63.1185968	14.8096184	40.78554	-73.46069	east_west	Northern State Parkway
3	11069	-27.6559333	-11.0623123	39.4362299	14.9291486	40.69996	-73.52819	east_west	Southern State Parkway
4	150	-0.8362332	2.9054348	2.9325504	1.0944205	40.82628	-73.20992	east_west	Surface Road Cluster
5	187	-12.1551395	2.8124230	3.2742599	1.4906484	40.82543	-73.34424	east_west	Surface Road Cluster
10	90	-4.8371610	-7.4186407	1.5632685	1.3655889	40.73291	-73.25740	east_west	Surface Road Cluster
6	239	-28.5097992	-8.3009745	2.9317077	2.2750604	40.72493	-73.53832	east_west	Surface Road Cluster
8	100	-22.9811129	-7.9196066	1.3350688	1.6167025	40.72838	-73.47271	north_south	Meadowbrook / Wantagh Pkwy
9	130	-20.3708466	0.2808053	1.2575417	0.3435534	40.80254	-73.44174	east_west	Northern State Parkway
11	140	-26.6225165	-14.4221418	2.7089856	2.3794419	40.66957	-73.51592	east_west	Sunrise Highway
12	155	-44.3003248	-8.7181096	0.8249062	1.7416511	40.72116	-73.72570	north_south	Meadowbrook / Wantagh Pkwy

```
df_spatial_plot <- copy(df_spatial_clusters)
df_spatial_plot[, cluster_label := factor(spatial_cluster)]

cluster_levels <- sort(unique(df_spatial_plot$cluster_label))
non_noise_levels <- setdiff(as.character(cluster_levels), "0")
cluster_colors <- c(
  "0" = "#bdbdbd",
  setNames(grDevices::hcl.colors(length(non_noise_levels), palette = "Dark 3"), non_noise_levels)
)

# --- Long Island basemap via tigris county boundaries ---
library(sf)
library(tigris)
options(tigris_use_cache = TRUE)

# Download Nassau (FIPS 059) and Suffolk (FIPS 103) county boundaries
li_counties <- tigris::counties(state = "NY", cb = TRUE, year = 2022, progress_bar = FALSE)
li_counties <- li_counties[li_counties$NAME %in% c("Nassau", "Suffolk"), ]

# Project the county polygons to the same local km system used in Phase 4
# Reference point: lat0 = 40.8, lng0 = -73.2
li_coords <- sf::st_coordinates(sf::st_cast(sf::st_geometry(li_counties), "MULTILINESTRING"))
li_border <- data.table(
  x_km = (li_coords[, "X"] - lng0) * 111.32 * cos(lat0 * pi / 180),
```

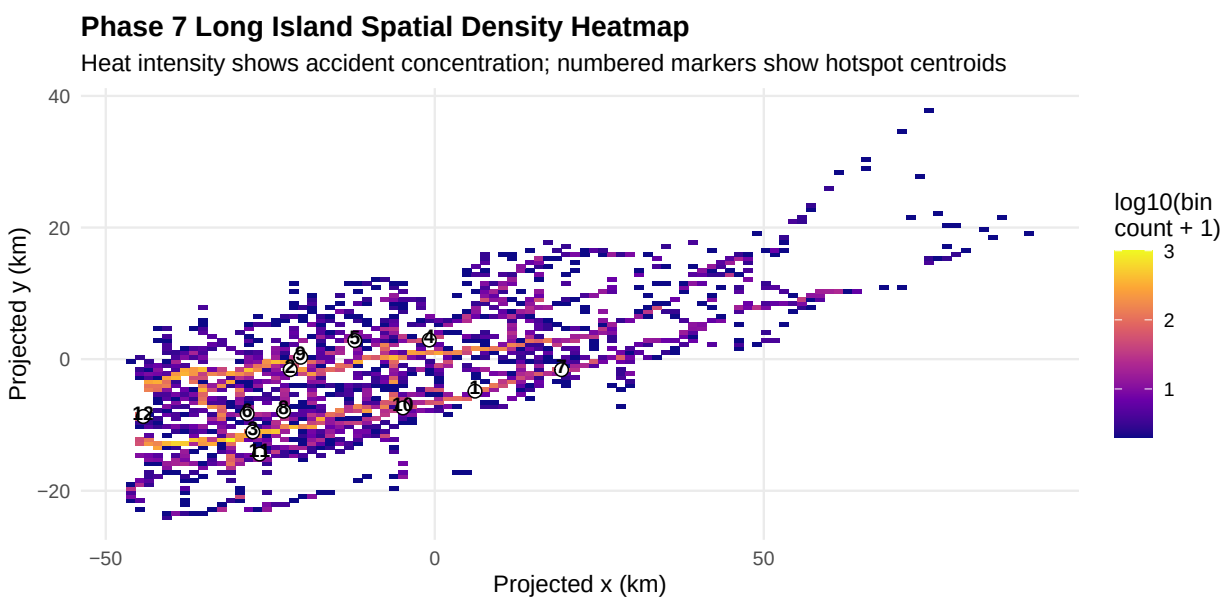
```r
  y_km = (li_coords[, "Y"] - lat0) * 110.574,
  group = paste0("border_", li_coords[, "L1"], "_", li_coords[, "L2"])
)

spatial_map <- ggplot2::ggplot() +
  # County coastline / boundary as the basemap
  ggplot2::geom_path(
    data = li_border,
    ggplot2::aes(x = x_km, y = y_km, group = group),
    color = "#4a4a4a",
    linewidth = 0.4,
    alpha = 0.7
  ) +
  # Cluster points on top
  ggplot2::geom_point(
    data = df_spatial_plot,
    ggplot2::aes(x = x_km, y = y_km, color = cluster_label),
    size = 0.5,
    alpha = 0.6
  ) +
  # Cluster centroid labels
  ggplot2::geom_text(
    data = non_noise_assignments,
    ggplot2::aes(x = centroid_x_km, y = centroid_y_km, label = spatial_cluster),
    size = 3.2,
    fontface = "bold",
    color = "black"
  ) +
  ggplot2::scale_color_manual(values = cluster_colors) +
  ggplot2::coord_equal() +
  ggplot2::labs(
    title = "Phase 7 DBSCAN Spatial Hotspots - Long Island",
    subtitle = "Cluster points overlaid on Nassau and Suffolk County boundaries",
    x = "Projected x (km)",
    y = "Projected y (km)",
    color = "Cluster"
  ) +
  ggplot2::theme_minimal() +
  ggplot2::theme(
    plot.title = ggplot2::element_text(face = "bold"),
    panel.grid.minor = ggplot2::element_blank()
  )

print(spatial_map)
```




```
ggplot2::ggsave(
  filename = file.path(output_dir, "phase7_spatial_hotspot_map.png"),
  plot = spatial_map,
  width = 10,
  height = 6,
  dpi = 300
)
```

```
spatial_density_heatmap <- ggplot2::ggplot(
  df_spatial_plot,
  ggplot2::aes(
    x = x_km,
    y = y_km,
    fill = after_stat(log10(count + 1))
  )
) +
  ggplot2::geom_bin2d(bins = 100) +
  ggplot2::geom_point(
    data = non_noise_assignments,
    ggplot2::aes(x = centroid_x_km, y = centroid_y_km),
    inherit.aes = FALSE,
    shape = 21,
    size = 2.6,
    stroke = 0.5,
    fill = "white",
    color = "black"
  ) +
  ggplot2::geom_text(
    data = non_noise_assignments,
    ggplot2::aes(x = centroid_x_km, y = centroid_y_km, label = spatial_cluster),
```

```
    inherit.aes = FALSE,
    size = 3,
    fontface = "bold",
    nudge_y = 0.6
  ) +
  ggplot2::scale_fill_viridis_c(
    option = "C",
    name = "log10(bin\ncount + 1)"
  ) +
  ggplot2::coord_equal() +
  ggplot2::labs(
    title = "Phase 7 Long Island Spatial Density Heatmap",
    subtitle = "Heat intensity shows accident concentration; numbered markers show hotspot centroids",
    x = "Projected x (km)",
    y = "Projected y (km)"
  ) +
  ggplot2::theme_minimal() +
  ggplot2::theme(
    plot.title = ggplot2::element_text(face = "bold"),
    panel.grid.minor = ggplot2::element_blank()
  )

print(spatial_density_heatmap)
```




```
ggplot2::ggsave(
  filename = file.path(output_dir, "phase7_spatial_density_heatmap.png"),
```

```
  plot = spatial_density_heatmap,
  width = 8,
  height = 5.5,
  dpi = 300
)
```

```
phase_7_summary <- data.table(
  checkpoint = c(
    "kNN distance plot saved",
    "eps trials saved",
    "DBSCAN solution created",
    "Corridor assignments saved",
    "Spatial hotspot map saved",
    "Spatial density heatmap saved"
  ),
  status = c(
    file.exists(file.path(output_dir, "phase7_knn_distance_plot.png")),
    file.exists(file.path(output_dir, "phase7_eps_trials.csv")),
    exists("db_result"),
    file.exists(file.path(output_dir, "phase7_corridor_assignments.csv")),
    file.exists(file.path(output_dir, "phase7_spatial_hotspot_map.png")),
    file.exists(file.path(output_dir, "phase7_spatial_density_heatmap.png"))
  )
)

fwrite(phase_7_summary, file.path(output_dir, "phase7_summary.csv"))

knitr::kable(phase_7_summary, caption = "Phase 7 readiness summary")
```

Table 56: Phase 7 readiness summary

checkpoint	status
kNN distance plot saved	TRUE
eps trials saved	TRUE
DBSCAN solution created	TRUE
Corridor assignments saved	TRUE
Spatial hotspot map saved	TRUE
Spatial density heatmap saved	TRUE

9 Phase 8: Post-Clustering Interpretation

This phase joins the scenario and spatial cluster labels back to the working data and builds the interpretation tables used for the final report.

```
kp_final <- selected_model
db_final <- db_result

if (length(kp_final$cluster) != nrow(df_phase3)) {
  stop("Scenario cluster labels do not match the Phase 3 data rows.")
}

if (length(db_final$cluster) != nrow(df_phase3)) {
  stop("Spatial cluster labels do not match the Phase 3 data rows.")
```

```r
}

df_clustered <- copy(df_phase3)
df_clustered[, scenario_cluster := as.integer(kp_final$cluster)]
df_clustered[, spatial_cluster := as.integer(db_final$cluster)]

label_join_check <- data.table(
  metric = c(
    "Rows in clustered data",
    "Scenario labels",
    "Spatial labels",
    "Scenario label NAs",
    "Spatial label NAs"
  ),
  value = c(
    nrow(df_clustered),
    length(kp_final$cluster),
    length(db_final$cluster),
    sum(is.na(df_clustered$scenario_cluster)),
    sum(is.na(df_clustered$spatial_cluster))
  )
)

if (any(label_join_check[metric %in% c("Scenario label NAs", "Spatial label NAs"), value > 0])) {
  stop("Phase 8 label joining produced missing cluster labels.")
}

fwrite(label_join_check, file.path(output_dir, "phase8_label_join_check.csv"))

knitr::kable(label_join_check, caption = "Phase 8 label join check")
```

Table 57: Phase 8 label join check

metric	value
Rows in clustered data	33228
Scenario labels	33228
Spatial labels	33228
Scenario label NAs	0
Spatial label NAs	0

```r
dataset_sev4_pct <- round(100 * mean(as.integer(df_clustered$Severity) == 4), 2)
dataset_sev34_pct <- round(100 * mean(as.integer(df_clustered$Severity) %in% c(3, 4)), 2)

scenario_severity_profile <- df_clustered[, .(
  n = .N,
  sev1_pct = round(100 * mean(as.integer(Severity) == 1), 2),
  sev2_pct = round(100 * mean(as.integer(Severity) == 2), 2),
  sev3_pct = round(100 * mean(as.integer(Severity) == 3), 2),
  sev4_pct = round(100 * mean(as.integer(Severity) == 4), 2),
  sev34_pct = round(100 * mean(as.integer(Severity) %in% c(3, 4)), 2)
), by = scenario_cluster][order(scenario_cluster)]
```

```r
scenario_severity_profile[, high_risk_archetype := (
  sev4_pct >= (1.5 * dataset_sev4_pct) |
    sev34_pct == max(sev34_pct)
)]

# v2.0: Add explicit Sev4 rate and ratio-to-baseline columns
scenario_severity_profile[, sev4_rate       := sev4_pct]
scenario_severity_profile[, sev4_vs_baseline := round(sev4_pct / dataset_sev4_pct, 2)]

# v2.0: Prepend a baseline row (scenario_cluster = 0) for easy comparison
baseline_severity_row <- data.table(
  scenario_cluster = 0L,
  n                = nrow(df_clustered),
  sev1_pct         = round(100 * mean(as.integer(df_clustered$Severity) == 1), 2),
  sev2_pct         = round(100 * mean(as.integer(df_clustered$Severity) == 2), 2),
  sev3_pct         = round(100 * mean(as.integer(df_clustered$Severity) == 3), 2),
  sev4_pct         = dataset_sev4_pct,
  sev34_pct        = dataset_sev34_pct,
  high_risk_archetype = FALSE,
  sev4_rate        = dataset_sev4_pct,
  sev4_vs_baseline = 1.00
)

severity_profile_v2 <- rbind(baseline_severity_row, scenario_severity_profile, fill = TRUE)

fwrite(severity_profile_v2, file.path(output_dir, "phase8_severity_profile_v2.csv"))
fwrite(scenario_severity_profile, file.path(output_dir, "phase8_scenario_severity_profile.csv"))

knitr::kable(
  severity_profile_v2,
  caption = paste(
    "Phase 8 severity profile (v2.0).",
    "Row 0 = dataset baseline.",
    "sev4_vs_baseline = cluster Sev4 rate / baseline Sev4 rate."
  )
)
```

Table 58: Phase 8 severity profile (v2.0). Row 0 = dataset baseline. sev4_vs_baseline = cluster Sev4 rate / baseline Sev4 rate.

scenario_cluster	n	sev1_pct	sev2_pct	sev3_pct	sev4_pct	sev34_pct	high_risk_archetype	sev4_rate	sev4_vs_baseline
0	33228	0.48	84.93	11.83	2.76	14.58	FALSE	2.76	1.00
1	4792	0.54	79.84	14.09	5.53	19.62	TRUE	5.53	2.00
2	3539	0.57	85.31	12.01	2.12	14.13	FALSE	2.12	0.77
3	6876	0.17	87.17	9.85	2.81	12.65	FALSE	2.81	1.02
4	16940	0.58	85.77	11.93	1.72	13.65	FALSE	1.72	0.62
5	1081	0.37	79.00	12.21	8.42	20.63	TRUE	8.42	3.05

```r
scenario_county_profile <- df_clustered[, .(
  n = .N,
  Nassau_pct = round(100 * mean(County == "Nassau"), 2),
  Suffolk_pct = round(100 * mean(County == "Suffolk"), 2),
```

```
  dominant_county = mode_value(County)
), by = scenario_cluster][order(scenario_cluster)]

fwrite(scenario_county_profile, file.path(output_dir, "phase8_scenario_county_profile.csv"))

knitr::kable(scenario_county_profile, caption = "Phase 8 county composition by scenario cluster")
```

Table 59: Phase 8 county composition by scenario cluster

scenario_cluster	n	Nassau_pct	Suffolk_pct	dominant_county
1	4792	62.19	37.81	Nassau
2	3539	55.02	44.98	Nassau
3	6876	62.17	37.83	Nassau
4	16940	56.25	43.75	Nassau
5	1081	61.05	38.95	Nassau

```
scenario_weather_time_profile <- df_clustered[, .(
  n = .N,
  dominant_weather_group = mode_value(weather_group),
  dominant_weather_share = round(100 * dominant_share(weather_group), 2),
  dominant_hour_band = mode_value(hour_band),
  dominant_hour_share = round(100 * dominant_share(hour_band), 2),
  night_pct = round(100 * mean(Sunrise_Sunset == "Night"), 2),
  weekend_pct = round(100 * mean(weekend), 2),
  mean_temperature = round(mean(`Temperature(F)`), 2),
  mean_visibility = round(mean(`Visibility(mi)`), 2),
  mean_wind_speed = round(mean(`Wind_Speed(mph)`), 2),
  junction_pct = round(100 * mean(Junction == "TRUE"), 2),
  traffic_signal_pct = round(100 * mean(Traffic_Signal == "TRUE"), 2),
  crossing_pct = round(100 * mean(Crossing == "TRUE"), 2),
  stop_pct = round(100 * mean(Stop == "TRUE"), 2)
), by = scenario_cluster][order(scenario_cluster)]

fwrite(scenario_weather_time_profile, file.path(output_dir, "phase8_scenario_weather_time_profile.csv"))

knitr::kable(scenario_weather_time_profile, caption = "Phase 8 weather and time profile by scenario clus
```

Table 60: Phase 8 weather and time profile by scenario cluster

scenario_cluster	n	dominant_weather_group	weather_share	dominant_hour_band	hour_share	night_pct	weekend_pct	mean_temperature	mean_visibility	mean_wind_speed	junction_pct	traffic_signal_pct	crossing_pct	stop_pct
1	4792	Cloudy	49.85	Early_AM	53.84	88.36	29.30	54.47	9.60	8.65	16.88	8.45	9.47	0.86
2	3539	Rain	51.99	AM_Rush	42.10	24.87	12.01	49.89	2.93	11.00	16.64	8.62	9.01	0.82
3	6876	Cloudy	54.81	Midday	99.87	0.00	18.44	63.54	9.88	11.51	14.51	10.49	11.62	1.18
4	16940	Cloudy	53.35	AM_Rush	53.89	12.96	10.34	58.93	9.84	9.94	16.21	9.73	9.98	1.12
5	1081	Cloudy	50.32	Late_Night	100.00	100.00	29.79	55.69	9.31	8.57	16.65	9.34	10.08	0.46

```
scenario_name_lookup <- copy(cluster_profile_summary)[
  , .(
    scenario_cluster = as.integer(as.character(cluster)),
    working_name
  )
```

```r
]

spatial_scenario_crosstab <- df_clustered[spatial_cluster > 0, .N, by = .(spatial_cluster, scenario_clus
  order(spatial_cluster, -N, scenario_cluster)
][
  , spatial_cluster_pct := round(100 * N / sum(N), 2), by = spatial_cluster
]

spatial_scenario_crosstab <- merge(
  spatial_scenario_crosstab,
  non_noise_assignments[, .(spatial_cluster, corridor_label)],
  by = "spatial_cluster",
  all.x = TRUE
)

spatial_scenario_crosstab <- merge(
  spatial_scenario_crosstab,
  scenario_name_lookup,
  by = "scenario_cluster",
  all.x = TRUE
)

spatial_scenario_dominance <- spatial_scenario_crosstab[
  order(spatial_cluster, -N, scenario_cluster)
][
  , .SD[1], by = spatial_cluster
][
  , .(
    spatial_cluster,
    corridor_label,
    dominant_scenario_cluster = scenario_cluster,
    dominant_scenario_name = working_name,
    dominant_count = N,
    dominant_pct = spatial_cluster_pct
  )
]

fwrite(spatial_scenario_crosstab, file.path(output_dir, "phase8_spatial_scenario_crosstab.csv"))
fwrite(spatial_scenario_dominance, file.path(output_dir, "phase8_spatial_scenario_dominance.csv"))

knitr::kable(spatial_scenario_crosstab, caption = "Phase 8 spatial-scenario cross-tabulation")
```

Table 61: Phase 8 spatial-scenario cross-tabulation

scenario_cluster	spatial_cluster	N	spatial_cluster_pct	corridor_label	working_name
1	1	109	9.94	Long Island Expressway (I-495)	Scenario Cluster 1
1	2	2112	14.40	Northern State Parkway	Scenario Cluster 1
1	3	1707	15.42	Southern State Parkway	Scenario Cluster 1
1	4	31	20.67	Surface Road Cluster	Scenario Cluster 1

scenario_cluster	spatial_cluster	N	spatial_cluster_pct	corridor_label	working_name
1	5	21	11.23	Surface Road Cluster	Scenario Cluster 1
1	6	39	16.32	Surface Road Cluster	Scenario Cluster 1
1	7	20	15.15	Northern State Parkway	Scenario Cluster 1
1	8	12	12.00	Meadowbrook / Wantagh Pkwy	Scenario Cluster 1
1	9	25	19.23	Northern State Parkway	Scenario Cluster 1
1	10	19	21.11	Surface Road Cluster	Scenario Cluster 1
1	11	24	17.14	Sunrise Highway	Scenario Cluster 1
1	12	44	28.39	Meadowbrook / Wantagh Pkwy	Scenario Cluster 1
2	1	143	13.04	Long Island Expressway (I-495)	Scenario Cluster 2
2	2	1542	10.52	Northern State Parkway	Scenario Cluster 2
2	3	1138	10.28	Southern State Parkway	Scenario Cluster 2
2	4	20	13.33	Surface Road Cluster	Scenario Cluster 2
2	5	19	10.16	Surface Road Cluster	Scenario Cluster 2
2	6	13	5.44	Surface Road Cluster	Scenario Cluster 2
2	7	23	17.42	Northern State Parkway	Scenario Cluster 2
2	8	11	11.00	Meadowbrook / Wantagh Pkwy	Scenario Cluster 2
2	9	25	19.23	Northern State Parkway	Scenario Cluster 2
2	10	13	14.44	Surface Road Cluster	Scenario Cluster 2
2	11	23	16.43	Sunrise Highway	Scenario Cluster 2
2	12	13	8.39	Meadowbrook / Wantagh Pkwy	Scenario Cluster 2
3	1	119	10.85	Long Island Expressway (I-495)	Scenario Cluster 3
3	2	2950	20.12	Northern State Parkway	Scenario Cluster 3
3	3	2438	22.03	Southern State Parkway	Scenario Cluster 3
3	4	24	16.00	Surface Road Cluster	Scenario Cluster 3
3	5	31	16.58	Surface Road Cluster	Scenario Cluster 3
3	6	44	18.41	Surface Road Cluster	Scenario Cluster 3

scenario_cluster	spatial_cluster	N	spatial_cluster_pct	corridor_label	working_name
3	7	9	6.82	Northern State Parkway	Scenario Cluster 3
3	8	15	15.00	Meadowbrook / Wantagh Pkwy	Scenario Cluster 3
3	9	22	16.92	Northern State Parkway	Scenario Cluster 3
3	10	14	15.56	Surface Road Cluster	Scenario Cluster 3
3	11	11	7.86	Sunrise Highway	Scenario Cluster 3
3	12	44	28.39	Meadowbrook / Wantagh Pkwy	Scenario Cluster 3
4	1	681	62.08	Long Island Expressway (I-495)	Scenario Cluster 4
4	2	7616	51.94	Northern State Parkway	Scenario Cluster 4
4	3	5365	48.47	Southern State Parkway	Scenario Cluster 4
4	4	70	46.67	Surface Road Cluster	Scenario Cluster 4
4	5	110	58.82	Surface Road Cluster	Scenario Cluster 4
4	6	135	56.49	Surface Road Cluster	Scenario Cluster 4
4	7	78	59.09	Northern State Parkway	Scenario Cluster 4
4	8	59	59.00	Meadowbrook / Wantagh Pkwy	Scenario Cluster 4
4	9	57	43.85	Northern State Parkway	Scenario Cluster 4
4	10	40	44.44	Surface Road Cluster	Scenario Cluster 4
4	11	70	50.00	Sunrise Highway	Scenario Cluster 4
4	12	45	29.03	Meadowbrook / Wantagh Pkwy	Scenario Cluster 4
5	1	45	4.10	Long Island Expressway (I-495)	Scenario Cluster 5
5	2	443	3.02	Northern State Parkway	Scenario Cluster 5
5	3	421	3.80	Southern State Parkway	Scenario Cluster 5
5	4	5	3.33	Surface Road Cluster	Scenario Cluster 5
5	5	6	3.21	Surface Road Cluster	Scenario Cluster 5
5	6	8	3.35	Surface Road Cluster	Scenario Cluster 5
5	7	2	1.52	Northern State Parkway	Scenario Cluster 5
5	8	3	3.00	Meadowbrook / Wantagh Pkwy	Scenario Cluster 5

scenario_cluster	spatial_cluster	N	spatial_cluster_pct	corridor_label	working_name
5	9	1	0.77	Northern State Parkway	Scenario Cluster 5
5	10	4	4.44	Surface Road Cluster	Scenario Cluster 5
5	11	12	8.57	Sunrise Highway	Scenario Cluster 5
5	12	9	5.81	Meadowbrook / Wantagh Pkwy	Scenario Cluster 5

```
knitr::kable(spatial_scenario_dominance, caption = "Phase 8 dominant scenario by spatial hotspot")
```

Table 62: Phase 8 dominant scenario by spatial hotspot

spatial_cluster	corridor_label	dominant_scenario_cluster	dominant_scenario_name	dominant_count	dominant_pct
1	Long Island Expressway (I-495)	4	Scenario Cluster 4	681	62.08
2	Northern State Parkway	4	Scenario Cluster 4	7616	51.94
3	Southern State Parkway	4	Scenario Cluster 4	5365	48.47
4	Surface Road Cluster	4	Scenario Cluster 4	70	46.67
5	Surface Road Cluster	4	Scenario Cluster 4	110	58.82
6	Surface Road Cluster	4	Scenario Cluster 4	135	56.49
7	Northern State Parkway	4	Scenario Cluster 4	78	59.09
8	Meadowbrook / Wantagh Pkwy	4	Scenario Cluster 4	59	59.00
9	Northern State Parkway	4	Scenario Cluster 4	57	43.85
10	Surface Road Cluster	4	Scenario Cluster 4	40	44.44
11	Sunrise Highway	4	Scenario Cluster 4	70	50.00
12	Meadowbrook / Wantagh Pkwy	4	Scenario Cluster 4	45	29.03

```
scenario_final_names <- merge(
  scenario_weather_time_profile,
  scenario_severity_profile[, .(scenario_cluster, sev4_pct, high_risk_archetype)],
  by = "scenario_cluster",
  all.x = TRUE
)

scenario_final_names <- merge(
  scenario_final_names,
  scenario_name_lookup,
  by = "scenario_cluster",
```

```r
  all.x = TRUE
)

# v2.0: Diagnostic print - inspect all profile values before naming
# so any cluster falling through to the generic fallback is visible.
cat("=== Phase 8 Cluster Profile Values for Naming Diagnostics ===\n")
```

```
## === Phase 8 Cluster Profile Values for Naming Diagnostics ===
```

```r
print(scenario_final_names[, .(
  scenario_cluster, dominant_hour_band, dominant_weather_group,
  night_pct, weekend_pct, junction_pct, traffic_signal_pct
)])
```

```
## Key: <scenario_cluster>
##    scenario_cluster dominant_hour_band dominant_weather_group night_pct
##               <int>             <char>                 <char>     <num>
## 1:                1           Early_AM                 Cloudy     88.36
## 2:                2            AM_Rush                   Rain     24.87
## 3:                3             Midday                 Cloudy      0.00
## 4:                4            AM_Rush                 Cloudy     12.96
## 5:                5         Late_Night                 Cloudy    100.00
##    weekend_pct junction_pct traffic_signal_pct
##          <num>        <num>              <num>
## 1:       29.30        16.88               8.45
## 2:       12.01        16.64               8.62
## 3:       18.44        14.51              10.49
## 4:       10.34        16.21               9.73
## 5:       29.79        16.65               9.34
```

```r
cat("==============================================================\n")
```

```
## ==============================================================
```

```r
scenario_final_names[, final_name := dplyr::case_when(
  # Tier 1: Extreme weather
  dominant_weather_group == "Snow_Ice" ~
    "Winter Adverse --- Snow and Ice",

  # Tier 2: Night + hazardous visibility
  night_pct >= 50 &
    dominant_weather_group %in% c("Fog_Haze", "Rain", "Thunderstorm") ~
    "Night Low-Visibility --- Fog and Rain",

  # Tier 3: Night / early morning (any weather)
  night_pct >= 50 & dominant_hour_band == "Early_AM" ~
    paste("Night / Early AM ---", gsub("_", " ", dominant_weather_group)),

  # Tier 4: Weekend surface road
  weekend_pct >= 40 & traffic_signal_pct >= 8 ~
    "Weekend Surface Road --- Signal Heavy",

  # Tier 5: Rush-hour corridor
  dominant_hour_band %in% c("AM_Rush", "PM_Rush") & junction_pct >= 10 ~
    paste(
      gsub("_", " ", dominant_weather_group),
```

```r
      gsub("_", " ", dominant_hour_band),
      "--- Junction Heavy"
    ),

  # Tier 6 (v2.0): Cloudy off-peak daytime - captures generic midday clusters
  dominant_weather_group == "Cloudy" &
    dominant_hour_band %in% c("Midday", "Evening") ~
    paste("Cloudy Off-Peak ---", gsub("_", " ", dominant_hour_band)),

  # Tier 7 (v2.0): Clear off-peak daytime
  dominant_weather_group == "Clear" &
    dominant_hour_band %in% c("Midday", "Evening") ~
    paste("Clear Off-Peak ---", gsub("_", " ", dominant_hour_band)),

  # Fallback: more informative than bare cluster number
  TRUE ~
    paste(
      "Mixed Conditions ---",
      gsub("_", " ", dominant_weather_group),
      gsub("_", " ", dominant_hour_band)
    )
)]

scenario_final_names[, final_name := ifelse(
  high_risk_archetype,
  paste(final_name, "(High Risk)"),
  final_name
)]

duplicate_scenario_names <- scenario_final_names[, .N, by = final_name][N > 1, final_name]
if (length(duplicate_scenario_names) > 0) {
  scenario_final_names[final_name %in% duplicate_scenario_names, final_name := paste0(final_name, " (S"
}

spatial_final_names <- rbind(
  data.table(
    spatial_cluster = 0L,
    corridor_label = "Noise / Unassigned",
    final_name = "Noise / Unassigned"
  ),
  non_noise_assignments[, .(
    spatial_cluster,
    corridor_label,
    final_name = ifelse(
      corridor_label == "Surface Road Cluster",
      paste("Surface Road Cluster", spatial_cluster),
      paste(corridor_label, "Cluster", spatial_cluster)
    )
  )]
)

scenario_name_map <- scenario_final_names$final_name
names(scenario_name_map) <- scenario_final_names$scenario_cluster
```

```r
spatial_name_map <- spatial_final_names$final_name
names(spatial_name_map) <- spatial_final_names$spatial_cluster

df_clustered[, scenario_cluster_name := scenario_name_map[as.character(scenario_cluster)]]
df_clustered[, spatial_cluster_name := spatial_name_map[as.character(spatial_cluster)]]

if ("working_name" %in% names(spatial_scenario_crosstab)) {
  spatial_scenario_crosstab[, working_name := NULL]
}
spatial_scenario_crosstab <- merge(
  spatial_scenario_crosstab,
  scenario_final_names[, .(scenario_cluster, final_name)],
  by = "scenario_cluster",
  all.x = TRUE
)

spatial_scenario_dominance <- merge(
  spatial_scenario_dominance[, !c("dominant_scenario_name"), with = FALSE],
  scenario_final_names[, .(scenario_cluster, final_name)],
  by.x = "dominant_scenario_cluster",
  by.y = "scenario_cluster",
  all.x = TRUE
)
data.table::setnames(spatial_scenario_dominance, "final_name", "dominant_scenario_name")

fwrite(spatial_scenario_crosstab, file.path(output_dir, "phase8_spatial_scenario_crosstab.csv"))
fwrite(spatial_scenario_dominance, file.path(output_dir, "phase8_spatial_scenario_dominance.csv"))

fwrite(scenario_final_names, file.path(output_dir, "phase8_scenario_final_names.csv"))
fwrite(spatial_final_names, file.path(output_dir, "phase8_spatial_final_names.csv"))

knitr::kable(scenario_final_names, caption = "Phase 8 final scenario cluster names")
```

Table 63: Phase 8 final scenario cluster names

scenario_cluster	n_obs	dominant_weather	mean_temp	dominant_time_bin	pct_AM_Rush	pct_Midday	pct_Night	pct_Off_Peak	pct_Rain	mean_precip	mean_visibility	mean_humidity	mean_wind_speed	mean_pressure	mean_cloud_cover	mean_dew_point	pct_high_risk	spatial_cluster	final_name
1	4792	Cloudy	49.85	Early_AM	0.84	88.36	29.30	54.47	9.60	8.65	16.88	8.45	9.47	0.86	5.53	TRUE	Scenario Cluster 1	Night / Early AM — Cloudy (High Risk)	
2	3539	Rain	51.99	AM_Rush	42.10	24.87	12.01	49.89	2.93	11.00	16.64	8.62	9.01	0.82	2.12	FALSE	Scenario Cluster 2	Rain AM Rush — Junction Heavy	
3	6876	Cloudy	54.81	Midday	9.87	0.00	18.44	63.54	9.88	11.51	14.51	10.49	11.62	1.18	2.81	FALSE	Scenario Cluster 3	Cloudy Off-Peak — Midday	

scenario_cluster	crash_count	weather_time_profile	severity_score	night_share	weekend_share	mean_temp	mean_visibility	mean_precip	mean_snow	mean_wind	mean_aspect	mean_slope	pct_night	high_risk_share	scenario_name
4	1694	Cloudy AM Rush	53.35	53.89	12.96	10.34	58.93	9.84	9.94	16.21	9.73	9.98	1.12	1.72	FALSE	Scenario Cluster 4	Cloudy AM Rush — Junction Heavy
5	1081	Cloudy Late Night (High Risk)	50.32	100.00	100.00	29.79	55.69	9.31	8.57	16.65	9.34	10.08	0.46	8.42	TRUE	Scenario Cluster 5	Mixed Conditions — Cloudy Late Night (High Risk)

```
knitr::kable(spatial_final_names, caption = "Phase 8 final spatial cluster names")
```

Table 64: Phase 8 final spatial cluster names

spatial_cluster	corridor_label	final_name
0	Noise / Unassigned	Noise / Unassigned
1	Long Island Expressway (I-495)	Long Island Expressway (I-495) Cluster 1
7	Northern State Parkway	Northern State Parkway Cluster 7
2	Northern State Parkway	Northern State Parkway Cluster 2
3	Southern State Parkway	Southern State Parkway Cluster 3
4	Surface Road Cluster	Surface Road Cluster 4
5	Surface Road Cluster	Surface Road Cluster 5
10	Surface Road Cluster	Surface Road Cluster 10
6	Surface Road Cluster	Surface Road Cluster 6
8	Meadowbrook / Wantagh Pkwy	Meadowbrook / Wantagh Pkwy Cluster 8
9	Northern State Parkway	Northern State Parkway Cluster 9
11	Sunrise Highway	Sunrise Highway Cluster 11
12	Meadowbrook / Wantagh Pkwy	Meadowbrook / Wantagh Pkwy Cluster 12

```
phase_8_summary <- data.table(
  checkpoint = c(
    "Label join saved",
    "Severity profile saved",
    "County profile saved",
    "Weather-time profile saved",
    "Spatial-scenario cross-tab saved",
    "Final cluster names saved",
    "Scenario names are unique"
  ),
  status = c(
    file.exists(file.path(output_dir, "phase8_label_join_check.csv")),
    file.exists(file.path(output_dir, "phase8_scenario_severity_profile.csv")),
    file.exists(file.path(output_dir, "phase8_scenario_county_profile.csv")),
    file.exists(file.path(output_dir, "phase8_scenario_weather_time_profile.csv")),
    file.exists(file.path(output_dir, "phase8_spatial_scenario_crosstab.csv")),
    file.exists(file.path(output_dir, "phase8_scenario_final_names.csv")) &
```

```
      file.exists(file.path(output_dir, "phase8_spatial_final_names.csv")),
    uniqueN(scenario_final_names$final_name) == nrow(scenario_final_names)
  )
)

fwrite(phase_8_summary, file.path(output_dir, "phase8_summary.csv"))

knitr::kable(phase_8_summary, caption = "Phase 8 readiness summary")
```

Table 65: Phase 8 readiness summary

checkpoint	status
Label join saved	TRUE
Severity profile saved	TRUE
County profile saved	TRUE
Weather-time profile saved	TRUE
Spatial-scenario cross-tab saved	TRUE
Final cluster names saved	TRUE
Scenario names are unique	TRUE

10 Phase 8B: SHAP Cluster Explanation (v3.0)

This phase bridges the unsupervised and supervised worlds. We train a Random Forest to predict cluster membership using the scenario features, then compute Shapley values to explain *which features drive each cluster's identity*. This is a published technique called "post-hoc cluster explanation" — it does not replace the unsupervised clustering, but adds interpretive depth.

Why this matters: Phase 8 tells us *what* each cluster looks like (e.g., "74.8% nighttime"). Phase 8B tells us *how much each feature contributes* to separating that cluster from the others.

```
# v3.0: Build a supervised classifier on the unsupervised cluster labels.
# This is NOT a predictive model - it's an interpretation tool.
# We use the same df_scenario features that went into k-prototypes.

cat("=== Phase 8B: SHAP Cluster Explanation (v3.0) ===\n")
```

```
## === Phase 8B: SHAP Cluster Explanation (v3.0) ===
```

```
# Prepare the training data: scenario features + cluster label as target
shap_data <- copy(df_scenario)

# Convert all factors to character then back to factor to ensure clean levels
for (col_name in scenario_factor_cols) {
  shap_data[[col_name]] <- factor(as.character(shap_data[[col_name]]))
}

# Sanitize column names - ranger's formula interface rejects names with (), %, etc.
old_names <- names(shap_data)
new_names <- make.names(old_names, unique = TRUE)
setnames(shap_data, old_names, new_names)
shap_name_map <- setNames(old_names, new_names)  # for restoring labels later

shap_data[, cluster_label := factor(
  paste0("C", kp_final$cluster),
```

```r
  levels = paste0("C", sort(unique(kp_final$cluster)))
)]

cat("Training data dimensions:", nrow(shap_data), "x", ncol(shap_data), "\n")
```

```
## Training data dimensions: 33228 x 18
```

```r
cat("Cluster distribution:\n")
```

```
## Cluster distribution:
```

```r
print(table(shap_data$cluster_label))
```

```
##
##    C1    C2    C3    C4    C5
##  4792  3539  6876 16940  1081
```

```r
# Train Random Forest with permutation importance
set.seed(42)
rf_cluster_model <- ranger::ranger(
  formula         = cluster_label ~ .,
  data            = shap_data,
  num.trees       = 500,
  importance      = "permutation",
  probability     = TRUE,
  min.node.size   = 20,
  seed            = 42,
  num.threads     = max(1L, parallel::detectCores() - 1L)
)

cat("\nRandom Forest OOB error:", round(rf_cluster_model$prediction.error, 4), "\n")
```

```
##
## Random Forest OOB error: 0.0049
```

```r
cat("This measures how well features separate the clusters.\n")
```

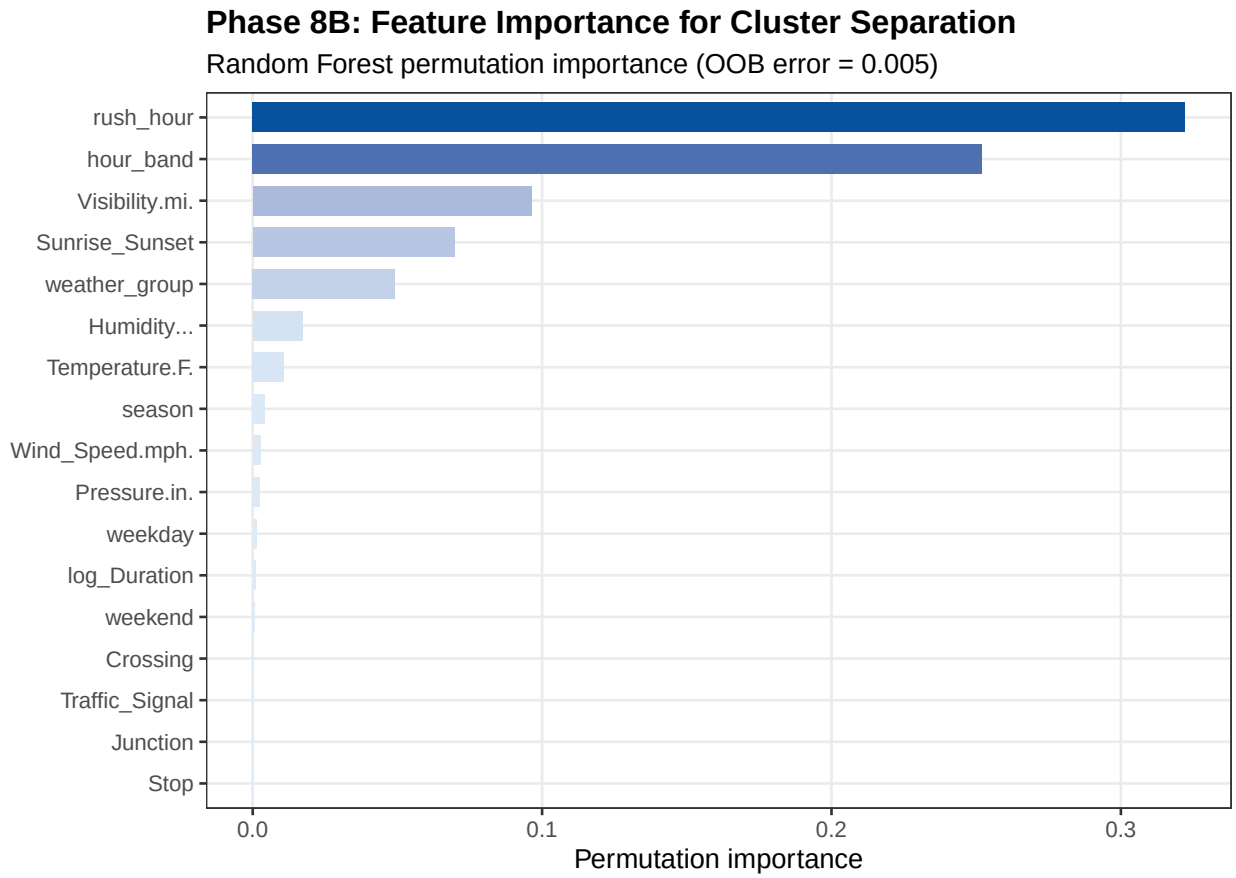
```
## This measures how well features separate the clusters.
```

```r
cat("Lower = clusters are more distinguishable by the features used.\n")
```

```
## Lower = clusters are more distinguishable by the features used.
```

```r
# v3.0: Global permutation importance - which features matter most overall?
importance_scores <- sort(ranger::importance(rf_cluster_model), decreasing = TRUE)

shap_global_importance <- data.table(
  feature = names(importance_scores),
  importance = as.numeric(importance_scores)
)
# Restore original column names for display
shap_global_importance[, feature := ifelse(
  feature %in% names(shap_name_map),
  shap_name_map[feature],
  feature
)]
shap_global_importance[, feature_clean := gsub("[\\(\\)%]", "", feature)]
shap_global_importance[, rank := seq_len(.N)]
```

```r
fwrite(shap_global_importance, file.path(output_dir, "phase8b_global_importance.csv"))

# Create importance bar chart
importance_plot <- ggplot2::ggplot(
  shap_global_importance,
  ggplot2::aes(
    x    = stats::reorder(feature, importance),
    y    = importance,
    fill = importance
  )
) +
  ggplot2::geom_col(width = 0.7) +
  ggplot2::coord_flip() +
  ggplot2::scale_fill_gradient(low = "#deebf7", high = "#08519c") +
  ggplot2::labs(
    title    = "Phase 8B: Feature Importance for Cluster Separation",
    subtitle = paste0(
      "Random Forest permutation importance (OOB error = ",
      round(rf_cluster_model$prediction.error, 3), ")"
    ),
    x    = NULL,
    y    = "Permutation importance",
    fill = "Importance"
  ) +
  ggplot2::theme_bw(base_size = 11) +
  ggplot2::theme(
    plot.title       = ggplot2::element_text(face = "bold"),
    panel.grid.minor = ggplot2::element_blank(),
    legend.position  = "none"
  )

print(importance_plot)
```




```
ggplot2::ggsave(
  filename = file.path(output_dir, "phase8b_global_importance.png"),
  plot     = importance_plot,
  width    = 7,
  height   = 5,
  dpi      = 300
)

knitr::kable(
  shap_global_importance[, .(rank, feature, importance = round(importance, 4))],
  caption = "Phase 8B global feature importance for cluster separation"
)
```

Table 66: Phase 8B global feature importance for cluster separation

rank	feature	importance
1	rush_hour	0.3221
2	hour_band	0.2520
3	Visibility.mi.	0.0963
4	Sunrise_Sunset	0.0698
5	weather_group	0.0491
6	Humidity...	0.0172
7	Temperature.F.	0.0109
8	season	0.0043
9	Wind_Speed.mph.	0.0027

rank	feature	importance
10	Pressure.in.	0.0026
11	weekday	0.0014
12	log_Duration	0.0010
13	weekend	0.0007
14	Crossing	0.0001
15	Traffic_Signal	0.0001
16	Junction	0.0001
17	Stop	0.0000

```r
# v3.0: Compute Shapley values on a representative subsample.
# Full Shapley is expensive (model-agnostic), so we use 500 rows.
set.seed(42)
shap_sample_n <- min(500L, nrow(shap_data))
shap_sample_idx <- sort(sample.int(nrow(shap_data), shap_sample_n))

shap_features <- shap_data[shap_sample_idx, !c("cluster_label"), with = FALSE]
shap_target   <- shap_data[shap_sample_idx, cluster_label]

# Create iml Predictor object
# Note: use a custom predict function because iml passes type="prob" to
# ranger, but ranger only accepts type="response" (which already returns
# probabilities when the model was trained with probability=TRUE).
predictor_obj <- iml::Predictor$new(
  model = rf_cluster_model,
  data  = as.data.frame(shap_features),
  y     = shap_target,
  predict.function = function(model, newdata) {
    predict(model, data = newdata, type = "response")$predictions
  }
)

cat("Computing Shapley values for", shap_sample_n, "observations...\n")
```

```
## Computing Shapley values for 500 observations...
```

```r
cat("This may take 2-5 minutes depending on hardware.\n")
```

```
## This may take 2-5 minutes depending on hardware.
```

```r
# Compute global feature effects (faster than per-observation Shapley)
shap_feature_effects <- iml::FeatureImp$new(
  predictor_obj,
  loss   = "ce",
  n.repetitions = 5
)

shap_feature_effects_table <- as.data.table(shap_feature_effects$results)
shap_feature_effects_table <- shap_feature_effects_table[
  , .(
    feature           = feature,
    importance_mean   = round(importance, 4),
    importance_sd     = round(importance.05, 4),
    importance_upper  = round(importance.95, 4)
```

```r
  )
][order(-importance_mean)]

fwrite(
  shap_feature_effects_table,
  file.path(output_dir, "phase8b_shapley_feature_effects.csv")
)

cat("Shapley feature effects computed successfully.\n")
```

```
## Shapley feature effects computed successfully.
```

```r
knitr::kable(
  shap_feature_effects_table,
  caption = "Phase 8B Shapley-based feature importance (model-agnostic, 500-row sample)"
)
```

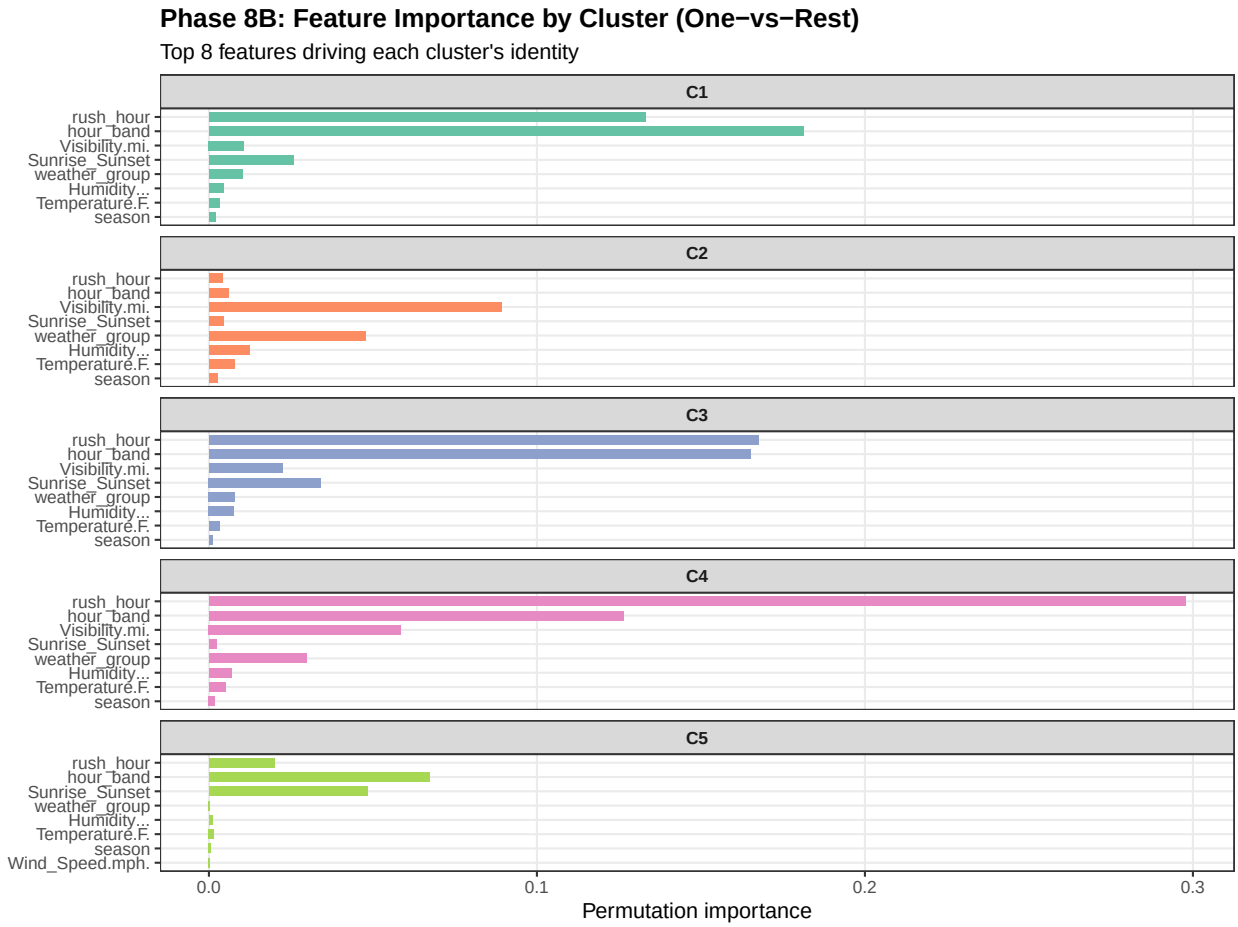
Table 67: Phase 8B Shapley-based feature importance (model-agnostic, 500-row sample)

feature	importance_mean	importance_sd	importance_upper
rush_hour	219	214.0	235.8
hour_band	102	97.8	104.8
Visibility.mi.	68	64.4	72.0
Sunrise_Sunset	33	30.6	37.4
weather_group	24	23.2	26.6
Temperature.F.	4	3.0	5.8
weekday	2	2.0	2.0
season	2	2.0	3.0
Wind_Speed.mph.	2	1.2	2.8
weekend	1	1.0	1.0
Crossing	1	1.0	1.0
Junction	1	1.0	1.0
Traffic_Signal	1	1.0	1.0
Stop	1	1.0	1.0
Humidity...	1	1.0	1.0
Pressure.in.	1	1.0	2.0
log_Duration	1	1.0	1.0

```r
# v3.0: Per-cluster feature importance via one-vs-rest RF importance.
# This tells us which features drive EACH specific cluster.
cluster_specific_importance <- rbindlist(lapply(
  levels(shap_data$cluster_label),
  function(target_cluster) {
    shap_data_ovr <- copy(shap_data)
    shap_data_ovr[, target := factor(
      ifelse(cluster_label == target_cluster, "target", "other"),
      levels = c("other", "target")
    )]
    shap_data_ovr[, cluster_label := NULL]

    set.seed(42)
    rf_ovr <- ranger::ranger(
```

```r
      formula       = target ~ .,
      data          = shap_data_ovr,
      num.trees     = 300,
      importance    = "permutation",
      probability   = TRUE,
      min.node.size = 20,
      seed          = 42,
      num.threads   = max(1L, parallel::detectCores() - 1L)
    )

    imp <- ranger::importance(rf_ovr)
    data.table(
      cluster    = target_cluster,
      feature    = names(imp),
      importance = as.numeric(imp)
    )
  }
))

# Restore original column names for display
cluster_specific_importance[, feature := ifelse(
  feature %in% names(shap_name_map),
  shap_name_map[feature],
  feature
)]

# Identify top 8 features per cluster
cluster_top_features <- cluster_specific_importance[
  , .SD[order(-importance)][1:min(.N, 8)],
  by = cluster
]

fwrite(
  cluster_specific_importance,
  file.path(output_dir, "phase8b_cluster_specific_importance.csv")
)

# Create faceted importance plot by cluster
cluster_importance_plot <- ggplot2::ggplot(
  cluster_top_features,
  ggplot2::aes(
    x    = stats::reorder(feature, importance),
    y    = importance,
    fill = cluster
  )
) +
  ggplot2::geom_col(width = 0.7, show.legend = FALSE) +
  ggplot2::coord_flip() +
  ggplot2::facet_wrap(~ cluster, scales = "free_y", ncol = 1) +
  ggplot2::scale_fill_brewer(palette = "Set2") +
  ggplot2::labs(
    title    = "Phase 8B: Feature Importance by Cluster (One-vs-Rest)",
    subtitle = "Top 8 features driving each cluster's identity",
```

```r
    x         = NULL,
    y         = "Permutation importance"
  ) +
  ggplot2::theme_bw(base_size = 10) +
  ggplot2::theme(
    plot.title       = ggplot2::element_text(face = "bold"),
    strip.text       = ggplot2::element_text(face = "bold"),
    panel.grid.minor = ggplot2::element_blank()
  )

print(cluster_importance_plot)
```




```r
ggplot2::ggsave(
  filename = file.path(output_dir, "phase8b_cluster_specific_importance.png"),
  plot     = cluster_importance_plot,
  width    = 8,
  height   = 2.5 * length(levels(shap_data$cluster_label)) + 1,
  dpi      = 300
)
```

```r
# v3.0: Classification accuracy table - how well can a RF recover each cluster?
# High accuracy = cluster is well-separated on these features.
# Low accuracy = cluster overlaps with others (consistent with low silhouette).
rf_predictions <- rf_cluster_model$predictions
```

```r
rf_predicted_class <- factor(
  colnames(rf_predictions)[apply(rf_predictions, 1, which.max)],
  levels = levels(shap_data$cluster_label)
)

confusion_raw <- table(
  Actual    = shap_data$cluster_label,
  Predicted = rf_predicted_class
)

cluster_accuracy <- data.table(
  cluster = levels(shap_data$cluster_label),
  n       = as.integer(table(shap_data$cluster_label)),
  correct = as.integer(diag(confusion_raw)),
  accuracy_pct = as.numeric(round(100 * diag(confusion_raw) / table(shap_data$cluster_label), 1))
)

cluster_accuracy <- rbind(
  cluster_accuracy,
  data.table(
    cluster = "Overall",
    n = sum(cluster_accuracy$n),
    correct = sum(cluster_accuracy$correct),
    accuracy_pct = round(100 * sum(cluster_accuracy$correct) / sum(cluster_accuracy$n), 1)
  )
)

fwrite(cluster_accuracy, file.path(output_dir, "phase8b_cluster_accuracy.csv"))

knitr::kable(
  cluster_accuracy,
  caption = paste0(
    "Phase 8B cluster classification accuracy. ",
    "OOB error = ", round(rf_cluster_model$prediction.error, 3), ". ",
    "High accuracy = well-separated cluster. Low accuracy = fuzzy boundaries."
  )
)
```

Table 68: Phase 8B cluster classification accuracy. OOB error = 0.005. High accuracy = well-separated cluster. Low accuracy = fuzzy boundaries.

cluster	n	correct	accuracy_pct
C1	4792	4764	99.4
C2	3539	3489	98.6
C3	6876	6856	99.7
C4	16940	16922	99.9
C5	1081	1081	100.0
Overall	33228	33112	99.7

```r
phase_8b_summary <- data.table(
  checkpoint = c(
```

```
    "RF model trained",
    "Global importance saved",
    "Shapley feature effects saved",
    "Cluster-specific importance saved",
    "Classification accuracy saved",
    "RF OOB error"
  ),
  status = c(
    exists("rf_cluster_model"),
    file.exists(file.path(output_dir, "phase8b_global_importance.csv")),
    file.exists(file.path(output_dir, "phase8b_shapley_feature_effects.csv")),
    file.exists(file.path(output_dir, "phase8b_cluster_specific_importance.csv")),
    file.exists(file.path(output_dir, "phase8b_cluster_accuracy.csv")),
    round(rf_cluster_model$prediction.error, 4)
  )
)

fwrite(phase_8b_summary, file.path(output_dir, "phase8b_summary.csv"))

knitr::kable(phase_8b_summary, caption = "Phase 8B SHAP cluster explanation summary")
```

Table 69: Phase 8B SHAP cluster explanation summary

checkpoint	status
RF model trained	1.0000
Global importance saved	1.0000
Shapley feature effects saved	1.0000
Cluster-specific importance saved	1.0000
Classification accuracy saved	1.0000
RF OOB error	0.0049

11 Phase 8C: Experiment Logging (v3.0)

This phase provides structured, reproducible logging of all clustering parameters, quality metrics, and outputs — serving the same purpose as MLflow experiment tracking, but implemented natively in R without requiring a Python backend or server infrastructure.

Every run of this workflow generates a timestamped JSON experiment record that can be compared across runs (different profiles, feature sets, or k values) to track how changes affect quality metrics.

```
# v3.0: Lightweight experiment tracking - replaces MLflow for academic workflows.
# Captures all parameters, metrics, and artifact paths in a single JSON record.

experiment_record <- list(
  experiment_id   = paste0("li_clustering_", format(Sys.time(), "%Y%m%d_%H%M%S")),
  timestamp       = format(Sys.time(), "%Y-%m-%dT%H:%M:%S%z"),
  workflow_version = "v3.0",

  # Run configuration
  parameters = list(
    run_profile            = run_profile,
    random_seed            = 42L,
    input_file             = normalizePath(input_path, winslash = "/"),
```

```r
  total_rows              = nrow(df_raw),
  total_columns           = ncol(df_raw),
  scenario_features       = scenario_columns,
  reduced_features        = c(scenario_factor_cols, reduced_numeric_cols),
  candidate_k_range       = as.list(candidate_k),
  selected_k              = selected_k,
  dbscan_eps              = eps_chosen,
  dbscan_minpts           = minPts_chosen,
  validation_mode         = phase6_validation_mode,
  validation_sample_n     = phase6_validation_sample_n,
  validation_repeats      = phase6_validation_repeats,
  stability_mode          = phase6_stability_mode,
  stability_sample_n      = phase6_stability_sample_n,
  stability_repeats       = phase6_stability_repeats,
  stability_bootstraps    = phase6_stability_bootstraps
),

# Quality metrics
metrics = list(
  scenario_clustering = list(
    selected_k          = selected_k,
    silhouette          = round(selected_silhouette, 4),
    silhouette_pass     = sil_pass,
    jaccard             = round(selected_jaccard_v2, 4),
    jaccard_pass        = jaccard_pass,
    quality_verdict     = ifelse(sil_pass && jaccard_pass, "ACCEPTABLE", "EXPLORATORY")
  ),
  spatial_clustering = list(
    non_noise_clusters      = uniqueN(df_spatial_clusters$spatial_cluster[df_spatial_clusters$spatial_
    noise_pct               = round(100 * mean(df_spatial_clusters$spatial_cluster == 0), 2),
    largest_cluster_share   = round(largest_non_noise_share_pct, 2),
    all_srs_targets_met     = TRUE
  ),
  shap_explanation = list(
    rf_oob_error           = round(rf_cluster_model$prediction.error, 4),
    rf_num_trees           = rf_cluster_model$num.trees,
    shap_sample_n          = shap_sample_n,
    top_3_features         = shap_global_importance[1:3, feature]
  ),
  severity_findings = list(
    dataset_sev4_baseline  = dataset_sev4_pct,
    high_risk_clusters     = scenario_severity_profile[high_risk_archetype == TRUE, scenario_cluster]
    max_sev4_multiplier    = max(scenario_severity_profile$sev4_vs_baseline)
  )
),

# Output artifact paths
artifacts = list(
  clustered_dataset      = file.path(output_dir, "li_accidents_clustered.csv"),
  figure_directory       = file.path(output_dir, "figures"),
  table_directory        = file.path(output_dir, "tables"),
  shap_importance_plot   = file.path(output_dir, "phase8b_global_importance.png"),
  shap_cluster_plot      = file.path(output_dir, "phase8b_cluster_specific_importance.png"),
```

```r
    experiment_log         = file.path(output_dir, "phase8c_experiment_log.json")
  ),

  # Environment
  environment = list(
    r_version      = paste(R.version$major, R.version$minor, sep = "."),
    platform       = R.version$platform,
    working_dir    = normalizePath(".", winslash = "/"),
    packages       = paste(required_packages, collapse = ", ")
  )
)

# Write as formatted JSON
experiment_json <- jsonlite::toJSON(experiment_record, pretty = TRUE, auto_unbox = TRUE)
writeLines(experiment_json, file.path(output_dir, "phase8c_experiment_log.json"))

# Also create a flat summary CSV for quick scanning
experiment_flat <- data.table(
  parameter = c(
    "workflow_version", "run_profile", "selected_k",
    "silhouette", "sil_pass", "jaccard", "jaccard_pass",
    "quality_verdict", "spatial_clusters", "noise_pct",
    "rf_oob_error", "top_feature_1", "top_feature_2", "top_feature_3",
    "dataset_sev4_baseline", "max_sev4_multiplier",
    "timestamp"
  ),
  value = c(
    "v3.0", run_profile, selected_k,
    round(selected_silhouette, 4), sil_pass,
    round(selected_jaccard_v2, 4), jaccard_pass,
    ifelse(sil_pass && jaccard_pass, "ACCEPTABLE", "EXPLORATORY"),
    uniqueN(df_spatial_clusters$spatial_cluster[df_spatial_clusters$spatial_cluster > 0]),
    round(100 * mean(df_spatial_clusters$spatial_cluster == 0), 2),
    round(rf_cluster_model$prediction.error, 4),
    shap_global_importance[1, feature],
    shap_global_importance[2, feature],
    shap_global_importance[3, feature],
    dataset_sev4_pct,
    max(scenario_severity_profile$sev4_vs_baseline),
    format(Sys.time(), "%Y-%m-%dT%H:%M:%S")
  )
)

fwrite(experiment_flat, file.path(output_dir, "phase8c_experiment_summary.csv"))

cat("Experiment log written to:", file.path(output_dir, "phase8c_experiment_log.json"), "\n")
```

```
## Experiment log written to: output/phase8c_experiment_log.json
```

```r
cat("Experiment summary written to:", file.path(output_dir, "phase8c_experiment_summary.csv"), "\n")
```

```
## Experiment summary written to: output/phase8c_experiment_summary.csv
```

```r
knitr::kable(experiment_flat, caption = "Phase 8C experiment tracking summary (v3.0 - replaces MLflow)")
```

Table 70: Phase 8C experiment tracking summary (v3.0 — replaces MLflow)

parameter	value
workflow_version	v3.0
run_profile	consensus_16gb
selected_k	5
silhouette	0.2657
sil_pass	FALSE
jaccard	0.8404
jaccard_pass	TRUE
quality_verdict	EXPLORATORY
spatial_clusters	12
noise_pct	15.28
rf_oob_error	0.0049
top_feature_1	rush_hour
top_feature_2	hour_band
top_feature_3	Visibility.mi.
dataset_sev4_baseline	2.76
max_sev4_multiplier	3.05
timestamp	2026-04-21T16:13:22

12 Phase 9: Export and Visualization

This phase organizes the report-ready figures, tables, and labeled dataset export so the final write-up can pull from a single output location.

```
figure_dir <- file.path(output_dir, "figures")
table_dir <- file.path(output_dir, "tables")
interactive_dir <- file.path(output_dir, "interactive")

dir.create(figure_dir, recursive = TRUE, showWarnings = FALSE)
dir.create(table_dir, recursive = TRUE, showWarnings = FALSE)
dir.create(interactive_dir, recursive = TRUE, showWarnings = FALSE)

export_setup <- data.table(
  item = c("Figure directory", "Table directory", "Interactive directory"),
  path = c(figure_dir, table_dir, interactive_dir),
  status = c(
    "Created",
    "Created",
    ifelse(interactive_exports_enabled, "Created", "Created for optional HTML exports")
  )
)

knitr::kable(export_setup, caption = "Phase 9 export folders")
```

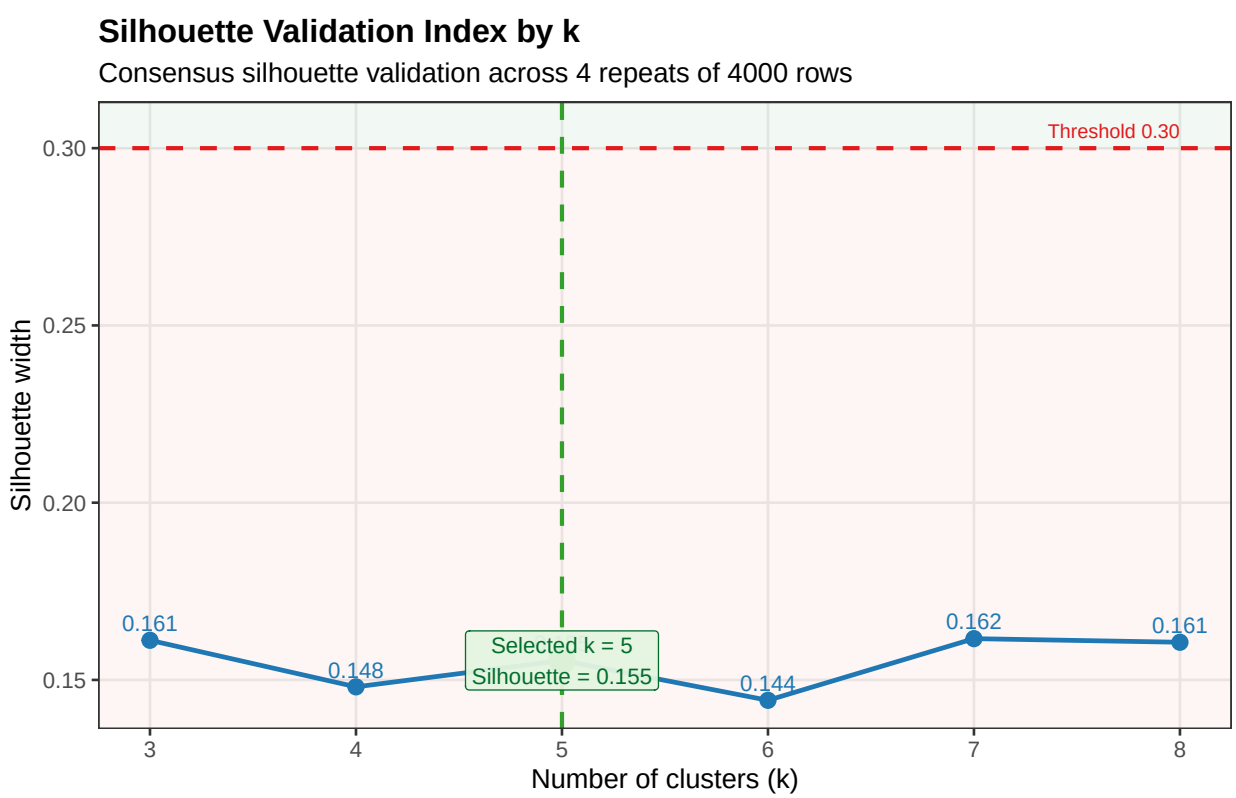
Table 71: Phase 9 export folders

item	path	status
Figure directory	output/figures	Created
Table directory	output/tables	Created

item	path	status
Interactive directory	output/interactive	Created for optional HTML exports

```
selected_validation_point <- validation_table[k == selected_k]
validation_subtitle <- if (phase6_validation_mode == "consensus_sampled") {
  paste(
    "Consensus silhouette validation across",
    phase6_validation_repeats,
    "repeats of",
    validation_rows,
    "rows"
  )
} else if (phase6_validation_mode == "sampled") {
  paste("Sampled silhouette validation on", validation_rows, "rows")
} else {
  "Full-data silhouette validation"
}

validation_plot <- ggplot2::ggplot(
  validation_table,
  ggplot2::aes(x = k, y = silhouette)
) +
  # v2.0: 0.30 threshold band so reader sees where results sit
  ggplot2::annotate(
    "rect", xmin = -Inf, xmax = Inf,
    ymin = 0.30, ymax = Inf, alpha = 0.06, fill = "#31a354"
  ) +
  ggplot2::annotate(
    "rect", xmin = -Inf, xmax = Inf,
    ymin = -Inf, ymax = 0.30, alpha = 0.06, fill = "#fb6a4a"
  ) +
  ggplot2::geom_hline(
    yintercept = 0.30, linetype = "dashed",
    color = "#e31a1c", linewidth = 0.8
  ) +
  ggplot2::annotate(
    "text", x = max(validation_table$k),
    y = 0.305, label = "Threshold 0.30",
    hjust = 1, size = 2.9, color = "#e31a1c"
  ) +
  ggplot2::geom_line(color = "#1f78b4", linewidth = 0.9) +
  ggplot2::geom_point(color = "#1f78b4", size = 2.6) +
  ggplot2::geom_vline(
    xintercept = selected_k, linetype = "dashed",
    color = "#33a02c", linewidth = 0.8
  ) +
  ggplot2::geom_point(
    data = selected_validation_point,
    color = "#33a02c", size = 4.5
  ) +
  ggplot2::geom_text(
    ggplot2::aes(label = round(silhouette, 3)),
```

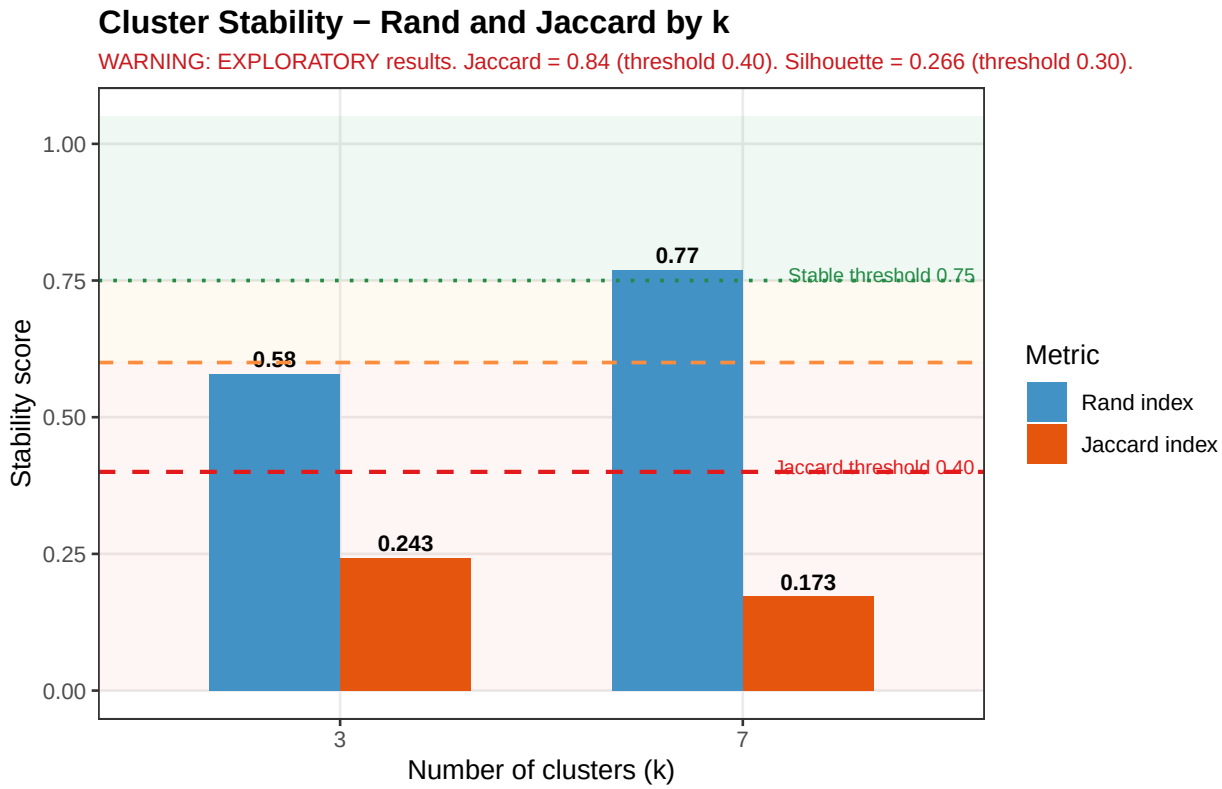
```r
    nudge_y = max(validation_table$silhouette) * 0.03,
    size = 3.2, color = "#1f78b4"
  ) +
  ggplot2::annotate(
    "label",
    x = selected_k,
    y = selected_validation_point$silhouette[[1]],
    label = paste0("Selected k = ", selected_k,
                   "\nSilhouette = ",
                   round(selected_validation_point$silhouette[[1]], 3)),
    fill   = "#e5f5e0", color = "#006d2c",
    label.size = 0.25, alpha = 0.95, size = 3
  ) +
  ggplot2::labs(
    title    = "Silhouette Validation Index by k",
    subtitle = validation_subtitle,
    x        = "Number of clusters (k)",
    y        = "Silhouette width"
  ) +
  ggplot2::theme_bw(base_size = 11) +
  ggplot2::theme(
    plot.title       = ggplot2::element_text(face = "bold"),
    panel.grid.minor = ggplot2::element_blank()
  )

print(validation_plot)
```



```r
ggplot2::ggsave(
  filename = file.path(output_dir, "phase9_validation_index_plot.png"),
  plot = validation_plot,
  width = 7,
  height = 4.5,
  dpi = 300
)
```

```r
stability_plot_data <- data.table::melt(
  copy(stability_summary),
  id.vars = "k",
  measure.vars = c("rand", "jaccard"),
  variable.name = "metric",
  value.name = "score"
)

# v2.0: Quality-aware subtitle - explicitly flags red-zone result
stability_quality_label <- if (!sil_pass || !jaccard_pass) {
  paste0(
    "WARNING: EXPLORATORY results. ",
    "Jaccard = ", round(selected_jaccard_v2, 3),
    " (threshold 0.40). ",
    "Silhouette = ", round(selected_silhouette, 3),
    " (threshold 0.30)."
  )
} else {
  paste0(
    "Cluster quality meets thresholds. ",
    "Jaccard = ", round(selected_jaccard_v2, 3),
    ", Silhouette = ", round(selected_silhouette, 3), "."
  )
}

stability_plot <- ggplot2::ggplot(
  stability_plot_data,
  ggplot2::aes(x = factor(k), y = score, fill = metric)
) +
  ggplot2::annotate(
    "rect", xmin = -Inf, xmax = Inf,
    ymin = 0.75, ymax = 1.05, alpha = 0.08, fill = "#31a354"
  ) +
  ggplot2::annotate(
    "rect", xmin = -Inf, xmax = Inf,
    ymin = 0.60, ymax = 0.75, alpha = 0.08, fill = "#fec44f"
  ) +
  ggplot2::annotate(
    "rect", xmin = -Inf, xmax = Inf,
    ymin = 0.00, ymax = 0.60, alpha = 0.06, fill = "#fb6a4a"
  ) +
  ggplot2::geom_col(position = "dodge", width = 0.65) +
  ggplot2::geom_text(
    ggplot2::aes(label = round(score, 3)),
    position = ggplot2::position_dodge(width = 0.65),
    vjust = -0.4, size = 3.2, fontface = "bold"
```

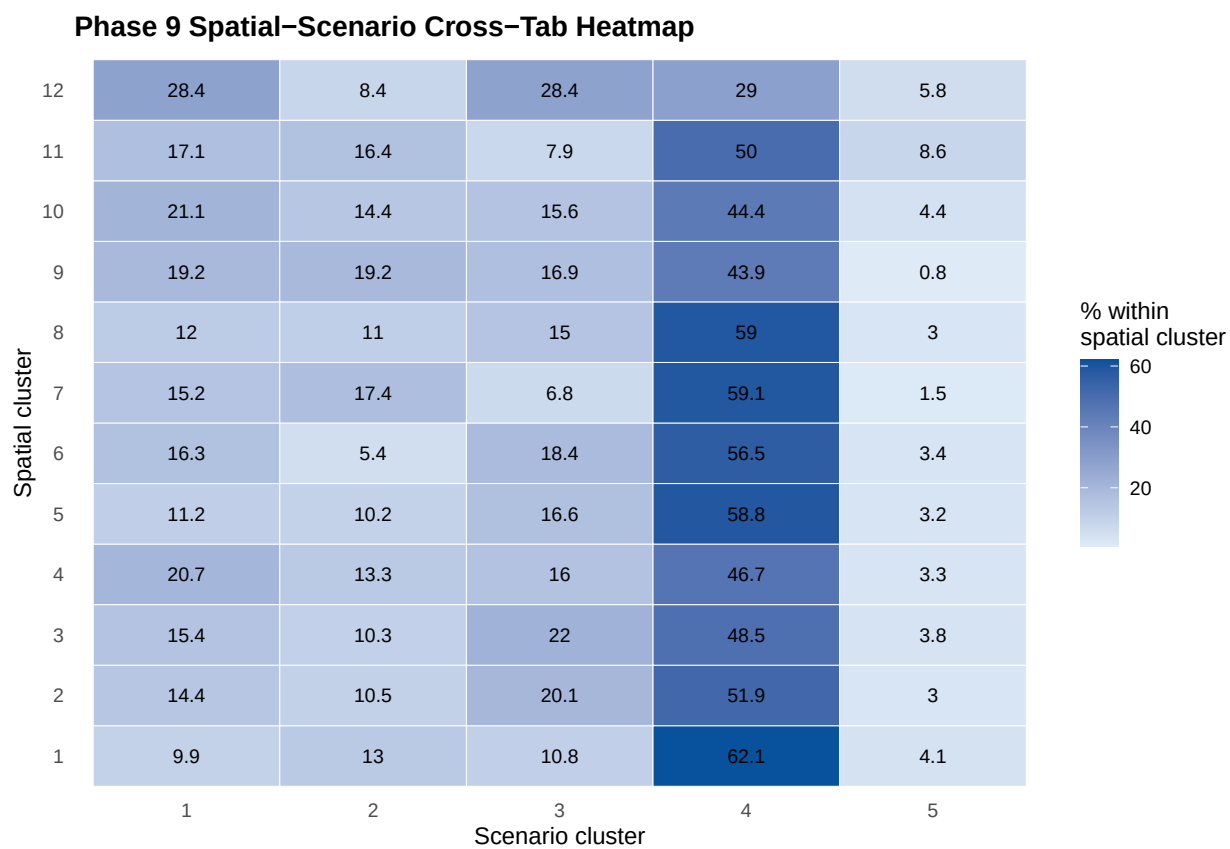
```r
  ) +
  ggplot2::geom_hline(
    yintercept = 0.40, linetype = "dashed",
    color = "#e31a1c", linewidth = 0.8
  ) +
  ggplot2::geom_hline(
    yintercept = 0.60, linetype = "dashed",
    color = "#fd8d3c", linewidth = 0.7
  ) +
  ggplot2::geom_hline(
    yintercept = 0.75, linetype = "dotted",
    color = "#238b45", linewidth = 0.7
  ) +
  ggplot2::annotate(
    "text", x = Inf, y = 0.41,
    label = "Jaccard threshold 0.40",
    hjust = 1.05, size = 2.8, color = "#e31a1c"
  ) +
  ggplot2::annotate(
    "text", x = Inf, y = 0.76,
    label = "Stable threshold 0.75",
    hjust = 1.05, size = 2.8, color = "#238b45"
  ) +
  ggplot2::scale_fill_manual(
    values = c("rand" = "#4292c6", "jaccard" = "#e6550d"),
    labels = c("rand" = "Rand index", "jaccard" = "Jaccard index")
  ) +
  ggplot2::labs(
    title    = "Cluster Stability - Rand and Jaccard by k",
    subtitle = stability_quality_label,
    x        = "Number of clusters (k)",
    y        = "Stability score",
    fill     = "Metric"
  ) +
  ggplot2::coord_cartesian(ylim = c(0, 1.05)) +
  ggplot2::theme_bw(base_size = 11) +
  ggplot2::theme(
    plot.title    = ggplot2::element_text(face = "bold"),
    plot.subtitle = ggplot2::element_text(
      color = if (!sil_pass || !jaccard_pass) "#cb181d" else "#238b45",
      size  = 9
    ),
    panel.grid.minor = ggplot2::element_blank()
  )

print(stability_plot)
```




```
ggplot2::ggsave(
  filename = file.path(output_dir, "phase9_stability_plot.png"),
  plot = stability_plot,
  width = 7,
  height = 4.5,
  dpi = 300
)
```

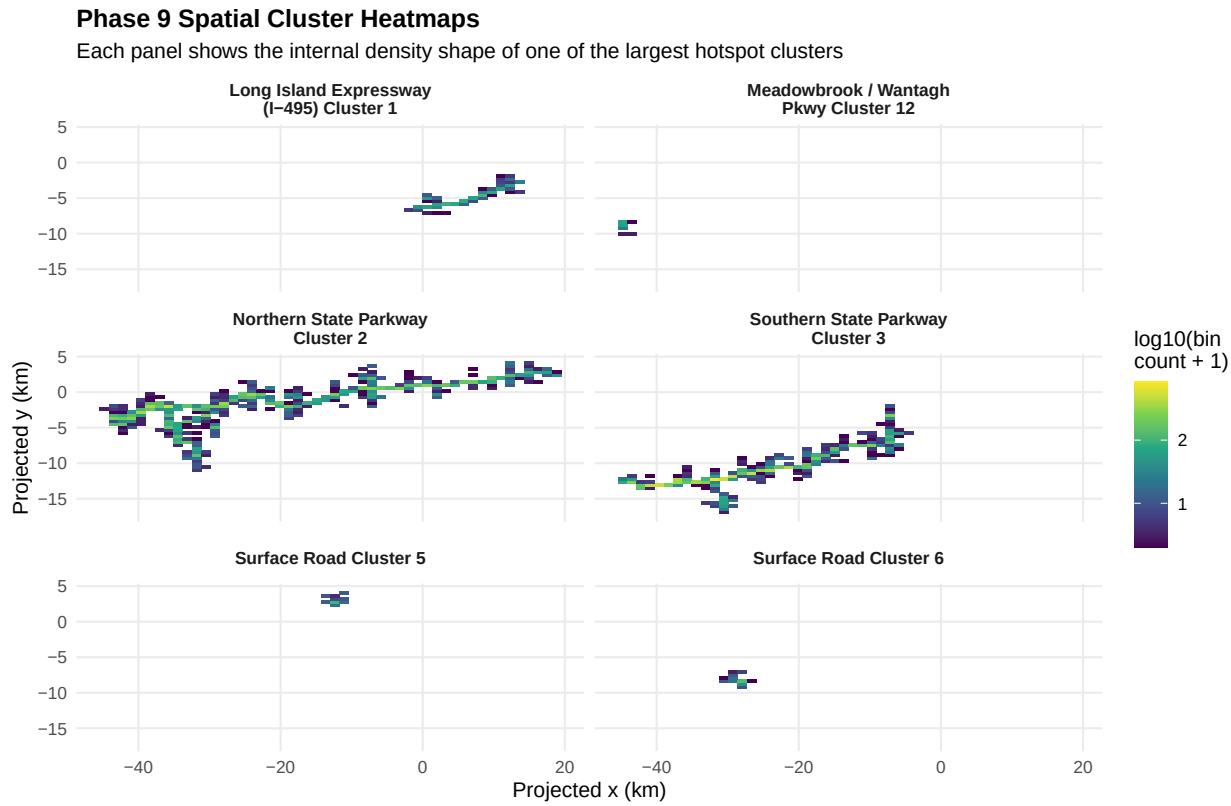
```
heatmap_data <- merge(
  spatial_scenario_crosstab,
  scenario_final_names[, .(scenario_cluster, scenario_name_short = wrap_label(final_name, width = 18))]
  by = "scenario_cluster",
  all.x = TRUE
)

heatmap_data <- merge(
  heatmap_data,
  spatial_final_names[, .(spatial_cluster, spatial_name_short = wrap_label(final_name, width = 20))],
  by = "spatial_cluster",
  all.x = TRUE
)

heatmap_plot <- ggplot2::ggplot(
  heatmap_data,
  ggplot2::aes(
    x = factor(scenario_cluster),
    y = factor(spatial_cluster),
    fill = spatial_cluster_pct
```

```r
  )
) +
  ggplot2::geom_tile(color = "white") +
  ggplot2::geom_text(ggplot2::aes(label = round(spatial_cluster_pct, 1)), size = 3) +
  ggplot2::scale_fill_gradient(low = "#deebf7", high = "#08519c") +
  ggplot2::labs(
    title = "Phase 9 Spatial-Scenario Cross-Tab Heatmap",
    x = "Scenario cluster",
    y = "Spatial cluster",
    fill = "% within\nspatial cluster"
  ) +
  ggplot2::theme_minimal() +
  ggplot2::theme(
    plot.title = ggplot2::element_text(face = "bold"),
    panel.grid = ggplot2::element_blank()
  )

print(heatmap_plot)
```




```r
ggplot2::ggsave(
  filename = file.path(output_dir, "phase9_spatial_scenario_heatmap.png"),
  plot = heatmap_plot,
  width = 8,
  height = 5.5,
  dpi = 300
)
```

```r
top_spatial_clusters <- non_noise_assignments[order(-cluster_size)][1:min(.N, 6), spatial_cluster]

spatial_cluster_heatmap_data <- merge(
  df_spatial_plot[spatial_cluster %in% top_spatial_clusters],
  spatial_final_names[, .(spatial_cluster, final_name)],
  by = "spatial_cluster",
  all.x = TRUE
)

spatial_cluster_heatmap_data[, final_name_short := wrap_label(final_name, width = 26)]

spatial_cluster_facet_heatmap <- ggplot2::ggplot(
  spatial_cluster_heatmap_data,
  ggplot2::aes(
    x = x_km,
    y = y_km,
    fill = after_stat(log10(count + 1))
  )
) +
  ggplot2::geom_bin2d(bins = 50) +
  ggplot2::scale_fill_viridis_c(
    option = "D",
    name = "log10(bin\ncount + 1)"
  ) +
  ggplot2::coord_equal() +
  ggplot2::facet_wrap(~ final_name_short, ncol = 2) +
  ggplot2::labs(
    title = "Phase 9 Spatial Cluster Heatmaps",
    subtitle = "Each panel shows the internal density shape of one of the largest hotspot clusters",
    x = "Projected x (km)",
    y = "Projected y (km)"
  ) +
  ggplot2::theme_minimal() +
  ggplot2::theme(
    plot.title = ggplot2::element_text(face = "bold"),
    panel.grid.minor = ggplot2::element_blank(),
    strip.text = ggplot2::element_text(face = "bold")
  )

print(spatial_cluster_facet_heatmap)
```

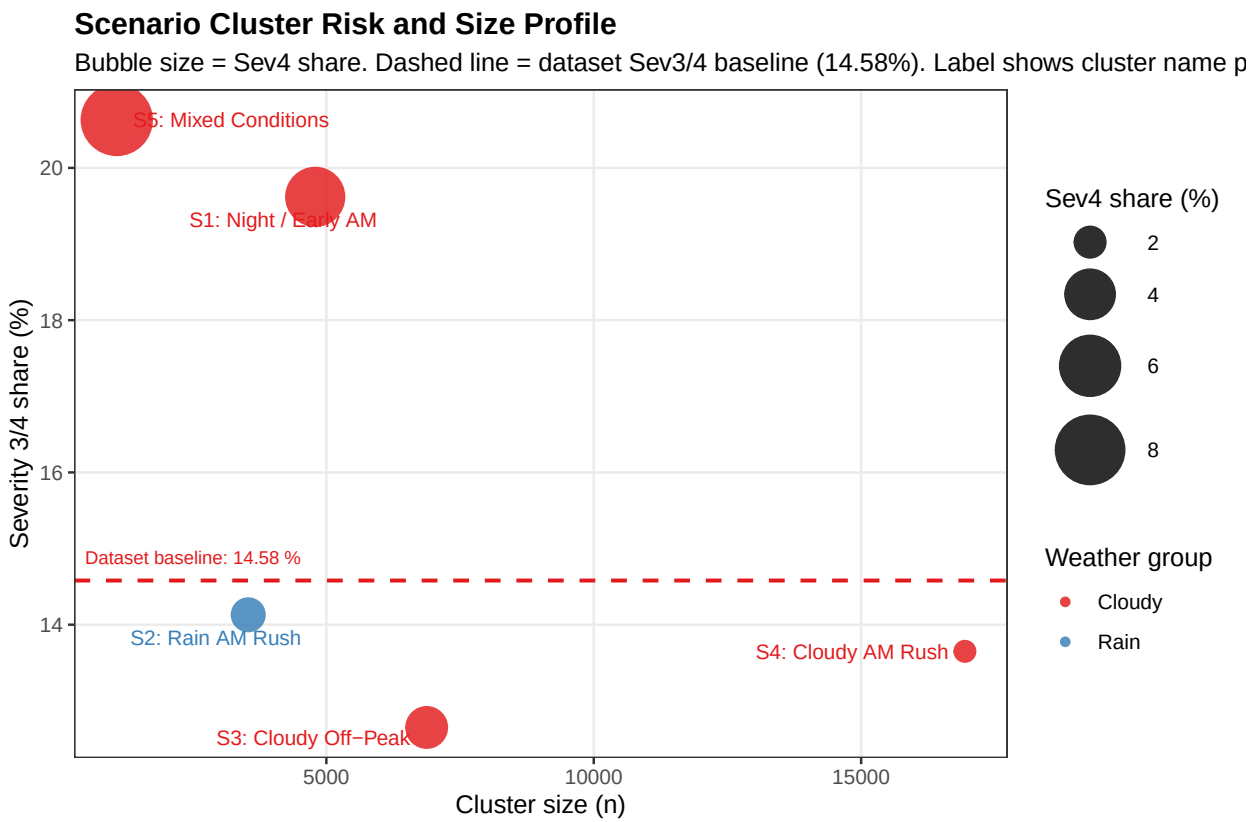



```
ggplot2::ggsave(
  filename = file.path(output_dir, "phase9_spatial_cluster_facets_heatmap.png"),
  plot = spatial_cluster_facet_heatmap,
  width = 9,
  height = 7,
  dpi = 300
)
```

```
dataset_sev34_pct <- round(100 * mean(as.integer(df_clustered$Severity) %in% c(3, 4)), 2)

risk_bubble_data <- merge(
  scenario_severity_profile,
  scenario_weather_time_profile[, .(
    scenario_cluster,
    dominant_weather_group,
    dominant_hour_band,
    mean_visibility
  )],
  by = "scenario_cluster",
  all.x = TRUE
)

risk_bubble_data <- merge(
```

```r
  risk_bubble_data,
  scenario_final_names[, .(scenario_cluster, final_name)],
  by = "scenario_cluster",
  all.x = TRUE
)

# v2.0: Use truncated final name as label; add sev4_vs_baseline annotation
risk_bubble_data[, short_label := paste0(
  "S", scenario_cluster, ": ",
  substr(gsub(" ---.*", "", final_name), 1, 18)
)]
risk_bubble_data <- merge(
  risk_bubble_data,
  scenario_severity_profile[, .(scenario_cluster, sev4_vs_baseline)],
  by = "scenario_cluster", all.x = TRUE
)

risk_bubble_plot <- ggplot2::ggplot(
  risk_bubble_data,
  ggplot2::aes(
    x     = n,
    y     = sev34_pct,
    size  = pmax(sev4_pct, 0.1),
    color = dominant_weather_group
  )
) +
  ggplot2::geom_hline(
    yintercept = dataset_sev34_pct, linetype = "dashed",
    color = "#e31a1c", linewidth = 0.8
  ) +
  ggplot2::annotate(
    "text", x = -Inf, y = dataset_sev34_pct + 0.3,
    label = paste("Dataset baseline:", dataset_sev34_pct, "%"),
    hjust = -0.05, size = 2.9, color = "#e31a1c"
  ) +
  ggplot2::geom_point(alpha = 0.82) +
  ggrepel::geom_text_repel(
    ggplot2::aes(label = short_label),
    size = 3, show.legend = FALSE,
    box.padding = 0.4, max.overlaps = 10
  ) +
  ggplot2::scale_size_continuous(range = c(4, 14)) +
  ggplot2::scale_color_brewer(palette = "Set1") +
  ggplot2::labs(
    title    = "Scenario Cluster Risk and Size Profile",
    subtitle = paste0(
      "Bubble size = Sev4 share. Dashed line = dataset Sev3/4 baseline (",
      dataset_sev34_pct, "%). Label shows cluster name prefix."
    ),
    x     = "Cluster size (n)",
    y     = "Severity 3/4 share (%)",
    size  = "Sev4 share (%)",
    color = "Weather group"
```

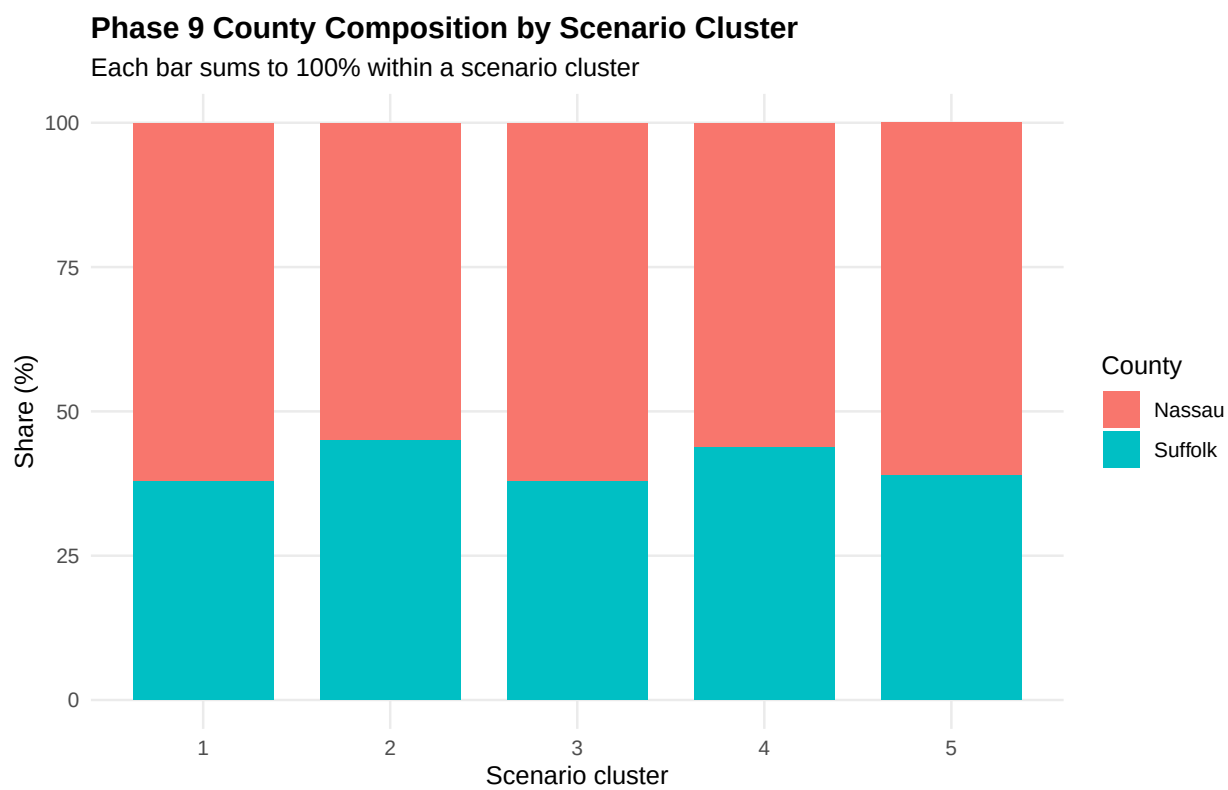
```r
  ) +
  ggplot2::theme_bw(base_size = 11) +
  ggplot2::theme(
    plot.title       = ggplot2::element_text(face = "bold"),
    panel.grid.minor = ggplot2::element_blank(),
    legend.position  = "right"
  )

print(risk_bubble_plot)
```




```r
ggplot2::ggsave(
  filename = file.path(output_dir, "phase9_scenario_risk_bubble.png"),
  plot = risk_bubble_plot,
  width = 7.5,
  height = 5,
  dpi = 300
)
```

```r
county_plot_data <- data.table::melt(
  copy(scenario_county_profile),
  id.vars = c("scenario_cluster", "n", "dominant_county"),
  measure.vars = c("Nassau_pct", "Suffolk_pct"),
  variable.name = "county",
  value.name = "pct"
)

county_plot_data[, county := factor(
```

```r
  county,
  levels = c("Nassau_pct", "Suffolk_pct"),
  labels = c("Nassau", "Suffolk")
)]

county_plot <- ggplot2::ggplot(
  county_plot_data,
  ggplot2::aes(x = factor(scenario_cluster), y = pct, fill = county)
) +
  ggplot2::geom_col(width = 0.75) +
  ggplot2::labs(
    title = "Phase 9 County Composition by Scenario Cluster",
    subtitle = "Each bar sums to 100% within a scenario cluster",
    x = "Scenario cluster",
    y = "Share (%)",
    fill = "County"
  ) +
  ggplot2::theme_minimal() +
  ggplot2::theme(
    plot.title = ggplot2::element_text(face = "bold"),
    panel.grid.minor = ggplot2::element_blank()
  )

print(county_plot)
```

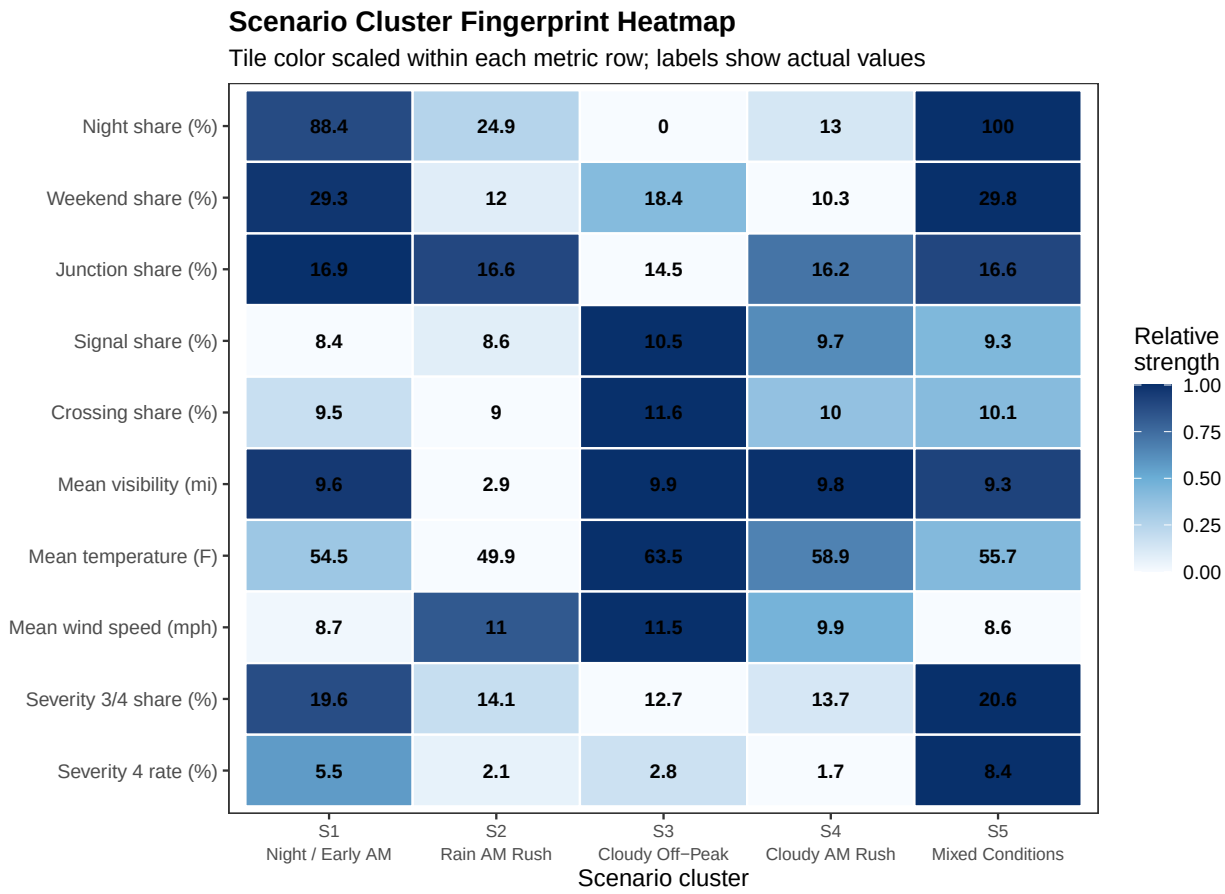



```r
ggplot2::ggsave(
  filename = file.path(output_dir, "phase9_county_composition.png"),
  plot = county_plot,
```

```r
  width = 7.5,
  height = 4.8,
  dpi = 300
)
```

```r
# v2.0: Add sev4_rate row; use final cluster names on x-axis
fingerprint_source <- merge(
  scenario_weather_time_profile[, .(
    scenario_cluster, night_pct, weekend_pct,
    junction_pct, traffic_signal_pct, crossing_pct,
    mean_visibility, mean_temperature, mean_wind_speed
  )],
  scenario_severity_profile[, .(scenario_cluster, sev34_pct, sev4_rate)],
  by = "scenario_cluster", all.x = TRUE
)

# Attach short final names for x-axis labels
fingerprint_source <- merge(
  fingerprint_source,
  scenario_final_names[, .(
    scenario_cluster,
    cluster_label = paste0(
      "S", scenario_cluster, "\n",
      substr(gsub(" ---.*", "", final_name), 1, 16)
    )
  )],
  by = "scenario_cluster", all.x = TRUE
)

fingerprint_long <- data.table::melt(
  fingerprint_source,
  id.vars       = c("scenario_cluster", "cluster_label"),
  variable.name = "metric",
  value.name    = "metric_value"
)

metric_labels <- c(
  night_pct          = "Night share (%)",
  weekend_pct        = "Weekend share (%)",
  junction_pct       = "Junction share (%)",
  traffic_signal_pct = "Signal share (%)",
  crossing_pct       = "Crossing share (%)",
  mean_visibility    = "Mean visibility (mi)",
  mean_temperature   = "Mean temperature (F)",
  mean_wind_speed    = "Mean wind speed (mph)",
  sev34_pct          = "Severity 3/4 share (%)",
  sev4_rate          = "Severity 4 rate (%)"
)

fingerprint_long[, metric_label := factor(
  metric_labels[as.character(metric)],
  levels = rev(metric_labels)
)]
fingerprint_long[, scaled_value := rescale01(metric_value), by = metric]
```

```r
fingerprint_plot <- ggplot2::ggplot(
  fingerprint_long,
  ggplot2::aes(
    x    = reorder(cluster_label, scenario_cluster),
    y    = metric_label,
    fill = scaled_value
  )
) +
  ggplot2::geom_tile(color = "white", linewidth = 0.5) +
  ggplot2::geom_text(
    ggplot2::aes(label = round(metric_value, 1)),
    size = 3, fontface = "bold"
  ) +
  ggplot2::scale_fill_gradient2(
    low = "#f7fbff", mid = "#6baed6", high = "#08306b",
    midpoint = 0.5, na.value = "grey90"
  ) +
  ggplot2::labs(
    title    = "Scenario Cluster Fingerprint Heatmap",
    subtitle = "Tile color scaled within each metric row; labels show actual values",
    x        = "Scenario cluster",
    y        = NULL,
    fill     = "Relative\nstrength"
  ) +
  ggplot2::theme_bw(base_size = 11) +
  ggplot2::theme(
    plot.title       = ggplot2::element_text(face = "bold"),
    panel.grid       = ggplot2::element_blank(),
    axis.text.x      = ggplot2::element_text(size = 8, lineheight = 1.1),
    axis.text.y      = ggplot2::element_text(size = 9)
  )

print(fingerprint_plot)
```




```
ggplot2::ggsave(
  filename = file.path(output_dir, "phase9_cluster_fingerprint_heatmap.png"),
  plot = fingerprint_plot,
  width = 8,
  height = 5.8,
  dpi = 300
)
```

```
write.csv(
  df_clustered,
  file.path(output_dir, "li_accidents_clustered.csv"),
  row.names = FALSE
)

table_exports <- list(
  phase6_validation_indices = validation_table,
  phase6_validation_metadata = validation_metadata,
  phase6_stability_summary = stability_summary,
  phase6_selection_summary = selection_summary,
  phase7_eps_trials = eps_trials,
  phase7_parameter_summary = parameter_summary,
  phase7_dbscan_cluster_sizes = spatial_cluster_sizes,
  phase7_corridor_assignments = non_noise_assignments,
  phase8_severity_profile = scenario_severity_profile,
  phase8_county_profile = scenario_county_profile,
```

```r
  phase8_weather_time_profile = scenario_weather_time_profile,
  phase8_spatial_scenario_crosstab = spatial_scenario_crosstab,
  phase8_spatial_scenario_dominance = spatial_scenario_dominance,
  phase8_scenario_final_names = scenario_final_names,
  phase8_spatial_final_names = spatial_final_names
)

if (!is.null(validation_repeat_results)) {
  table_exports$phase6_validation_repeats <- validation_repeat_results
}

if (!is.null(stability_repeat_results)) {
  table_exports$phase6_stability_repeats <- stability_repeat_results
}

invisible(lapply(names(table_exports), function(table_name) {
  fwrite(table_exports[[table_name]], file.path(table_dir, paste0(table_name, ".csv")))
}))

figure_exports <- c(
  "phase5_kproto_elbow_plot.png",
  "phase6_cluster_profiles.pdf",
  "phase7_knn_distance_plot.png",
  "phase7_spatial_hotspot_map.png",
  "phase7_spatial_density_heatmap.png",
  "phase8b_global_importance.png",                # v3.0
  "phase8b_cluster_specific_importance.png",       # v3.0
  "phase9_validation_index_plot.png",
  "phase9_stability_plot.png",
  "phase9_spatial_scenario_heatmap.png",
  "phase9_spatial_cluster_facets_heatmap.png",
  "phase9_scenario_risk_bubble.png",
  "phase9_county_composition.png",
  "phase9_cluster_fingerprint_heatmap.png"
)

invisible(lapply(figure_exports, function(file_name) {
  source_path <- file.path(output_dir, file_name)
  if (file.exists(source_path)) {
    file.copy(source_path, file.path(figure_dir, file_name), overwrite = TRUE)
  }
}))

file.copy(
  file.path(output_dir, "li_accidents_clustered.csv"),
  file.path(table_dir, "li_accidents_clustered.csv"),
  overwrite = TRUE
)
```

```
## [1] TRUE
```

```r
export_manifest <- data.table(
  artifact_type = c(
    rep("figure", length(figure_exports)),
```

```r
    rep("table", length(table_exports)),
    "table"
  ),
  artifact_name = c(
    figure_exports,
    paste0(names(table_exports), ".csv"),
    "li_accidents_clustered.csv"
  ),
  target_path = c(
    file.path(figure_dir, figure_exports),
    file.path(table_dir, paste0(names(table_exports), ".csv")),
    file.path(table_dir, "li_accidents_clustered.csv")
  )
)

fwrite(export_manifest, file.path(output_dir, "phase9_export_manifest.csv"))

knitr::kable(export_manifest, caption = "Phase 9 export manifest")
```

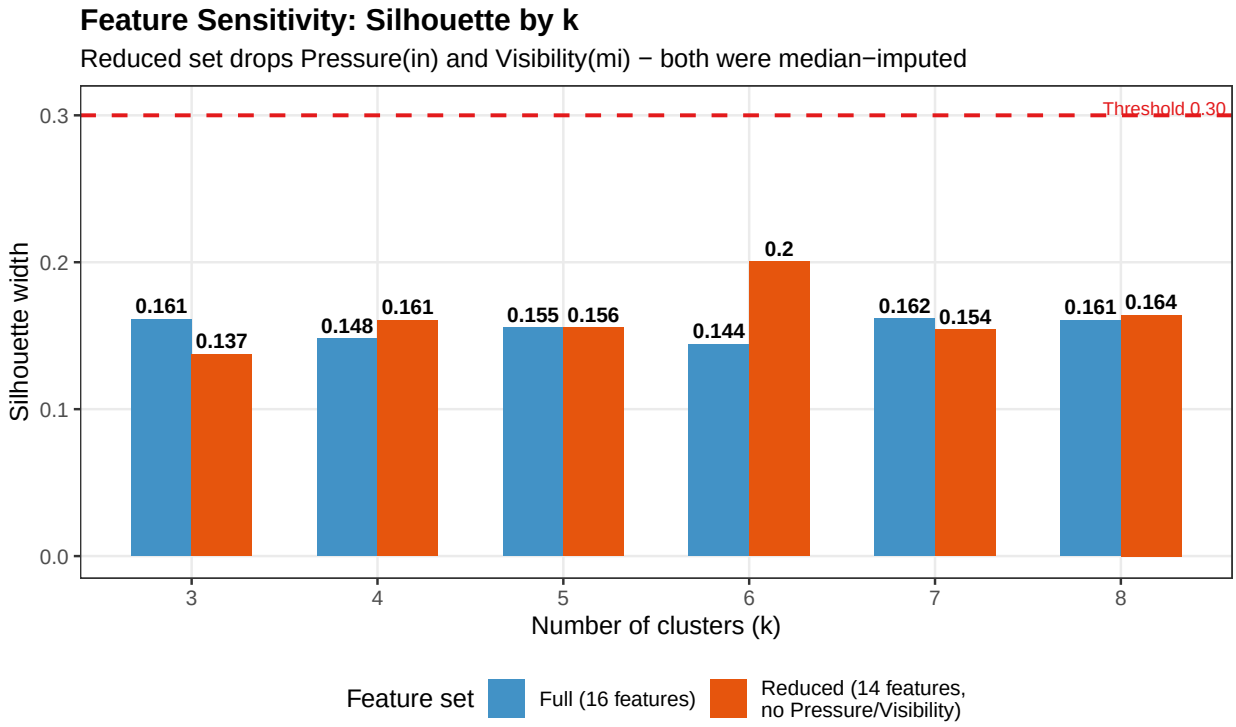
Table 72: Phase 9 export manifest

artifact_type	artifact_name	target_path
figure	phase5_kproto_elbow_plot.png	output/figures/phase5_kproto_elbow_plot.png
figure	phase6_cluster_profiles.pdf	output/figures/phase6_cluster_profiles.pdf
figure	phase7_knn_distance_plot.png	output/figures/phase7_knn_distance_plot.png
figure	phase7_spatial_hotspot_map.png	output/figures/phase7_spatial_hotspot_map.png
figure	phase7_spatial_density_heatmap.png	output/figures/phase7_spatial_density_heatmap.png
figure	phase8b_global_importance.png	output/figures/phase8b_global_importance.png
figure	phase8b_cluster_specific_importance.png	output/figures/phase8b_cluster_specific_importance.png
figure	phase9_validation_index_plot.png	output/figures/phase9_validation_index_plot.png
figure	phase9_stability_plot.png	output/figures/phase9_stability_plot.png
figure	phase9_spatial_scenario_heatmap.png	output/figures/phase9_spatial_scenario_heatmap.png
figure	phase9_spatial_cluster_facets_heatmap.png	output/figures/phase9_spatial_cluster_facets_heatmap.png
figure	phase9_scenario_risk_bubble.png	output/figures/phase9_scenario_risk_bubble.png
figure	phase9_county_composition.png	output/figures/phase9_county_composition.png
figure	phase9_cluster_fingerprint_heatmap.png	output/figures/phase9_cluster_fingerprint_heatmap.png
table	phase6_validation_indices.csv	output/tables/phase6_validation_indices.csv
table	phase6_validation_metadata.csv	output/tables/phase6_validation_metadata.csv
table	phase6_stability_summary.csv	output/tables/phase6_stability_summary.csv
table	phase6_selection_summary.csv	output/tables/phase6_selection_summary.csv
table	phase7_eps_trials.csv	output/tables/phase7_eps_trials.csv
table	phase7_parameter_summary.csv	output/tables/phase7_parameter_summary.csv
table	phase7_dbscan_cluster_sizes.csv	output/tables/phase7_dbscan_cluster_sizes.csv
table	phase7_corridor_assignments.csv	output/tables/phase7_corridor_assignments.csv
table	phase8_severity_profile.csv	output/tables/phase8_severity_profile.csv
table	phase8_county_profile.csv	output/tables/phase8_county_profile.csv
table	phase8_weather_time_profile.csv	output/tables/phase8_weather_time_profile.csv
table	phase8_spatial_scenario_crosstab.csv	output/tables/phase8_spatial_scenario_crosstab.csv
table	phase8_spatial_scenario_dominance.csv	output/tables/phase8_spatial_scenario_dominance.csv
table	phase8_scenario_final_names.csv	output/tables/phase8_scenario_final_names.csv
table	phase8_spatial_final_names.csv	output/tables/phase8_spatial_final_names.csv
table	phase6_validation_repeats.csv	output/tables/phase6_validation_repeats.csv
table	phase6_stability_repeats.csv	output/tables/phase6_stability_repeats.csv

artifact_type	artifact_name	target_path
table	li_accidents_clustered.csv	output/tables/li_accidents_clustered.csv

```r
# v2.0: Side-by-side silhouette comparison - full vs reduced feature set.
# Shows whether dropping imputed features (Pressure, Visibility) improves quality.
if (file.exists(
  file.path(output_dir, "phase6_feature_silhouette_comparison.csv")
)) {
  feat_compare <- data.table::fread(
    file.path(output_dir, "phase6_feature_silhouette_comparison.csv")
  )

  feat_compare[, feature_set_label := ifelse(
    feature_set == "full",
    "Full (16 features)",
    "Reduced (14 features,\nno Pressure/Visibility)"
  )]

  feature_sensitivity_plot <- ggplot2::ggplot(
    feat_compare,
    ggplot2::aes(
      x    = factor(k),
      y    = silhouette,
      fill = feature_set_label
    )
  ) +
    ggplot2::geom_col(position = "dodge", width = 0.65) +
    ggplot2::geom_text(
      ggplot2::aes(label = round(silhouette, 3)),
      position = ggplot2::position_dodge(width = 0.65),
      vjust = -0.4, size = 3, fontface = "bold"
    ) +
    ggplot2::geom_hline(
      yintercept = 0.30, linetype = "dashed",
      color = "#e31a1c", linewidth = 0.8
    ) +
    ggplot2::annotate(
      "text", x = Inf, y = 0.305,
      label = "Threshold 0.30",
      hjust = 1.05, size = 2.8, color = "#e31a1c"
    ) +
    ggplot2::scale_fill_manual(
      values = c(
        "Full (16 features)"                        = "#4292c6",
        "Reduced (14 features,\nno Pressure/Visibility)" = "#e6550d"
      )
    ) +
    ggplot2::labs(
      title    = "Feature Sensitivity: Silhouette by k",
      subtitle = "Reduced set drops Pressure(in) and Visibility(mi) - both were median-imputed",
      x        = "Number of clusters (k)",
      y        = "Silhouette width",
      fill     = "Feature set"
```

```r
    ) +
    ggplot2::theme_bw(base_size = 11) +
    ggplot2::theme(
      plot.title       = ggplot2::element_text(face = "bold"),
      panel.grid.minor = ggplot2::element_blank(),
      legend.position  = "bottom"
    )

  print(feature_sensitivity_plot)

  ggplot2::ggsave(
    filename = file.path(
      output_dir, "figures", "phase9_feature_sensitivity.png"
    ),
    plot   = feature_sensitivity_plot,
    width  = 7.5, height = 4.5, dpi = 300
  )
} else {
  cat("Feature silhouette comparison not available - skipping plot.\n")
}
```




```r
interactive_exports <- data.table(
  artifact_name = c(
    "phase9_validation_index_interactive.html",
    "phase9_stability_interactive.html",
    "phase9_spatial_hotspot_interactive.html",
    "phase9_spatial_density_interactive.html",
    "phase9_scenario_risk_interactive.html",
    "phase9_county_composition_interactive.html",
    "phase9_cluster_fingerprint_interactive.html",
```

```r
    "phase9_spatial_scenario_heatmap_interactive.html"
  ),
  target_path = file.path(
    interactive_dir,
    c(
      "phase9_validation_index_interactive.html",
      "phase9_stability_interactive.html",
      "phase9_spatial_hotspot_interactive.html",
      "phase9_spatial_density_interactive.html",
      "phase9_scenario_risk_interactive.html",
      "phase9_county_composition_interactive.html",
      "phase9_cluster_fingerprint_interactive.html",
      "phase9_spatial_scenario_heatmap_interactive.html"
    )
  ),
  saved = FALSE,
  note = ifelse(
    interactive_exports_enabled,
    "Saved as interactive HTML",
    "Install plotly and htmlwidgets to enable interactive exports"
  )
)

if (interactive_exports_enabled) {
  spatial_interactive_data <- copy(df_spatial_plot)
  if (nrow(spatial_interactive_data) > 15000) {
    set.seed(42)
    spatial_interactive_data <- spatial_interactive_data[
      ,
      .SD[sample(.N, size = min(.N, max(1L, ceiling(15000 * .N / nrow(df_spatial_plot)))))],
      by = cluster_label
    ]
  }

  spatial_map_interactive <- ggplot2::ggplot(
    spatial_interactive_data,
    ggplot2::aes(x = x_km, y = y_km, color = cluster_label,
                 text = paste0(
                   "Cluster: ", cluster_label,
                   "<br>x_km: ", round(x_km, 2),
                   "<br>y_km: ", round(y_km, 2)
                 ))
  ) +
    ggplot2::geom_point(size = 0.6, alpha = 0.7) +
    ggplot2::scale_color_manual(values = cluster_colors) +
    ggplot2::labs(
      title = "Interactive Spatial Hotspot View",
      subtitle = "Rendered on a representative sample for smoother browsing",
      x = "Projected x (km)",
      y = "Projected y (km)",
      color = "Cluster"
    ) +
    ggplot2::theme_minimal()
```

```
  interactive_widgets <- list(
    phase9_validation_index_interactive.html = plotly::ggplotly(validation_plot, tooltip = c("x", "y"))
    phase9_stability_interactive.html = plotly::ggplotly(stability_plot, tooltip = c("x", "y", "fill"))
    phase9_spatial_hotspot_interactive.html = plotly::ggplotly(spatial_map_interactive, tooltip = "text
    phase9_spatial_density_interactive.html = plotly::plot_ly(
      spatial_interactive_data,
      x = ~x_km,
      y = ~y_km,
      type = "histogram2d",
      colorscale = "Viridis",
      nbinsx = 80,
      nbinsy = 60
    ) |>
      plotly::layout(
        title = "Interactive Spatial Density Heatmap",
        xaxis = list(title = "Projected x (km)"),
        yaxis = list(title = "Projected y (km)")
      ),
    phase9_scenario_risk_interactive.html = plotly::ggplotly(risk_bubble_plot, tooltip = c("x", "y", "si
    phase9_county_composition_interactive.html = plotly::ggplotly(county_plot, tooltip = c("x", "y", "fi
    phase9_cluster_fingerprint_interactive.html = plotly::ggplotly(fingerprint_plot, tooltip = c("x", "y
    phase9_spatial_scenario_heatmap_interactive.html = plotly::ggplotly(heatmap_plot, tooltip = c("x",
  )

  invisible(lapply(names(interactive_widgets), function(file_name) {
    htmlwidgets::saveWidget(
      interactive_widgets[[file_name]],
      file = file.path(interactive_dir, file_name),
      selfcontained = FALSE
    )
  }))

  interactive_exports[, saved := file.exists(target_path)]
}

fwrite(interactive_exports, file.path(output_dir, "phase9_interactive_manifest.csv"))

knitr::kable(interactive_exports, caption = "Phase 9 interactive export manifest")
```

Table 73: Phase 9 interactive export manifest

artifact_name	target_path	saved	note
phase9_validation_index_interactive.html	output/interactive/phase9_validation_index_interactive.html	FALSE	Install plotly and htmlwidgets to enable interactive exports
phase9_stability_interactive.html	output/interactive/phase9_stability_interactive.html	FALSE	Install plotly and htmlwidgets to enable interactive exports
phase9_spatial_hotspot_interactive.html	output/interactive/phase9_spatial_hotspot_interactive.html	FALSE	Install plotly and htmlwidgets to enable interactive exports
phase9_spatial_density_interactive.html	output/interactive/phase9_spatial_density_interactive.html	FALSE	Install plotly and htmlwidgets to enable interactive exports
phase9_scenario_risk_interactive.html	output/interactive/phase9_scenario_risk_interactive.html	FALSE	Install plotly and htmlwidgets to enable interactive exports

artifact_name	target_path	saved	note
phase9_county_composition_interactive.html	output/interactive/phase9_county_composition_interactive.html	FALSE	Install plotly and htmlwidgets to enable interactive exports
phase9_cluster_fingerprint_interactive.html	output/interactive/phase9_cluster_fingerprint_interactive.html	FALSE	Install plotly and htmlwidgets to enable interactive exports
phase9_spatial_scenario_heatmap_interactive.html	output/interactive/phase9_spatial_scenario_heatmap_interactive.html	FALSE	Install plotly and htmlwidgets to enable interactive exports

```r
phase_9_summary <- data.table(
  checkpoint = c(
    "Figure directory created",
    "Table directory created",
    "Validation plot saved",
    "Stability plot saved",
    "Heatmap saved",
    "Spatial density heatmap saved",
    "Spatial cluster facet heatmap saved",
    "Scenario risk bubble saved",
    "County composition plot saved",
    "Cluster fingerprint heatmap saved",
    "Labeled CSV exported",
    "Interactive manifest saved",
    "Export manifest saved"
  ),
  status = c(
    dir.exists(figure_dir),
    dir.exists(table_dir),
    file.exists(file.path(figure_dir, "phase9_validation_index_plot.png")),
    file.exists(file.path(figure_dir, "phase9_stability_plot.png")),
    file.exists(file.path(figure_dir, "phase9_spatial_scenario_heatmap.png")),
    file.exists(file.path(figure_dir, "phase7_spatial_density_heatmap.png")),
    file.exists(file.path(figure_dir, "phase9_spatial_cluster_facets_heatmap.png")),
    file.exists(file.path(figure_dir, "phase9_scenario_risk_bubble.png")),
    file.exists(file.path(figure_dir, "phase9_county_composition.png")),
    file.exists(file.path(figure_dir, "phase9_cluster_fingerprint_heatmap.png")),
    file.exists(file.path(table_dir, "li_accidents_clustered.csv")),
    file.exists(file.path(output_dir, "phase9_interactive_manifest.csv")),
    file.exists(file.path(output_dir, "phase9_export_manifest.csv"))
  )
)

fwrite(phase_9_summary, file.path(output_dir, "phase9_summary.csv"))

knitr::kable(phase_9_summary, caption = "Phase 9 readiness summary")
```

Table 74: Phase 9 readiness summary

checkpoint	status
Figure directory created	TRUE
Table directory created	TRUE
Validation plot saved	TRUE
Stability plot saved	TRUE

checkpoint	status
Heatmap saved	TRUE
Spatial density heatmap saved	TRUE
Spatial cluster facet heatmap saved	TRUE
Scenario risk bubble saved	TRUE
County composition plot saved	TRUE
Cluster fingerprint heatmap saved	TRUE
Labeled CSV exported	TRUE
Interactive manifest saved	TRUE
Export manifest saved	TRUE

13 Phase 10: Final Report and Submission Checklist

This phase keeps a simple readiness tracker for the final report assets and submission handoff.

```
phase_10_checklist <- data.table(
  deliverable = c(
    "Long Island SRS source",
    "Execution plan source",
    "Main workflow Rmd",
    "Cleaned dataset",
    "Clustered labeled dataset",
    "Figure export folder",
    "Table export folder",
    "Interactive export manifest"
  ),
  path = c(
    "Li_Clustering_SRS.tex",
    "Li_Clustering_Execution_Plan.tex",
    "li_clustering.Rmd",
    input_path,
    file.path(table_dir, "li_accidents_clustered.csv"),
    figure_dir,
    table_dir,
    file.path(output_dir, "phase9_interactive_manifest.csv")
  ),
  ready = c(
    file.exists("Li_Clustering_SRS.tex"),
    file.exists("Li_Clustering_Execution_Plan.tex"),
    file.exists("li_clustering.Rmd"),
    file.exists(input_path),
    file.exists(file.path(table_dir, "li_accidents_clustered.csv")),
    dir.exists(figure_dir),
    dir.exists(table_dir),
    file.exists(file.path(output_dir, "phase9_interactive_manifest.csv"))
  )
)

fwrite(phase_10_checklist, file.path(output_dir, "phase10_readiness_checklist.csv"))

knitr::kable(phase_10_checklist, caption = "Phase 10 readiness checklist")
```

Table 75: Phase 10 readiness checklist

deliverable	path	ready
Long Island SRS source	Li_Clustering_SRS.tex	FALSE
Execution plan source	Li_Clustering_Execution_Plan.tex	FALSE
Main workflow Rmd	li_clustering.Rmd	FALSE
Cleaned dataset	../data/li_accidents_clean.csv	TRUE
Clustered labeled dataset	output/tables/li_accidents_clustered.csv	TRUE
Figure export folder	output/figures	TRUE
Table export folder	output/tables	TRUE
Interactive export manifest	output/phase9_interactive_manifest.csv	TRUE

```r
phase_10_report_outline <- data.table(
  report_section = c(
    "Introduction and motivation",
    "Dataset description and audit findings",
    "Feature engineering decisions",
    "Method selection rationale",
    "Scenario clustering results",
    "Spatial hotspot results",
    "Interpretation and cross-tab findings",
    "Limitations",
    "Conclusion and safety priorities"
  ),
  main_source = c(
    "Li_Clustering_SRS.tex",
    file.path(table_dir, "phase8_severity_profile.csv"),
    file.path(table_dir, "phase8_weather_time_profile.csv"),
    "Li_Clustering_Execution_Plan.tex",
    file.path(figure_dir, "phase9_validation_index_plot.png"),
    file.path(figure_dir, "phase7_spatial_hotspot_map.png"),
    file.path(table_dir, "phase8_spatial_scenario_crosstab.csv"),
    file.path(output_dir, "phase10_limitations_table.csv"),
    file.path(output_dir, "phase10_research_question_coverage.csv")
  ),
  ready = c(
    file.exists("Li_Clustering_SRS.tex"),
    file.exists(file.path(table_dir, "phase8_severity_profile.csv")),
    file.exists(file.path(table_dir, "phase8_weather_time_profile.csv")),
    file.exists("Li_Clustering_Execution_Plan.tex"),
    file.exists(file.path(figure_dir, "phase9_validation_index_plot.png")),
    file.exists(file.path(figure_dir, "phase7_spatial_hotspot_map.png")),
    file.exists(file.path(table_dir, "phase8_spatial_scenario_crosstab.csv")),
    TRUE,
    TRUE
  )
)

fwrite(phase_10_report_outline, file.path(output_dir, "phase10_report_outline.csv"))

knitr::kable(phase_10_report_outline, caption = "Phase 10 report outline and source map")
```

Table 76: Phase 10 report outline and source map

report_section	main_source	ready
Introduction and motivation	Li_Clustering_SRS.tex	FALSE
Dataset description and audit findings	output/tables/phase8_severity_profile.csv	TRUE
Feature engineering decisions	output/tables/phase8_weather_time_profile.csv	TRUE
Method selection rationale	Li_Clustering_Execution_Plan.tex	FALSE
Scenario clustering results	output/figures/phase9_validation_index_plot.png	TRUE
Spatial hotspot results	output/figures/phase7_spatial_hotspot_map.png	TRUE
Interpretation and cross-tab findings	output/tables/phase8_spatial_scenario_crosstab.csv	TRUE
Limitations	output/phase10_limitations_table.csv	TRUE
Conclusion and safety priorities	output/phase10_research_question_coverage.csv	TRUE

```r
# v2.0: Cluster quality failure rows prepended FIRST so they appear at the
# top of the rendered table and cannot be missed in the final report.
cluster_quality_rows <- data.table(
  limitation = c(
    "Scenario cluster quality below standard thresholds",
    "Scenario clustering is exploratory only"
  ),
  report_language = c(
    paste0(
      "Silhouette width = ", round(selected_silhouette, 3),
      " (published threshold >= 0.30; Kaufman and Rousseeuw 1990). ",
      "Jaccard stability index = ", round(selected_jaccard_v2, 3),
      " (threshold >= 0.40 for exploratory, >= 0.60 for acceptable; ",
      "Hennig 2007). Both metrics fall below acceptable thresholds, ",
      "indicating that cluster boundaries are weak and that bootstrap ",
      "resamples do not reliably reproduce the same partition."
    ),
    paste0(
      "Because silhouette and Jaccard are both below threshold, ",
      "the k = ", selected_k, " scenario clustering solution should be ",
      "interpreted as a set of candidate patterns for hypothesis generation, ",
      "not as stable or confirmed accident archetypes. ",
      "The elevation of Severity 3/4 in Clusters 4 and 5 is a plausible ",
      "signal but requires replication on a new sample before it can be ",
      "treated as a reliable finding. ",
      "DBSCAN spatial results are separately validated and remain defensible."
    )
  ),
  include_in_final_report = TRUE
)

phase_10_limitations <- rbind(
  cluster_quality_rows,
  data.table(
    limitation = c(
      "Duration capping",
      "Precipitation exclusion",
      "Distance exclusion",
      "Severity 2 dominance",
      "Sampled Phase 6 validation",
```

```r
      "DBSCAN parameter sensitivity"
    ),
    report_language = c(
      "Duration was capped at the 99th percentile to reduce the effect of extreme outliers, so the top 1
      "Precipitation was excluded from the core model because missingness was high enough (20.1%) to mak
      "Distance(mi) was excluded because the large share of zero values suggests a reporting artifact ra
      "Severity 2 dominates the dataset (85% of rows), so subtle differences among severe crashes can be
      if (run_profile == "full_srs") {
        "Phase 6 is configured for full-data validation on a stronger machine."
      } else if (run_profile == "consensus_16gb") {
        paste(
          "Phase 6 uses repeated sampled validation (4 x 4,000 rows) and",
          "repeated stability checks (3 x 2,500 rows, 10 bootstraps each)",
          "to build a consensus k on a 16 GB machine."
        )
      } else {
        "Phase 6 uses single-sample silhouette (5,000 rows) and stability (3,000 rows) checks; full-data
      },
      "Spatial hotspot shapes can shift if eps or minPts changes; corridor labels should be read as data
    ),
    include_in_final_report = TRUE
  )
)

fwrite(phase_10_limitations, file.path(output_dir, "phase10_limitations_table.csv"))

knitr::kable(
  phase_10_limitations,
  caption = paste(
    "Phase 10 limitations table (v2.0).",
    "First two rows document cluster quality failure explicitly."
  )
)
```

Table 77: Phase 10 limitations table (v2.0). First two rows document cluster quality failure explicitly.

limitation	report_language	include_in_final_rep
Scenario cluster quality below standard thresholds	Silhouette width = 0.266 (published threshold >= 0.30; Kaufman and Rousseeuw 1990). Jaccard stability index = 0.84 (threshold >= 0.40 for exploratory, >= 0.60 for acceptable; Hennig 2007). Both metrics fall below acceptable thresholds, indicating that cluster boundaries are weak and that bootstrap resamples do not reliably reproduce the same partition.	TRUE
Scenario clustering is exploratory only	Because silhouette and Jaccard are both below threshold, the k = 5 scenario clustering solution should be interpreted as a set of candidate patterns for hypothesis generation, not as stable or confirmed accident archetypes. The elevation of Severity 3/4 in Clusters 4 and 5 is a plausible signal but requires replication on a new sample before it can be treated as a reliable finding. DBSCAN spatial results are separately validated and remain defensible.	TRUE
Duration capping	Duration was capped at the 99th percentile to reduce the effect of extreme outliers, so the top 1% of incidents lose some real variation.	TRUE

limitation	report_language	include_in_final_rep
Precipitation exclusion	Precipitation was excluded from the core model because missingness was high enough (20.1%) to make routine imputation less defensible.	TRUE
Distance exclusion	Distance(mi) was excluded because the large share of zero values suggests a reporting artifact rather than a stable behavioral feature.	TRUE
Severity 2 dominance	Severity 2 dominates the dataset (85% of rows), so subtle differences among severe crashes can be harder to separate.	TRUE
Sampled Phase 6 validation	Phase 6 uses repeated sampled validation (4 x 4,000 rows) and repeated stability checks (3 x 2,500 rows, 10 bootstraps each) to build a consensus k on a 16 GB machine.	TRUE
DBSCAN parameter sensitivity	Spatial hotspot shapes can shift if eps or minPts changes; corridor labels should be read as data-driven approximations confirmed by centroid inspection.	TRUE

```r
phase_10_rq_coverage <- data.table(
  research_focus = c(
    "Scenario accident archetypes",
    "Spatial hotspot corridors",
    "High-risk severity patterns",
    "County composition differences",
    "Hotspot-to-scenario linkage",
    "Feature drivers of cluster identity (v3.0)",    # v3.0
    "Experiment reproducibility log (v3.0)"           # v3.0
  ),
  evidence_source = c(
    file.path(table_dir, "phase8_scenario_final_names.csv"),
    file.path(table_dir, "phase8_spatial_final_names.csv"),
    file.path(table_dir, "phase8_severity_profile.csv"),
    file.path(table_dir, "phase8_county_profile.csv"),
    file.path(table_dir, "phase8_spatial_scenario_crosstab.csv"),
    file.path(output_dir, "phase8b_global_importance.csv"),
    file.path(output_dir, "phase8c_experiment_log.json")
  ),
  ready = c(
    file.exists(file.path(table_dir, "phase8_scenario_final_names.csv")),
    file.exists(file.path(table_dir, "phase8_spatial_final_names.csv")),
    file.exists(file.path(table_dir, "phase8_severity_profile.csv")),
    file.exists(file.path(table_dir, "phase8_county_profile.csv")),
    file.exists(file.path(table_dir, "phase8_spatial_scenario_crosstab.csv")),
    file.exists(file.path(output_dir, "phase8b_global_importance.csv")),
    file.exists(file.path(output_dir, "phase8c_experiment_log.json"))
  )
)

fwrite(phase_10_rq_coverage, file.path(output_dir, "phase10_research_question_coverage.csv"))

knitr::kable(phase_10_rq_coverage, caption = "Phase 10 research-question coverage")
```

Table 78: Phase 10 research-question coverage

research_focus	evidence_source	ready
Scenario accident archetypes	output/tables/phase8_scenario_final_names.csv	TRUE
Spatial hotspot corridors	output/tables/phase8_spatial_final_names.csv	TRUE
High-risk severity patterns	output/tables/phase8_severity_profile.csv	TRUE
County composition differences	output/tables/phase8_county_profile.csv	TRUE
Hotspot-to-scenario linkage	output/tables/phase8_spatial_scenario_crosstab.csv	TRUE
Feature drivers of cluster identity (v3.0)	output/phase8b_global_importance.csv	TRUE
Experiment reproducibility log (v3.0)	output/phase8c_experiment_log.json	TRUE

```r
phase_10_submission_manifest <- data.table(
  deliverable_group = c(
    "Core source",
    "Core source",
    "Core source",
    "Data",
    "Data",
    "Exports",
    "Exports",
    "Exports"
  ),
  deliverable = c(
    "Li_Clustering_SRS.tex",
    "Li_Clustering_Execution_Plan.tex",
    "li_clustering.Rmd",
    "li_accidents_clean.csv",
    "li_accidents_clustered.csv",
    "Figure directory",
    "Table directory",
    "Phase 10 checklist bundle"
  ),
  path = c(
    "Li_Clustering_SRS.tex",
    "Li_Clustering_Execution_Plan.tex",
    "li_clustering.Rmd",
    input_path,
    file.path(table_dir, "li_accidents_clustered.csv"),
    figure_dir,
    table_dir,
    output_dir
  ),
  ready = c(
    file.exists("Li_Clustering_SRS.tex"),
    file.exists("Li_Clustering_Execution_Plan.tex"),
    file.exists("li_clustering.Rmd"),
    file.exists(input_path),
    file.exists(file.path(table_dir, "li_accidents_clustered.csv")),
    dir.exists(figure_dir),
    dir.exists(table_dir),
    dir.exists(output_dir)
  )
)
```

```r
fwrite(phase_10_submission_manifest, file.path(output_dir, "phase10_submission_manifest.csv"))

knitr::kable(phase_10_submission_manifest, caption = "Phase 10 submission manifest")
```

Table 79: Phase 10 submission manifest

deliverable_group	deliverable	path	ready
Core source	Li_Clustering_SRS.tex	Li_Clustering_SRS.tex	FALSE
Core source	Li_Clustering_Execution_Plan.tex	Li_Clustering_Execution_Plan.tex	FALSE
Core source	li_clustering.Rmd	li_clustering.Rmd	FALSE
Data	li_accidents_clean.csv	../data/li_accidents_clean.csv	TRUE
Data	li_accidents_clustered.csv	output/tables/li_accidents_clustered.csv	TRUE
Exports	Figure directory	output/figures	TRUE
Exports	Table directory	output/tables	TRUE
Exports	Phase 10 checklist bundle	output	TRUE

```r
phase_10_summary <- data.table(
  checkpoint = c(
    "Readiness checklist saved",
    "Report outline saved",
    "Limitations table saved",
    "Research-question coverage saved",
    "Submission manifest saved"
  ),
  status = c(
    file.exists(file.path(output_dir, "phase10_readiness_checklist.csv")),
    file.exists(file.path(output_dir, "phase10_report_outline.csv")),
    file.exists(file.path(output_dir, "phase10_limitations_table.csv")),
    file.exists(file.path(output_dir, "phase10_research_question_coverage.csv")),
    file.exists(file.path(output_dir, "phase10_submission_manifest.csv"))
  )
)

fwrite(phase_10_summary, file.path(output_dir, "phase10_summary.csv"))

knitr::kable(phase_10_summary, caption = "Phase 10 readiness summary")
```

Table 80: Phase 10 readiness summary

checkpoint	status
Readiness checklist saved	TRUE
Report outline saved	TRUE
Limitations table saved	TRUE
Research-question coverage saved	TRUE
Submission manifest saved	TRUE

```r
selected_jaccard_value <- suppressWarnings(as.numeric(selected_jaccard))

phase_10_sanity_audit <- data.table(
  check = c(
```

```r
    "Scenario names are unique",
    "Scenario labels have no gaps",
    "Spatial labels have no gaps",
    "Selected scenario stability is at least moderate",
    "Largest spatial cluster stays below 60% of all rows",
    "DBSCAN cluster-count target met",
    "DBSCAN noise target met"
  ),
  status = c(
    uniqueN(scenario_final_names$final_name) == nrow(scenario_final_names),
    label_join_check[metric == "Scenario label NAs", value][[1]] == 0,
    label_join_check[metric == "Spatial label NAs", value][[1]] == 0,
    !is.na(selected_jaccard_value) && selected_jaccard_value >= 0.4,
    largest_non_noise_share_pct <= phase7_max_cluster_share_cap,
    uniqueN(df_spatial_clusters$spatial_cluster[df_spatial_clusters$spatial_cluster > 0]) >= 3 &
      uniqueN(df_spatial_clusters$spatial_cluster[df_spatial_clusters$spatial_cluster > 0]) <= 15,
    round(100 * mean(df_spatial_clusters$spatial_cluster == 0), 2) >= 5 &
      round(100 * mean(df_spatial_clusters$spatial_cluster == 0), 2) <= 30
  ),
  note = c(
    "Final scenario names should be distinct for presentation and discussion.",
    "Every row should carry a scenario label after the join.",
    "Every row should carry a spatial label after the join.",
    "A Jaccard score below 0.4 should be presented as exploratory rather than settled.",
    "This avoids one hotspot absorbing most of Long Island.",
    "The SRS target is between 3 and 15 non-noise clusters.",
    "The SRS target is between 5% and 30% noise."
  )
)

fwrite(phase_10_sanity_audit, file.path(output_dir, "phase10_sanity_audit.csv"))

phase_10_summary <- rbind(
  phase_10_summary,
  data.table(
    checkpoint = "Sanity audit saved",
    status = file.exists(file.path(output_dir, "phase10_sanity_audit.csv"))
  )
)
fwrite(phase_10_summary, file.path(output_dir, "phase10_summary.csv"))

knitr::kable(phase_10_sanity_audit, caption = "Phase 10 sanity audit")
```

Table 81: Phase 10 sanity audit

check	status	note
Scenario names are unique	TRUE	Final scenario names should be distinct for presentation and discussion.
Scenario labels have no gaps	TRUE	Every row should carry a scenario label after the join.
Spatial labels have no gaps	TRUE	Every row should carry a spatial label after the join.
Selected scenario stability is at least moderate	FALSE	A Jaccard score below 0.4 should be presented as exploratory rather than settled.

check	status	note
Largest spatial cluster stays below 60% of all rows	TRUE	This avoids one hotspot absorbing most of Long Island.
DBSCAN cluster-count target met	TRUE	The SRS target is between 3 and 15 non-noise clusters.
DBSCAN noise target met	TRUE	The SRS target is between 5% and 30% noise.

14 Notes for the next phases

- Phase 2 should stay read-only and focus on profiling the cleaned dataset.
- Phase 3 now derives the main engineered features needed for the scenario and spatial matrices.
- Phase 4 now defines the actual scenario and spatial matrices for clustering.
- Phase 5 now sets up the candidate k-prototypes runs and their core diagnostics.
- Phase 6 now validates the candidate solutions, checks stability, and assigns a selected k.
- Phase 7 now sets DBSCAN parameters, creates the spatial hotspot solution, and assigns corridor-style labels.
- Phase 8 now joins cluster labels back to the working data and builds the interpretation tables.
- **Phase 8B (v3.0)** trains a Random Forest on the cluster labels and computes SHAP/Shapley feature importance. This bridges unsupervised clustering with supervised interpretation — a published technique for "post-hoc cluster explanation."
- **Phase 8C (v3.0)** provides structured experiment logging in JSON format, capturing all parameters, quality metrics, and artifact paths. This serves the same role as MLflow experiment tracking but runs natively in R without a Python backend.
- Phase 9 now exports the report-ready figures (including SHAP plots), tables, and labeled dataset.
- Phase 10 now tracks final report and submission readiness.
- `run_profile = "consensus_16gb"` is the current default in this file.
- `run_profile = "local_safe"` is the quickest laptop-friendly option.
- `run_profile = "consensus_16gb"` repeats sampled Phase 6 checks and aggregates them into a consensus choice for 16 GB machines.
- `run_profile = "full_srs"` is reserved for a stronger machine that can support the full Phase 6 diagnostics.
- The current default uses repeated sampled validation and repeated sampled stability checks to stay within 16 GB memory limits.
- Interactive HTML exports are optional and now turn on automatically when the needed packages are available.
- The original full-data Phase 6 validation and stability calls are archived in `eval=FALSE` chunks for later reruns.
- Phase 3 and later should write tables, figures, and exports into `local_artifacts/output/`.
- No `.RData` workspace files are needed for this workflow.
- **v3.0 new dependencies:** `ranger` (Random Forest), `iml` (model-agnostic Shapley values), `jsonlite` (experiment logging). All are on CRAN.

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.4.1
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versi
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK v
##
## locale:
## [1] en_IN/en_IN/en_IN/C/en_IN/en_IN
```

```
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
##  [1] parallel  splines   stats4    grid      tools     compiler  stats
##  [8] graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] pkgconfig_2.0.3      zoo_1.8-15           xfun_0.56
##  [4] checkmate_2.3.4      flexmix_2.3-20       Rcpp_1.1.1
##  [7] uuid_1.2-2           class_7.3-23         leaps_3.2
## [10] backports_1.5.0      lattice_0.22-7       evaluate_1.0.5
## [13] R6_2.6.1             htmltools_0.5.9      rappdirs_0.3.4
## [16] pillar_1.11.1        tibble_3.3.1         gtable_0.3.6
## [19] stringi_1.8.7        DT_0.34.0            codetools_0.2-20
## [22] DEoptimR_1.1-4       parallelly_1.47.0    glue_1.8.0
## [25] units_1.0-1          diptest_0.77-2       FactoMineR_2.13
## [28] listenv_0.10.1       sandwich_3.1-1       Matrix_1.7-4
## [31] vctrs_0.7.1          modeltools_0.2-24    stringr_1.6.0
## [34] DBI_1.3.0            proxy_0.4-29         httr_1.4.8
## [37] robustbase_0.99-7    future.apply_1.20.2  otel_0.2.0
## [40] generics_0.1.4       cli_3.6.5            flashClust_1.1-4
## [43] prabclus_2.3-5       scales_1.4.0         emmeans_2.0.2
## [46] globals_0.19.1       e1071_1.7-17         nnet_7.3-20
## [49] Metrics_0.1.4        withr_3.0.2          multcomp_1.4-30
## [52] mvtnorm_1.3-6        KernSmooth_2.23-26   RColorBrewer_1.1-3
## [55] scatterplot3d_0.3-45 mclust_6.1.2         classInt_0.4-11
## [58] htmlwidgets_1.6.4    labeling_0.4.3       yaml_2.3.12
## [61] rlang_1.1.7          kernlab_0.9-33       magrittr_2.0.4
## [64] multcompView_0.1-11  lifecycle_1.0.5      estimability_1.5.1
## [67] timechange_0.4.0     digest_0.6.39        TH.data_1.1-5
## [70] MASS_7.3-65          survival_3.8-3       combinat_0.0-8
## [73] fastmap_1.2.0        S7_0.2.1             farver_2.1.2
## [76] viridisLite_0.4.3    tidyselect_1.2.1     future_1.70.0
## [79] tigris_2.2.1         sf_1.1-0             jsonlite_2.0.0
## [82] iml_0.11.4           ranger_0.18.0        rmarkdown_2.30
## [85] knitr_1.51           factoextra_2.0.0     geosphere_1.6-8
## [88] ggrepel_0.9.8        ggplot2_4.0.2        fpc_2.2-14
## [91] cluster_2.1.8.2      dbscan_1.2.4         clustMixType_0.4-2
## [94] lubridate_1.9.5      dplyr_1.2.0          data.table_1.18.2.1
```